

研 究 生 毕 业 论 文

(申 请 硕 士 学 位)

论 文 题 目 _____ 面向广告与商业化场景的

_____ 内容应用与管理平台的设计与实现

作 者 姓 名 _____

院 系 _____

专 业 名 称 _____

指 导 教 师 _____

2026 年 4 月 17 日

学 号:

论文答辩日期: XXXX 年 XX 月 XX 日

指 导 教 师:

(签字)

Design and Implementation of a Content Application and Management Platform for Advertising and Commercial Scenarios

by

Supervised by

A dissertation submitted to
the graduate school of
in partial fulfilment of the requirements for the degree of
MASTER
in

April 17, 2026

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目： 面向广告与商业化场景的
内容应用与管理平台的设计与实现
专业 2024 级硕士生姓名：
指导教师（姓名、职称）：

摘 要

随着互联网广告与商业化业务的快速发展，广告产品形态、业务规则及运营策略持续演进，围绕投放、产品说明、业务规范与运营实践等沉淀了规模庞大、类型多样的专业内容资产。这类内容在业务培训、运营决策与客户支持等场景中具有重要价值。然而，随着业务线与内容规模持续增长，现有内容平台逐渐暴露出内容分散、跨场景复用困难、维护成本高以及质量难以评估与治理等问题，已难以满足广告与商业化场景的实际需求。

针对上述问题，本文结合企业内部广告与商业化业务的应用场景，设计并实现了一套面向广告与商业化场景的内容应用与管理平台。平台以内容全生命周期为主线，对内容建模、组织、应用与治理进行系统化设计，旨在提升内容资产的统一管理能力、应用效果与质量可控性。

在内容管理方面，平台提出了一种基于空间与应用分层的内容组织模型，实现对不同广告业务线及其应用场景的隔离与统一管理，并支持文章、问答、课程和案例等多种内容形态的规范化管理。结合类目树、标签体系、权限控制与审批流程，平台有效降低了内容维护成本，提升了内容资产的复用性与一致性。在内容应用方面，平台在支持传统页面化内容展示的基础上，引入基于检索增强生成（Retrieval-Augmented Generation, RAG）的智能问答能力，通过内容切片、向量索引、多通道召回与重排等技术手段，为上游智能应用提供高相关性的内容支持，拓展了内容资产的使用方式。在内容治理方面，针对内容规模扩大带来的质量问题，本文构建了一种围绕重复度、歧义度与覆盖度三类问题的内容治理机制，结合向量检索与大语言模型实现对内容质量的自动检测，并通过线上化治理任务实现内容治理的闭环处理。此外，平台通过数据洞察模块对内容使用效果和治理结果进行统计分析，为内容优化与治理策略调整提供数据支持。

实际应用结果表明，本文所设计和实现的平台在广告与商业化业务场景中有效提升了内容管理效率与应用效果，降低了内容治理的人力成本，对广告与商业化场景下内容平台的建设具有一定的工程实践价值和参考意义。

关键词： 广告与商业化内容；统一内容模型；检索增强生成（RAG）；内容治理闭环；数据洞察

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Design and Implementation of a Content Application
and Management Platform for Advertising and Commercial Scenarios
SPECIALIZATION: _____
POSTGRADUATE: _____
MENTOR: _____

Abstract

With the rapid growth of Internet advertising and commercialization businesses, product forms, business rules, and operational strategies continue to evolve. A large amount of professional content has been accumulated around ad delivery, product documentation, business specifications, and operational practices, supporting scenarios such as internal training, operational decision-making, and customer service. However, as business lines and content scale up, existing content platforms increasingly suffer from fragmented content sources, limited cross-scenario reuse, high maintenance costs, and insufficient capability for quality evaluation and governance.

To address these challenges, this thesis designs and implements a content application and management platform for advertising and commercial scenarios in an enterprise environment. Centered on the content lifecycle, the platform adopts a hierarchical organization model based on spaces and applications to achieve isolation and unified management across business lines, and supports multiple content types including articles, FAQs, courses, and cases.

For content consumption, the platform provides both page-based browsing and search, and introduces retrieval-augmented generation (RAG) for intelligent question answering through content slicing, vector indexing, multi-channel retrieval, and reranking, enabling upstream intelligent applications to obtain highly relevant content support and broadening the ways in which content assets can be utilized. For content quality assurance, the platform establishes an automated governance mechanism targeting duplication, ambiguity, and coverage-related issues, combining vector retrieval with large language models and operationalizing the results as online governance tasks to achieve

closed-loop content governance. In addition, a data insight module aggregates usage and governance metrics to support iterative optimization of content and governance strategies.

The practical application results demonstrate that the platform effectively improves content management efficiency and application performance in advertising and commercial scenarios, while reducing the labor costs associated with content governance. This work provides useful engineering insights and a practical reference for building content platforms tailored to advertising and commercial scenarios.

keywords: advertising and commercial content, unified content model, retrieval-augmented generation (RAG), closed-loop content governance, data insight

目 录

中文摘要	i
英文摘要	iii
目 录	v
插图清单	vii
附表清单	ix
第一章 引言	1
1.1 项目背景	1
1.2 国内外发展现状及分析	2
1.3 本文主要工作	4
1.4 论文的组织结构	5
第二章 相关技术综述	7
2.1 消息队列 RocketMQ	7
2.2 全文搜索引擎 Elasticsearch	8
2.3 Embedding 与 Milvus	10
2.4 检索增强生成 RAG	11
2.5 本章小结	12
第三章 内容应用与管理平台的分析与设计	15
3.1 系统整体概述	15
3.2 内容应用与管理平台需求分析	17
3.2.1 用户角色	17
3.2.2 内容管理功能需求	18
3.2.3 内容应用功能需求	26
3.2.4 内容治理功能需求	28
3.2.5 数据洞察功能需求	31
3.2.6 系统非功能需求	34
3.3 系统整体设计	35
3.4 系统的模块设计	38

3.4.1 内容管理模块设计	38
3.4.2 内容应用模块设计	41
3.4.3 内容治理模块设计	43
3.4.4 数据洞察模块设计	45
3.5 数据库设计	47
3.6 本章小结	61
第四章 内容应用与管理平台的实现	63
4.1 内容管理模块的实现	63
4.1.1 内容组织体系	64
4.1.2 多形态内容管理	65
4.1.3 权限与审批准布	68
4.2 内容应用模块的实现	72
4.2.1 多形态内容前置处理与索引构建	73
4.2.2 页面化内容应用机制	75
4.2.3 多通道召回	77
4.2.4 智能问答流程实现	79
4.3 内容治理模块的实现	82
4.3.1 重复度与歧义度检测算法实现	84
4.3.2 覆盖度检测实现	86
4.3.3 治理任务线上化与闭环机制	87
4.4 数据洞察模块的实现	89
4.4.1 内容应用数据采集机制	90
4.4.2 会话分析与效果指标设计	91
4.4.3 会话聚类与主题总结	92
4.5 本章小结	94
第五章 内容应用与管理平台测试	95
5.1 系统测试环境	95
5.2 单元测试	96
5.3 功能测试	96
5.3.1 内容管理功能测试	96
5.3.2 内容应用功能测试	97
5.3.3 内容治理与数据洞察功能测试	98

5.4 本章小结	99
第六章 总结与展望	101
6.1 总结	101
6.2 展望	102
版权及论文原创性说明	103
《学位论文出版授权书》	105

插图清单

2-1 RocketMQ 架构图	8
2-2 Elasticsearch 倒排索引示意图	9
3-1 内容管理用例图	19
3-2 内容应用用例图	26
3-3 内容治理用例图	29
3-4 数据洞察用例图	32
3-5 系统架构图	35
3-6 逻辑视图	36
3-7 开发视图	37
3-8 物理视图	38
3-9 空间与应用管理时序图	39
3-10 内容状态机图	40
3-11 内容发布流程时序图	41
3-12 索引构建流程图	42
3-13 智能问答流程图	43
3-14 重复度与歧义度模型检测流程示意图	44
3-15 质量治理任务线上化流程示意图	45
3-16 数据采集与指标生成链路示意图	46
3-17 数据库 E-R 图	47
4-1 应用管理效果图	65
4-2 内容创建与版本管理关键代码	66
4-3 文章列表效果图	67
4-4 文章创编效果图	68
4-5 content_auth 策略匹配关键代码	70
4-6 发布申请效果图	71
4-7 审批通过与发布事件触发关键代码	72

4-8 切片构建关键代码	74
4-9 索引构建关键代码	75
4-10 典型前台界面效果图	76
4-11 页面化内容展示关键代码	77
4-12 重排关键代码	79
4-13 智能问答效果图	80
4-14 问题改写提示词模板	81
4-15 RAG 召回与重排关键代码	81
4-16 生成回答提示词模板	82
4-17 智能问答关键代码	82
4-18 重复度与歧义度判别提示词模板	85
4-19 重复度与歧义度判别关键代码	85
4-20 重复度与歧义度检测关键代码	86
4-21 覆盖度判别提示词模板	87
4-22 治理任务线上化效果图	88
4-23 治理任务主动复检与巡检关键代码	89
4-24 会话分析效果图	92
4-25 会话聚类关键代码	93
4-26 聚类主题总结提示词模板	93

附表清单

3-1 用户角色表	18
3-2 创建空间用例描述表	20
3-3 创建应用用例描述表	20
3-4 创建类目用例描述表	21
3-5 创建标签用例描述表	21
3-6 创建内容用例描述表	22
3-7 编辑内容用例描述表	22
3-8 查看内容详情用例描述表	23
3-9 成员角色分配用例描述表	23
3-10 权限策略配置用例描述表	24
3-11 提交审核用例描述表	24
3-12 内容审批用例描述表	25
3-13 内容上线用例描述表	25
3-14 内容展示用例描述表	27
3-15 内容检索用例描述表	28
3-16 智能问答用例描述表	28
3-17 质量检测用例描述表	30
3-18 治理任务创建用例描述表	30
3-19 治理任务处理用例描述表	31
3-20 页面数据监控用例描述表	32
3-21 会话维度分析用例描述表	33
3-22 内容维度分析用例描述表	33
3-23 会话明细用例描述表	34
3-24 RBAC 权限矩阵表	40
3-25 空间表 (space)	48
3-26 应用表 (application)	48
3-27 内容表 (content)	49

3-28 知识内容表 (knowledge)	49
3-29 问答内容表 (faq)	50
3-30 课程内容表 (course)	50
3-31 课节表 (course_lesson)	50
3-32 案例详情集合 (MongoDB: case_detail)	51
3-33 标签表 (tag)	51
3-34 内容-标签关联表 (content_tag_ref)	52
3-35 类目表 (category)	52
3-36 内容-类目关联表 (content_category_ref)	53
3-37 用户表 (user)	53
3-38 角色表 (role)	53
3-39 权限表 (permission)	54
3-40 角色-权限关联表 (role_permission)	54
3-41 用户-角色绑定表 (user_role_binding)	54
3-42 内容权限表 (content_auth)	55
3-43 内容审批表 (content_approval)	55
3-44 主任务表 (govern_main_task)	56
3-45 重复歧义任务表 (govern_dup_amb_task)	57
3-46 覆盖度任务表 (govern_coverage_task)	57
3-47 事件明细表 (ODS: di_event_ods)	58
3-48 页面指标宽表 (DWS: di_page_metric_d)	58
3-49 内容指标宽表 (DWS: di_content_metric_d)	59
3-50 会话聚类指标宽表 (DWS: di_session_cluster_d)	60
3-51 会话明细表 (DWD: di_session_detail)	60
4-1 内容管理模块接口表	63
4-2 content_auth 动作码映射示例	69
4-3 内容应用模块接口表	72
4-4 RAG 召回关键参数	78
4-5 内容治理模块接口表	83
4-6 治理任务数据表 (核心字段)	83
4-7 治理任务状态口径示例	88
4-8 数据洞察模块接口表	90

附表清单	xiii
4-9 事件明细核心字段设计	91
4-10 ext 字段示例（典型事件）	91
5-1 测试环境描述表	95
5-2 创建空间/应用测试用例	97
5-3 内容提审与审批发布测试用例	97
5-4 内容检索测试用例	97
5-5 智能问答测试用例	98
5-6 重复/歧义治理测试用例	98
5-7 覆盖度治理测试用例	99
5-8 数据洞察链路测试用例	99

第一章 引言

1.1 项目背景

随着互联网广告与商业化业务的持续发展，广告产品形态、投放策略及业务规则不断演进，相关业务体系呈现出高度复杂化和专业化特征。在广告投放、产品能力说明、业务规范制定以及运营实践过程中，逐步沉淀了大量与广告与商业化业务密切相关的专业内容资产。这些内容涵盖业务规则解读、产品功能说明、操作流程指导以及典型业务案例等，是支撑广告与商业化业务运行和决策的重要基础^{[2][3]}。

随着业务规模扩大和业务线数量增加，广告与商业化内容在数量和类型上呈现持续增长趋势。内容逐步分散于不同业务线和系统中，缺乏统一的组织模型和管理规范，导致内容检索效率降低，内容维护高度依赖人工经验，难以形成系统化、可持续的内容管理体系^{[2][3]}。统一内容策略强调从内容粒度、复用关系与生命周期角度建立一致的组织方式，以降低跨场景维护成本^[2]。同时，围绕内容复用与组件化管理的研究强调以更细粒度的内容单元提升维护效率与跨场景复用能力^[2]。

在内容规模不断扩大的同时，内容的应用方式也在持续演进。传统依赖人工检索和页面浏览的内容使用模式，已难以满足广告与商业化业务对效率和准确性的要求。随着检索增强生成和智能问答等技术在业务系统中的逐步应用，内容的使用方式正从“人工查找”向“智能获取”转变^{[2][3]}。这一变化对内容的结构化程度、可检索性以及一致性提出了更高要求，同时也显著提升了内容在业务决策和问题处理过程中的使用频率。

随着内容被更广泛、更高频地应用，内容质量问题逐渐凸显。一方面，部分内容在长期演进过程中形成重复和冗余，降低了内容体系的整体可用性；另一方面，不同内容在描述同一业务问题时可能存在表述不一致或语义歧义，在智能问答等自动化应用场景下容易被进一步放大，影响内容理解的准确性。此外，由于缺乏系统化分析和评估手段，内容体系在部分业务领域存在覆盖不足的问题。随着内容规模和应用深度的提升，传统依赖人工维护的方式已难以有

效支撑内容质量的持续保障。

现有内容管理平台多以内容存储和页面展示为核心，主要提供基础的内容管理功能，在内容统一建模、多形态内容协同管理、智能化应用支持以及内容质量治理等方面能力有限，难以满足广告与商业化场景下内容规模大、应用频繁且质量要求高的实际需求。基于上述背景，本文设计并实现了一套面向广告与商业化场景的内容应用与管理平台，从内容全生命周期视角出发，对内容的生产、组织、应用和维护过程进行了系统性设计。平台在统一内容建模与组织方式的基础上，引入智能化的内容应用能力以及自动化与人工相结合的内容治理机制，实现了广告与商业化内容资产的统一管理、高效应用与质量可控，为复杂广告与商业化业务场景下的内容使用与持续维护提供了一种工程化解决方案。

1.2 国内外发展现状及分析

在内容管理与知识应用领域，国内外研究和工程实践主要围绕企业内容管理系统（Enterprise Content Management, ECM）和内容管理系统（Content Management System, CMS）展开。相关研究普遍关注内容的集中存储、统一建模、流程化管理以及权限控制等问题，通过引入内容生命周期管理和 workflows 机制，提高内容管理的规范性与可控性^{[2][1]}。此外，面向教育与企业信息化等场景的工程研究亦给出了基于开源企业内容管理系统的架构实践^{[2][1]}。

在工程实践层面，国外以 Microsoft 推出的 SharePoint、Atlassian 的 Confluence 等平台为代表，提供了面向组织级内容的统一管理与协作能力，支持内容版本控制、权限配置以及结构化组织^{[2][1]}。围绕 ECM 解决方案的实施路线图与落地挑战，已有研究指出，系统集成复杂度、治理边界划分以及实施阶段的流程适配是影响内容平台成效的重要因素^{[2][1]}。这类平台在内容管理和协同办公方面具备较高成熟度，但其核心使用模式仍以人工检索和页面浏览为主，对内容的深度应用和智能化利用支持相对有限。国内企业也普遍建设了面向内部知识沉淀的内容管理或知识平台，用于支撑业务培训和经验复用，但整体上仍以管理和展示为主要目标。

从研究脉络看，ECM/CMS 的发展不仅关注“统一存储与发布”，也逐步强调内容的复用性与组件化组织能力。相关综述研究指出，组件化内容管理通过将内容拆分为更细粒度的可复用单元，并以元数据、版本与流程对其进行管

理，从而提升跨产品、跨场景的复用效率并降低维护成本^{[2]1}。在复杂业务组织中，围绕内容治理责任划分与流程规范的讨论亦强调管理策略需要与组织协作机制协同推进，使内容质量控制从个体经验转向可持续的制度化运作^{[2]1}。

在内容应用层面，信息检索技术长期是内容平台实现“高效获取”的关键基础。以 Lucene/Elasticsearch 为代表的全文检索系统通过倒排索引、相关性排序与聚合分析，为关键词检索与结构化过滤提供了成熟的工程实现路径^{[2]1}。随着语义表示学习的发展，向量检索逐渐成为覆盖长尾问法与语义改写的重要补充，典型向量数据库系统在近似最近邻索引、标量过滤与分布式扩展方面形成了较为完善的系统化方案^{[2]1}。由此，现代内容应用往往呈现“稀疏检索 + 稠密检索”并行互补的发展趋势，为后续的智能问答与内容理解奠定基础。

随着业务复杂度和内容规模的持续提升，传统内容管理系统在内容应用层面的局限逐渐显现。近年来，随着大语言模型技术的发展，基于检索增强生成（Retrieval-Augmented Generation, RAG）的内容应用与智能问答技术成为研究热点。相关研究通过将外部知识检索与生成模型相结合，在一定程度上提升了知识密集型任务的准确性，并围绕检索策略、结果重排以及生成效果评估等方面展开了大量探索^{[2]1}。部分研究进一步将 RAG 技术应用于多模态文档和垂直领域场景，引入图片、表格等非文本内容的检索与理解能力，用于提升复杂业务问题的问答效果^{[2]1}。

从方法演进角度看，RAG 的研究重点已从“是否引入检索”转向“如何更有效地利用证据”。一方面，研究者探索通过重排与证据组织提升多文档证据的利用效率，并验证高质量重排对最终问答表现的增益^{[2]1}；另一方面，围绕大模型幻觉、引用一致性与可追溯性等问题的系统性研究也在推进，使得“回答可用性”不再仅以语义流畅为目标，而需兼顾事实一致性与引用可解释性^{[2]1}。

尽管 RAG 等智能内容应用技术在算法层面取得了显著进展，但现有研究多聚焦于模型能力和问答效果优化，通常假设知识来源具备较高质量和良好结构，较少关注内容在生产、维护和演进过程中的管理问题^{[2]1}。在实际业务场景中，内容往往来源多样、形态复杂且质量参差不齐，单纯依赖智能检索和生成模型，难以从根本上保障内容应用的稳定性和可靠性^{[2]1}。

与此同时，在内容质量与数据治理方向，已有研究从一致性、完整性、准确性以及治理流程等维度提出了较为系统的理论框架，强调通过指标化评估、流程化管理和工具化支撑提升内容与数据资产的可信度和可维护性^{[2]1}。这类研究在数据治理和企业信息管理领域具有较成熟的理论基础。然而，在具体工

程实践中，内容质量治理往往以离线分析或人工审查为主，与内容管理系统和智能应用系统相对独立，缺乏将质量检测结果直接反馈到内容管理和应用流程中的闭环机制，难以支撑内容规模持续增长和高频应用场景下的长期运营需求^{[2]1}。

进一步地，随着智能应用链路复杂度提升，内容一致性问题不仅表现为“重复冗余”，也可能体现为跨版本、跨口径的语义冲突与需求矛盾。相关研究从规则与形式化约束角度讨论了在大规模仓库中识别语义冲突的治理方法，也探索了利用大模型对矛盾进行自动检测与结构化输出的可行性^{[2]1}。这些方向为将“质量检测—任务化处理—复检回收”纳入内容平台主流程提供了方法参考。

综合来看，现有研究和系统在内容管理、智能应用以及内容治理等方面均取得了一定成果，但整体上仍呈现出相对割裂的发展状态。传统内容管理系统侧重内容管理但缺乏智能化应用能力，智能问答和 RAG 相关研究强调内容应用效果却忽视内容管理和治理的工程复杂性，而内容质量治理研究又难以与实际内容应用形成闭环。在广告与商业化场景下，内容规模大、更新频繁且应用方式多样，单一侧重某一方面的解决方案难以满足实际需求。因此，如何在统一内容管理的基础上，引入智能化内容应用能力，并通过系统化的内容治理机制保障内容质量，实现管理、应用与治理协同演进的内容平台，仍有待进一步研究和工程实践。

1.3 本文主要工作

广告与商业化内容在实际业务中具有规模大、更新频繁、内容形态多样以及应用场景复杂等特点，传统内容平台在统一管理、高效应用和质量保障等方面逐渐暴露出不足。为解决上述问题，本文在分析国内外内容管理、智能内容应用以及内容治理相关研究和工程实践的基础上，结合企业内部广告与商业化业务的实际需求，设计并实现了一套面向广告与商业化场景的内容应用与管理平台，对平台的整体架构与核心模块进行了系统性设计与实现。

本文所设计的平台以内容全生命周期管理为核心，对内容从创建、组织、应用到治理的全过程进行统一支撑，减少内容在不同系统之间的割裂状态，提高内容资产的质量可控性。围绕上述目标，本文的主要工作包括以下几个方面。

首先，在内容管理方面，设计并实现了一套统一的内容管理机制。平台通

过引入空间与应用的分层组织模型，对不同广告业务线及其应用场景进行隔离与管理，支持文章、问答、课程和案例等多种内容形态的统一建模与维护。同时，结合类目树、标签体系、权限控制和审批流程，实现内容从创建、维护到上线的规范化管理，降低内容维护成本，提高内容资产的一致性。

其次，在内容应用方面，平台在支持传统页面化内容展示的基础上，引入基于检索增强生成的智能问答能力，拓展内容资产的使用方式。针对不同内容形态的特点，设计了相应的内容切片、索引和召回策略，并通过多通道召回与重排机制提升内容匹配的相关性，为上游智能应用提供稳定、可靠的内容支撑，使内容能够在复杂业务场景中被高效获取和利用。

再次，在内容治理方面，针对内容规模扩大和应用频率提升带来的质量问题，构建了一套内容治理机制。平台围绕重复度、歧义度和覆盖度三个维度对内容质量进行分析，通过向量检索和大语言模型判别实现内容质量的自动检测，并将检测结果转化为线上化治理任务，指派相关人员进行处理，从而形成内容治理的闭环流程，提升内容体系的整体质量和可维护性。

最后，在数据分析与反馈方面，平台通过数据洞察模块对内容的使用情况和治理效果进行统计分析，从页面访问、用户会话和内容使用等多个维度对内容应用效果进行量化评估，为内容优化和治理策略调整提供数据支持，进一步支撑平台的持续演进和优化。

综上所述，本文通过对面向广告与商业化场景的内容应用与管理平台的设计与实现，探索了一种将内容管理、智能应用与内容治理进行一体化整合的工程化解决方案，为复杂业务场景下内容资产的高效利用和长期维护提供了实践参考。

1.4 论文的组织结构

本文围绕面向广告与商业化场景的内容应用与管理平台的设计与实现展开，全文共分为六章，各章节内容安排如下。

第一章为绪论，主要介绍了论文的研究背景和研究意义，分析了广告与商业化内容在实际业务场景下面临的管理、应用和质量治理问题，并对国内外相关研究和工程实践的发展现状进行了综述，最后概述了本文的主要研究内容和工作安排。

第二章为相关技术综述，主要介绍了本文系统设计与实现过程中所涉及的

关键技术和方法，包括内容管理相关技术、全文检索与向量检索技术、检索增强生成（RAG）技术以及大语言模型在智能问答中的应用等，为后续系统设计与实现提供技术基础。

第三章为平台需求分析与总体设计。该章节结合广告与商业化业务场景，对平台的功能需求和非功能需求进行了分析，在此基础上给出了平台的总体架构设计，并对各核心模块的设计思路进行了说明。

第四章为平台的详细设计与实现。该章节围绕内容管理、内容应用和内容治理等核心模块，对系统的关键功能设计和具体实现过程进行了详细阐述，并结合实际系统界面展示了平台的主要功能效果。

第五章为系统效果分析与评估。该章节通过对平台在实际业务场景中的运行情况进行分析，从内容管理效率、内容应用效果以及内容治理成效等多个维度对系统进行评估，验证平台设计的可行性和有效性。

第六章为总结与展望，对全文的研究工作进行了总结，分析了当前系统存在的不足，并对未来在功能扩展和技术优化等方面的研究方向进行了展望。

第二章 相关技术综述

2.1 消息队列 RocketMQ

Apache RocketMQ 是一种高性能、高可靠的分布式消息中间件，最初由阿里巴巴提出并开源，现已成为 Apache 顶级项目，广泛应用于异步通信、系统解耦和流量控制等业务场景。RocketMQ 采用发布—订阅（Publish-Subscribe）的消息模型，支持点对点和广播等多种消费方式，能够满足大规模分布式系统中对消息传递可靠性和稳定性的要求。

在消息组织方式上，RocketMQ 以主题（Topic）作为消息管理的基本单位，消息在 Broker 中按照队列（Queue）进行存储和分发。通过将同一主题下的消息分布到多个队列中，系统可以实现消息的并行写入和并行消费，从而提升整体吞吐能力。在消息存储方面，RocketMQ 采用顺序写磁盘的方式进行消息持久化，在保证可靠性的同时提高了写入效率。

在消息投递与消费机制方面，RocketMQ 支持生产者向指定主题发送消息，由消费者以订阅方式进行消费，并通过消费进度管理和投递确认机制保障消息的可达性。系统支持至少一次（At-Least-Once）的消息投递语义，并提供顺序消息和延迟消息等能力，使其能够适应对消息时序性和定时触发有要求的业务场景。

在系统架构层面，如图 2-1 所示，RocketMQ 由 NameServer、Broker、Producer 和 Consumer 等组件构成。NameServer 负责维护集群的路由信息，Broker 负责消息的存储与转发，Producer 和 Consumer 分别承担消息的生产与消费职责。该架构采用去中心化设计，支持 Broker 节点的动态扩展和故障转移，在节点异常情况下能够保证消息服务的连续性。

在分布式系统中，RocketMQ 常用于构建基于事件驱动的消息通信机制，通过消息队列对请求进行缓冲和调度，降低系统之间的直接依赖关系。同时，消息队列能够在高并发场景下对突发流量进行削峰填谷，并通过消费速率控制实现限速处理，从而提升整体系统的稳定性和可扩展性。

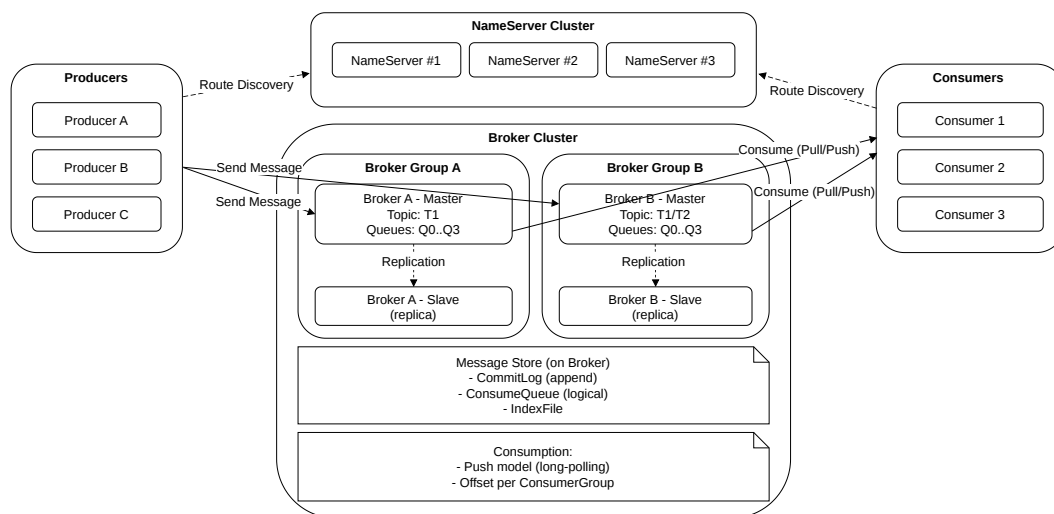


图 2-1: RocketMQ 架构图

2.2 全文搜索引擎 Elasticsearch

Elasticsearch 是一种基于 Lucene 的分布式搜索与分析引擎，面向文档（document）与字段（field）的数据模型，支持结构化、半结构化与非结构化内容的统一检索与聚合分析^[2]。其核心目标是在横向扩展的集群上，以近实时（near real-time）的方式提供高并发、低延迟的检索能力。

从体系结构上，集群由多个节点（node）构成，索引（index）是逻辑管理单元，索引被划分为若干主分片与副本分片（shard），分布在不同节点上以实现水平扩展与容错。写入采用分段（segment）追加与后台合并策略，结合刷新（refresh）机制实现近实时可见性；分布式检索采用“scatter - gather”模式在多分片间并发执行、汇总与重排结果。

在 Lucene 内核层面，文档写入后会形成多个不可变段（segment），段内包含倒排索引、存储字段（stored fields）、词典与统计信息等。段不可变的设计使得读取路径可以高度优化，而更新与删除则以追加与标记方式实现，并在后台通过段合并（merge）逐步回收删除标记与优化索引结构。近实时特性来自“写入缓冲区 → 刷新生成新段 → 搜索器可见”的流程：刷新并非强制落盘（flush），而是将内存中的索引结构转换为可检索的段，从而在吞吐与时效之间取得折中。

在文本索引方面，如图 2-2 所示，文档字段首先经过分析器（analyzer）管线处理：字符过滤（character filter）统一规范化，分词器（tokenizer）切分词

项，随后进行大小写、停用词、词干化或同义词等令牌过滤（token filter）。最终构建“词项 → 倒排列表 → 位置信息”的倒排索引结构，以支持关键词匹配、短语匹配与邻近度查询。

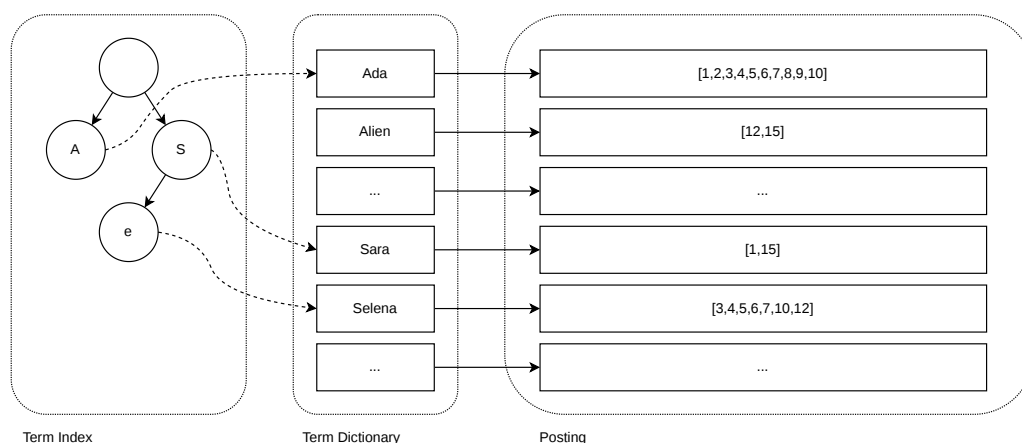


图 2-2: Elasticsearch 倒排索引示意图

在相关性计算方面，系统采用基于概率相关性的 **BM25** 打分，对词频、逆文档频率与字段长度进行归一化建模，兼顾高频项的收益递减与长文惩罚^[7]。查询表达使用 **DSL** 组合布尔、短语、范围、多字段匹配等算子，结果可基于高亮、分页与聚合（**aggregation**）进行呈现。除倒排索引外，列式存储的 **DocValues** 支持高效的排序、去重与聚合统计。

在查询执行上，分布式检索通常包含两个阶段：查询阶段（**query phase**）在每个相关分片上计算候选集合与局部 **TopK**，随后在协调节点汇总并执行全局合并；当需要返回文档详情、字段或高亮时，还会触发取回阶段（**fetch phase**）从对应分片读取 **stored fields** 或 **_source**。为降低跨分片开销，系统常在局部阶段尽量裁剪候选集合，并在全局阶段完成最终重排与分页。

聚合分析（**aggregation**）是 Elasticsearch 区别于传统检索系统的重要能力之一。通过 **terms**、**histogram**、**range** 等桶聚合（**bucket aggregation**）与 **sum**、**avg**、**percentiles** 等指标聚合（**metric aggregation**），系统能够在一次查询中同时完成“检索 + 统计”。其底层依赖 **DocValues** 的列式结构按字段扫描与分桶聚合，支持在大规模文档集合上进行近实时分析。

综上，Elasticsearch 通过“分布式分片—近实时索引—倒排结构—**BM25** 打分—聚合分析”的组合，实现了对大规模文档集合的高效关键词检索；其在内容检索、日志分析与监测告警等领域被广泛采用，并与稠密向量检索形成互

补^[2]。

2.3 Embedding 与 Milvus

Embedding 是将文本、图像等非结构化对象映射为稠密向量的一类表示学习方法，目标是在向量空间中保留语义相似性——语义相近的对象在空间中的距离更近。与基于关键词的稀疏表示相比，稠密向量能更好地涵盖同义改写与语义扩展，因此适用于语义召回与聚类等任务。

在表征层面，文本向量通常由预训练语言模型通过池化（如 CLS/平均池化）得到；相似度度量多采用余弦相似或内积。根据训练目标的不同，向量可以强调语义等价、主题相似或任务相关性，例如对比学习目标常用于增强“语义接近”的可分性。与词项级稀疏表示相比，稠密向量通过在连续空间中编码上下文信息，使得语义相近但词面差异较大的表达更容易被检索到。

在向量检索问题中，给定查询向量，需要在海量向量集合中返回相似度最高的 TopK 近邻。精确最近邻（Exact NN）在高维空间的复杂度与存储开销较高，因此近似最近邻（Approximate NN，ANN）成为主流路径：ANN 通过牺牲部分精确性换取显著的加速，并通过参数化控制召回率与时延之间的权衡。

Milvus 是面向向量数据的数据库系统，围绕索引构建、相似度检索与元数据过滤提供系统支持^[2]。其典型索引包括：基于聚类中心的 IVF（Inverted File）系列，用粗量化缩小检索候选；基于邻接图的 HNSW（Hierarchical Navigable Small World），以多层小世界图实现高效近邻搜索；以及结合产品量化（Product Quantization，PQ）的压缩索引，在牺牲部分精度的前提下降低存储与计算成本。

IVF 索引通常先对向量空间进行聚类，将向量分配到若干倒排桶中；检索时先选择与查询最接近的一小部分桶，再在桶内做局部搜索，从而显著减少候选规模。HNSW 则通过构建分层图结构，使搜索从稀疏高层开始逐步逼近，再在密集低层细化，具有较好的时延与召回表现。PQ 将高维向量分块并对每块进行量化编码，以更低的码长近似原向量，进而降低存储占用并加速相似度计算，但可能带来一定精度损失。

在系统层面，Milvus 通过服务端组件协同完成向量写入、索引构建与 TopK 检索，并支持与标量维度（如时间、类别）组合的过滤查询。面向带结构化过滤条件的向量检索场景，过滤式近似最近邻搜索的系统设计与性能权衡也逐渐

成为研究重点，这进一步说明向量数据库不仅要处理相似度计算，还需要协调过滤、索引与延迟之间的关系^{[2][3]}。

2.4 检索增强生成 RAG

检索增强生成（Retrieval-Augmented Generation, RAG）是一类将外部知识检索融入生成过程的模型化框架，其动机在于缓解大语言模型面对知识密集型任务时的幻觉与时效性不足问题^{[2][3]}。RAG 通过为生成模型提供与输入相关的证据文档，使生成不再仅依赖参数内隐知识，从而在开放域问答、企业知识问答与面向业务规则的辅助决策等场景中表现出较强的可控性。从基本机制看，RAG 将“参数化知识”与“非参数化知识”结合起来：前者由预训练语言模型内化在模型参数中，负责语言组织、推理模式与回答表达；后者则由检索系统从外部知识库中动态提供，负责补充事实、时效信息与领域细节。二者结合后，模型不再试图仅凭参数记忆回答所有问题，而是通过“先找证据，再据此生成”的方式完成回答，这也是 RAG 区别于纯生成式问答的核心所在^{[2][3]}。

典型流程包括：问题表示与检索、候选融合与重排、证据组织与条件生成。首先，系统需要将用户问题转换为适合检索的表达形式，这一过程既可能是基于关键词的直接检索，也可能涉及问题改写、查询扩展与多路召回；其次，系统从稀疏检索、稠密检索或混合检索通道中获得候选证据，并通过去重、融合与重排提升候选集合质量；最后，系统将问题与筛选后的证据共同送入生成模型，在约束上下文窗口的前提下输出回答文本，并可进一步附带引用、出处或跳转信息。在检索侧，RAG 并不局限于单一召回机制。稀疏检索擅长命中显式术语与精确表达，适用于制度名词、产品名与规则编号等场景；稠密检索则更适合处理同义改写、口语化表达与跨表述方式的语义匹配。实际系统中，二者往往并行使用，再通过重排模型或启发式策略进行融合，以在召回率、准确率与延迟之间取得平衡。这种“混合召回 + 重排”的结构也逐渐成为工程实践中的主流方案^{[2][3]}。在生成侧，证据组织方式直接影响回答质量。若证据片段过长，会挤占上下文窗口并引入无关噪声；若片段过短，则可能破坏语义完整性，导致模型无法形成足够的上下文理解。因此，知识切片粒度、相邻片段合并策略、证据排序方式以及输入模板设计，都会显著影响最终生成效果。除“问题 + 若干片段”的直接拼接方式外，也有研究关注先对证据做聚合、摘要或压缩，再进入生成阶段，以提高证据利用效率^{[2][3]}。

在模型融合形态上，既有“检索器 + 生成器”的级联方式，也有将多文档证据在解码阶段进行融合的方案。例如 **Fusion-in-Decoder (FiD)** 通过对多段证据分别编码、在解码时融合信息，以增强多证据推理能力^[21]。此外，也存在将重排任务与生成任务统一到同一模型或同一训练目标的研究方向，以期提升证据利用效率与最终回答质量^[21]。这说明 **RAG** 的研究重点已从“是否接入检索”逐步转向“如何高效利用检索结果”。从系统范式看，**RAG** 的关键挑战集中在“检索质量—证据组织—生成一致性”的耦合关系上：检索阶段的噪声、证据重复、片段粒度过细或过粗都会传导到生成阶段；而生成模型的上下文窗口有限，导致证据选择与压缩成为不可回避的环节。与此同时，若检索返回的证据彼此冲突、时间版本不一致或适用范围不同，生成模型还可能在看似“有依据”的情况下产出事实不一致的回答。相应地，研究工作围绕证据选择策略、长上下文建模、引用与事实一致性评价，以及回答可验证性等方面持续推进^[22]。

面向工程落地时，**RAG** 不仅是模型问题，也是一类系统协同问题。其效果依赖知识库构建、索引更新、权限过滤、查询改写、召回融合、提示词模板与引用解析等多个环节共同作用。特别是在企业知识场景中，知识内容持续变化、文档形态多样、访问权限细粒度且问法高度口语化，这要求系统在保证召回相关性的同时，还要满足时效性、可追溯性与安全合规等约束。因此，**RAG** 的实现通常需要与内容管理、索引构建和权限控制机制深度耦合，而不能简单理解为“向大模型前追加若干文档片段”。总体而言，**RAG** 将检索的可控性与生成的表达能力结合，形成“检索—重排—生成”的体系化范式；其研究与应用重点围绕检索与生成的协同、证据利用效率、多证据推理能力以及回答可追溯性等方向持续演进^[22]。对于本文所研究的内容应用与管理平台而言，**RAG** 既是实现智能问答能力的技术基础，也是连接内容管理、索引构建与前台问答体验的关键桥梁。

2.5 本章小结

本章围绕平台最关键的技术基座展开综述：消息中间件 **RocketMQ**、关键词检索引擎 **Elasticsearch**、**Embedding** 与 **Milvus**，以及检索增强生成（**RAG**）。其中，**Elasticsearch** 提供关键词检索与聚合分析能力，支撑页面化展示与结构化过滤；**Embedding** 负责将非结构化内容映射为稠密向量，**Milvus** 则围绕向量索引构建、近似最近邻检索与标量过滤提供系统支撑，二者共同构成语义召回

能力；RAG 将关键词检索与语义检索结果进一步组织并注入生成模型，结合证据引用提升回答的可解释性与可控性；RocketMQ 提供事件驱动的异步解耦能力，支撑索引构建与数据回收链路。以上技术共同构成了“生产—发布—应用—治理—评估”的核心支撑。

第三章 内容应用与管理平台的分析与设计

3.1 系统整体概述

面向广告与商业化场景的内容应用与管理平台是一个以内容资产全生命周期为核心的综合性平台系统，目标是在内容规模持续增长、业务线与应用场景不断扩展的背景下，实现内容的统一建模、规范化管理、高效应用以及质量可控。平台以“内容资产化”和“能力平台化”为设计导向，将内容从分散的业务沉淀与人工维护过程，转化为可组织、可追踪、可治理的系统化资产，并通过统一接口对上游业务系统、智能助手与运营工具提供稳定服务。

从业务视角看，平台服务于广告与商业化生态中的多类使用场景，包括业务人员的知识获取与运营决策支持、面向外部的产品说明与规范指引，以及客户支持与智能问答等高频咨询场景。上述场景对内容提出了共同要求：一是内容来源多、形态多样；二是内容更新频繁，依赖人工维护易产生不一致与过期；三是内容消费方式从“页面浏览”逐步演进到“搜索与问答驱动”，要求更高的检索质量与可解释性；四是内容质量问题需要可度量、可触发、可闭环，否则难以支撑规模化运营^{[2][1]}。因此，本平台在总体设计上采用“管理与应用协同、治理闭环驱动”的思路，以内容组织与权限体系为基础，以检索与智能问答能力为牵引，以质量治理和数据洞察为反馈闭环，实现内容体系的持续演进。

从系统关系看，平台在企业信息系统中处于“内容中台”的位置，与多类上下游系统形成协作链路。上游系统主要包括：业务线既有内容沉淀系统（产品规范、运营知识库等），用于提供内容来源与变更输入；统一身份与权限系统（如 SSO/IAM），用于提供用户身份、组织关系与权限上下文；对外服务入口系统（如内容门户、管理后台与智能助手），用于发起内容查询、检索与问答调用。下游系统主要包括：检索与索引平台（全文检索与向量检索服务），用于承载内容的检索与语义召回能力；消息与任务调度系统，用于承载内容发布后的异步更新与回收；数据仓库与 BI 分析系统，用于沉淀埋点明细与聚合

指标，支撑运营分析与治理评估；大模型服务平台，用于承载问答生成与治理判别等智能能力。通过与上述系统的接口协作，平台在“内容生产—发布—检索/问答—治理—评估”的链路中实现口径一致与数据可追溯。

从系统视角看，平台整体由内容管理、内容应用、内容治理与数据洞察四个核心域组成，并通过统一的数据模型、权限体系与事件机制实现协同。内容管理域负责内容的生产与维护，包括空间与应用的分层组织、内容形态管理（文章、问答、课程、案例等）、权限控制与审批发布等能力。该域的核心作用是将多来源、多形态内容纳入统一规范，形成可控的内容资产底座，并为后续检索、问答与治理提供一致的数据输入和业务语义^[2]。

内容应用域面向内容消费与智能化使用需求，提供两类能力：其一是面向页面化场景的内容展示与检索能力，通过统一内容接口支撑列表、目录导航、详情页、关键词搜索、标签聚合等常见页面组件；其二是面向智能问答场景的检索增强生成能力，通过内容切片、索引构建、多通道召回与重排等手段，为上游智能应用提供高相关性、可追溯的内容依据^[2]。与传统“内容库+搜索”不同，平台在应用域强调“可控检索”和“结果可解释”，即召回结果需可定位到具体内容片段与来源，以便在治理与迭代中形成明确反馈路径。

内容治理域面向内容规模化带来的质量问题，构建“自动检测—任务化分发—人工处理—复检回收”的闭环机制。治理对象主要包括重复度、歧义度与覆盖度三类典型问题：重复度治理用于识别内容冗余与重复维护；歧义度治理用于检测同一问题下的内容冲突、不一致或表述歧义；覆盖度治理用于从知识体系与用户需求侧识别内容缺口与薄弱点。平台将治理结果以标准化的治理任务形式落地，包括问题定位、影响范围、建议处理方式与责任归属，配合任务状态流转，使治理过程可追踪、可度量、可持续运营；其中，对冲突与不一致信息的处理通常需要结合可定位的证据与规则化策略进行收敛^[2]，从而避免治理仅停留在离线分析或一次性专项。

数据洞察域承担平台运行效果与治理成效的量化评估职责，通过对页面访问、用户会话、检索与问答链路日志、内容使用情况等数据进行统计分析，输出面向运营与产品迭代的指标体系。该域既用于评价内容应用效果（如内容被召回次数、生成引用次数、转人工率等），也用于反哺治理策略（如高频问题聚类、低命中内容识别、质量问题分布等），从而形成“应用驱动生产、数据反哺治理”的闭环运营机制。

在关键流程层面，平台围绕内容全生命周期形成若干核心链路：在内容生

产链路中，内容经创建与编辑后进入审批与发布流程，已发布状态内容作为对外可用的内容资产进入索引与应用链路；在索引构建链路中，平台对不同内容形态进行标准化处理（如切片、结构化字段抽取、元数据归一）并构建全文索引与向量索引，以支撑检索与问答；在内容消费链路中，用户通过页面浏览、搜索或智能问答触发内容召回，平台返回内容或为生成模型提供依据；在质量治理链路中，平台周期性或事件驱动地对内容进行质量检测，将发现的问题沉淀为治理任务并推动处理，处理结果再反向影响内容库与索引；在评估反馈链路中，数据洞察持续沉淀关键指标与行为数据，为内容生产策略、检索策略与治理规则提供优化依据。

总体而言，本平台以统一内容资产底座为核心，将内容管理、内容应用、内容治理与数据洞察四个域有机整合：内容管理域负责内容的生产与规范化，为后续应用与治理提供一致的数据输入；内容应用域面向不同消费场景提供展示、检索与问答能力，实现内容价值的直接输出；内容治理域通过质量检测与任务化闭环，保障内容体系的持续演进；数据洞察域则通过效果评估与问题定位，形成“应用驱动生产、数据反哺治理”的反馈机制。四个域相互协同、闭环迭代，既覆盖传统内容平台的基础管理需求，也面向智能化内容消费方式提供支撑，为广告与商业化业务在知识沉淀、信息一致性以及智能化服务能力方面提供稳定、可扩展的工程化方案。

3.2 内容应用与管理平台需求分析

3.2.1 用户角色

本平台的用户角色如表3-1所示，主要分为四种，分别是：平台负责人、业务负责人、业务运营和客户。平台负责人负责整个平台层面的配置与运维管理；业务负责人负责某一业务空间下内容体系的整体管理与维护；业务运营是平台内容生产和日常运营的主要执行角色；客户主要作为内容的消费方，通过页面或智能服务获取平台提供的内容支持。

除上述用户角色外，平台还需要与上游智能应用（如智能助手）进行接口对接。该类调用方通常以系统身份访问平台的内容检索与问答能力，其行为数据将被纳入会话分析与效果评估范围。

表 3-1: 用户角色表

用户角色	描述
平台负责人	平台的最高管理角色，负责整个平台的基础配置、空间管理及全局权限控制，保障平台稳定运行。
业务负责人	负责某一业务线或业务空间下内容体系的管理与运维，对内容结构、应用配置及业务权限进行整体把控。
业务运营	负责平台内容的创建、编辑和维护等日常运营工作，并在权限范围内查看相关内容使用数据。
客户	平台内容的消费角色，包括广告主、代理商等，通过页面浏览、搜索或智能助手等方式使用平台内容。

3.2.2 内容管理功能需求

内容管理功能的用例图如图 3-1 所示。内容管理模块作为平台的核心模块，主要负责内容资产的全生命周期管理，包括内容的生产、维护、组织、权限控制以及发布上线等环节。该模块旨在将分散在不同业务线和系统中的内容进行统一纳管，通过标准化的模型和流程，确保内容在流转过程中的一致性、安全性和可控性。

内容管理模块的功能需求可按“内容组织体系—多形态内容管理—权限与审批发布”三条主链路展开：内容组织体系用于建立业务归属与内容组织方式，多形态内容管理用于支撑内容的创建维护与详情读取，权限与审批发布用于约束可访问范围并生成对外发布口径。为保证论述结构清晰，本节按上述顺序给出需求说明，并列举若干代表性用例。

(1) 内容组织体系。平台采用“空间—应用—类目—标签”四级组织体系，用于统一表达业务归属、消费场景与内容组织方式。空间用于承载业务归属与成员范围，应用用于定义具体内容消费场景与配置集合（如可用内容形态、类目树与标签体系等）。围绕该组织体系，平台需要提供空间与应用的创建配置能力，并支持类目与标签作为内容组织与运营配置的基础元素，以满足多业务线、多入口条件下的内容聚合与筛选需求。

创建空间的用例描述如表 3-2 所示。空间作为平台的顶层组织边界，用于承载业务线归属、成员范围与权限隔离的基础语义。通过创建空间，平台为后续应用配置、内容生产与治理提供统一的归属与作用域，并完成默认角色、默认权限策略等初始配置的生成。

创建应用的用例描述如表 3-3 所示。应用用于刻画空间下的具体内容消费场景或业务域配置集合，例如门户展示、帮助中心或智能问答入口等。通过创

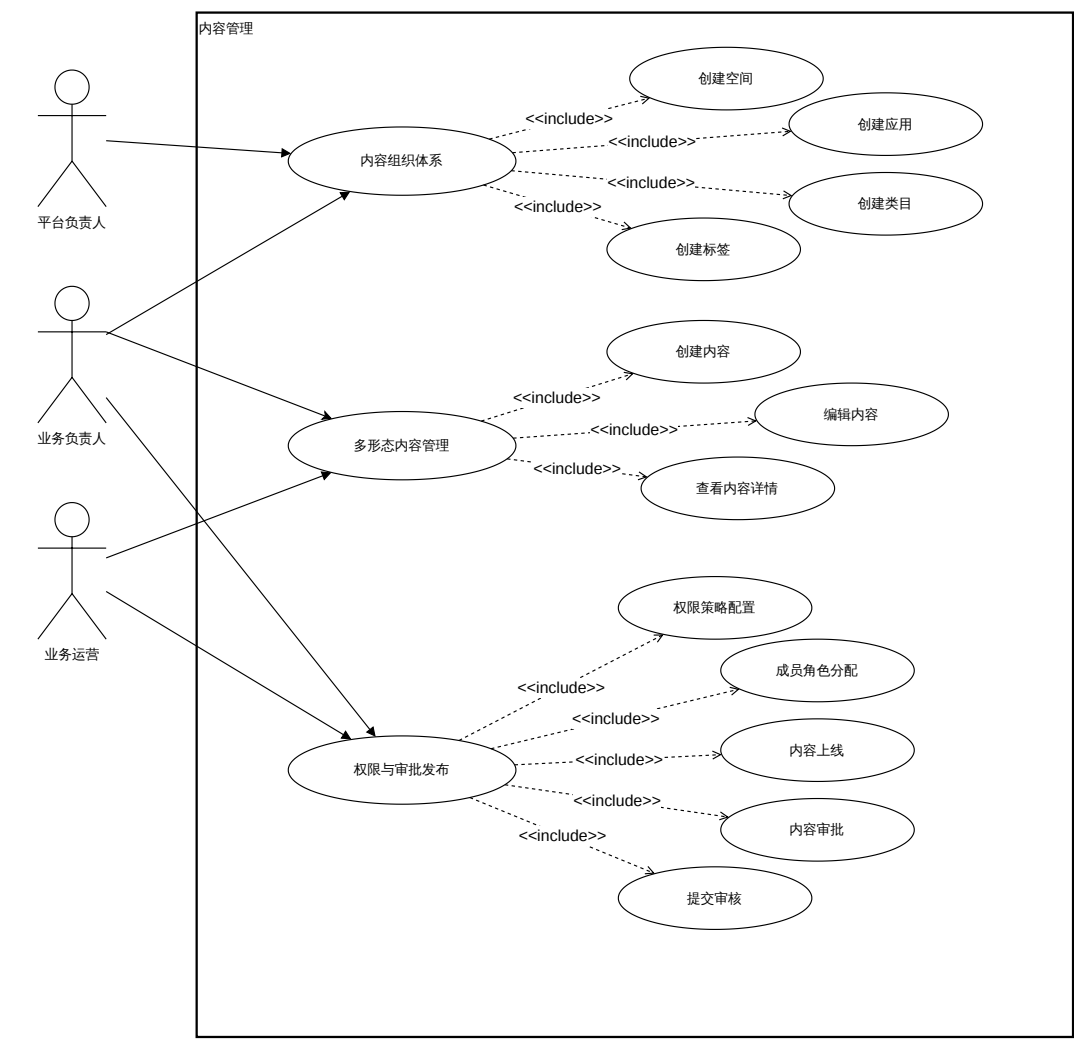


图 3-1: 内容管理用例图

建应用，系统能够在空间边界内进一步明确内容形态、类目树、标签体系等配置口径，并为后续内容生产与消费提供更细粒度的范围约束。

创建类目的用例描述如表 3-4 所示。类目用于构建目录导航结构，是页面化内容展示与内容聚合的重要入口。通过类目树的维护，平台能够将内容按主题与层级进行组织，使用户在浏览场景下可沿目录逐层定位，并为后续检索筛选与内容运营提供结构化维度。

创建标签的用例描述如表 3-5 所示。标签用于跨类目对内容进行补充标注与聚合，适合表达专题、版本、投放类型等横向维度。通过标签体系，平台能够在目录结构之外提供更灵活的筛选与运营配置方式，支撑内容的多角度检索与推荐。

表 3-2: 创建空间用例描述表

ID	CM-05
用例名称	创建空间
参与者	平台负责人
用例描述	创建业务空间以作为内容资产的组织边界与权限边界，并初始化空间基础配置。
触发条件	新业务线接入或现有业务需要建立独立治理边界。
前置条件	1. 当前用户具备空间管理权限。
后置条件	1. 系统生成空间 ID 并创建默认成员组与默认权限策略。
基本流程	1. 用户进入空间管理页面并点击新建空间。 2. 系统弹出空间信息编辑弹窗。 3. 用户填写空间名称、业务归属等信息并保存。 4. 系统校验信息并持久化保存空间配置。
替代流程	1. 若空间名称重复或信息校验失败，系统提示原因并要求用户修改。

表 3-3: 创建应用用例描述表

ID	CM-06
用例名称	创建应用
参与者	业务负责人
用例描述	在指定空间内创建应用并配置内容组织方式，以适配不同内容消费场景。
触发条件	业务需要新增内容门户、帮助中心或面向智能应用的知识库入口。
前置条件	1. 目标空间已存在且当前用户具备应用管理权限。
后置条件	1. 系统生成应用 ID 并初始化应用配置（可用形态、类目树与标签配置等）。
基本流程	1. 用户进入空间详情页并点击创建应用。 2. 用户选择应用类型并填写应用名称与基础配置。 3. 系统校验配置并创建应用。 4. 系统返回应用创建成功提示并展示应用配置入口。
替代流程	1. 若应用配置不完整（如未配置可用内容形态），系统提示并阻止创建。

（2）多形态内容管理。平台需要支持多种内容形态的统一纳管与一致化维护。不同内容形态在结构与消费方式上存在差异，但应具备统一的基础管理能力，包括创建、编辑、保存草稿以及详情读取等；同时需要在空间与应用边界内保持一致的元数据口径（如类目、标签、状态与权限）。

创建内容的用例描述如表 3-6 所示。内容创建是内容生命周期的起点，用于将业务知识、规范与案例等沉淀为统一的内容实体，并生成初始草稿以进入后续编辑与审批流程。该用例强调在统一模型下支持多形态内容，并确保内容归属、组织维度与基础属性在创建阶段即可被明确。

编辑内容的用例描述如表 3-7 所示。内容在演进过程中需要持续更新与迭

表 3-4: 创建类目用例描述表

ID	CM-11
用例名称	创建类目
参与者	业务负责人，业务运营
用例描述	在指定应用内新增类目节点，用于构建目录导航与内容聚合入口。
触发条件	新增业务主题或需要调整内容目录结构。
前置条件	1. 目标应用已存在且当前用户具备类目维护权限。
后置条件	1. 系统生成类目 ID 并将其挂载到指定父节点下。
基本流程	1. 用户进入应用配置中的类目管理页面。 2. 用户选择父节点并点击“新增类目”。 3. 用户填写类目名称等信息并保存。 4. 系统校验名称与层级合法性并持久化保存。
替代流程	1. 若类目名称冲突或层级不合法，系统提示原因并要求用户修改。

表 3-5: 创建标签用例描述表

ID	CM-12
用例名称	创建标签
参与者	业务运营
用例描述	在指定应用内新增标签，用于跨类目聚合、筛选与运营配置。
触发条件	需要新增运营标签以支持内容聚合与筛选。
前置条件	1. 目标应用已存在且当前用户具备标签维护权限。
后置条件	1. 系统生成标签 ID 并将其加入应用标签集合。
基本流程	1. 用户进入应用配置中的标签管理页面。 2. 用户点击“新增标签”并填写标签名称。 3. 系统校验标签名称合法性与重复冲突。 4. 系统保存标签并返回成功提示。
替代流程	1. 若标签名称重复或不合法，系统提示原因并要求用户修改。

代，编辑用例用于支持正文、属性字段与组织关系的修改，并保证修改结果可被保存与追踪。该用例同时为后续提审、发布与索引更新等链路提供稳定输入，降低口径不一致与过期信息的风险。

查看内容详情的用例描述如表 3-8所示。详情读取用于在管理与审核场景下核对内容正文、属性、状态与组织信息，并为编辑、提审与审批提供依据。该用例强调在权限可控的前提下，对不同内容形态以统一方式返回可展示的详情数据。

(3) 权限与审批发布。平台需要构建基于角色的访问控制（RBAC）体系，并结合内容对象级的 content_auth 策略授权，在空间与应用维度实现权限隔离与精细化授权。权限粒度应覆盖模块级与操作级，既能控制用户进入某一

表 3-6: 创建内容用例描述表

ID	CM-01
用例名称	创建内容
参与者	业务运营
用例描述	在指定空间与应用下创建新的内容实体，支持 knowledge、faq、course、case 等多种形态，并生成初始草稿。
触发条件	业务需要新增产品说明、规则解读或运营素材等内容资产。
前置条件	1. 目标空间与应用已创建并处于可用状态。 2. 当前用户拥有该空间下的内容创建权限。
后置条件	1. 系统生成唯一的内容 ID。 2. 内容状态置为“草稿”。
基本流程	1. 用户进入内容管理列表页，点击“新建”按钮。 2. 用户选择内容形态（如文章、问答等）。 3. 系统初始化空白草稿并跳转至编辑器。 4. 用户填写标题、正文及必要的属性字段（如类目、标签）。 5. 用户点击“保存”按钮。 6. 系统校验必填项与格式，校验通过后持久化存储。
替代流程	1. 若校验失败，系统提示具体错误字段，用户修改后可重新保存。

表 3-7: 编辑内容用例描述表

ID	CM-02
用例名称	编辑内容
参与者	业务运营
用例描述	对已存在的内容进行修改，包括正文更新与属性调整，并保存修改结果以进入后续提审与发布流程。
触发条件	内容信息发生变更或需要优化现有内容。
前置条件	1. 内容处于可编辑状态（如草稿、驳回或已发布）。 2. 当前用户拥有该内容的编辑权限。
后置条件	1. 内容变更被持久化保存。
基本流程	1. 用户在内容列表点击目标内容的“编辑”按钮。 2. 系统加载内容数据并进入编辑器。 3. 用户修改正文或属性信息。 4. 用户点击“保存”或“提交”。 5. 系统保存修改并记录变更。
替代流程	1. 若校验失败，系统提示需要补全或修正的字段。

空间/应用的访问权限，也能控制内容的创建、编辑、提审、审批、上线等具体操作权限。同时，审批发布机制用于生成一致的对外口径，确保只有满足合规与质量要求的内容可以进入“已发布”状态并对外可见。

成员角色分配的用例描述如表 3-9 所示。该用例用于在组织边界内建立“成员—角色—作用域”的映射关系，使不同用户在空间或应用范围内获得差异化的

表 3-8: 查看内容详情用例描述表

ID	CM-03
用例名称	查看内容详情
参与者	业务运营、业务负责人
用例描述	查看内容的详情信息与正文字段，以支持内容核对、编辑与提审前检查。
触发条件	用户需要核对某条内容的正文、属性或发布状态。
前置条件	1. 内容存在且当前用户具备访问权限。
后置条件	系统返回内容详情并展示。
基本流程	1. 用户在内容列表点击目标内容进入详情页。 2. 系统加载内容元数据（类型、标题、摘要、状态等）。 3. 系统按内容形态读取正文或结构字段并组装详情。 4. 系统返回详情并展示。
替代流程	1. 若内容无权限访问或状态不允许查看，系统拒绝并提示原因。

可见与可操作权限。通过角色分配，平台将权限控制从个体配置抽象为可复用的角色模型，降低权限维护复杂度。

表 3-9: 成员角色分配用例描述表

ID	CM-07
用例名称	成员角色分配
参与者	平台负责人，业务负责人
用例描述	为目标用户在空间或应用维度分配角色，以确定其可访问范围与可执行操作集合。
触发条件	新成员加入空间/应用或成员职责变更需要调整权限。
前置条件	1. 目标空间与应用已存在。 2. 当前用户具备成员管理权限。
后置条件	1. 系统更新用户角色映射并立即生效。
基本流程	1. 管理员进入空间或应用的成员管理页面。 2. 管理员选择目标用户并设置角色。 3. 系统校验管理权限与角色合法性。 4. 系统保存变更并返回成功提示。
替代流程	1. 若当前用户无权限执行授权操作，系统拒绝并提示原因。

权限策略配置的用例描述如表 3-10所示。该用例用于定义角色与权限项之间的映射规则，明确不同角色在不同模块与操作上的能力边界，并形成默认权限口径。通过统一配置，平台能够在新增业务空间或应用时复用权限模板，保证权限治理的一致性与可控性。

内容对外可用的前提是准确性与合规性可控。平台提供流程化的审核与发布机制，对内容状态流转进行约束。内容在生命周期中至少包含草稿、待审

表 3-10: 权限策略配置用例描述表

ID	CM-08
用例名称	权限策略配置
参与者	平台负责人
用例描述	配置空间或应用的默认权限策略与操作权限集合，实现对模块与操作级权限的精细化控制。
触发条件	业务治理要求调整默认权限口径或新增权限项。
前置条件	1. 当前用户具备全局权限配置能力。
后置条件	1. 系统更新权限策略并对后续鉴权生效。
基本流程	1. 用户进入权限配置页面选择目标空间/应用。 2. 用户配置角色与权限项映射关系并保存。 3. 系统校验配置合法性并持久化保存。
替代流程	1. 若配置导致关键角色缺失必要权限，系统提示风险并阻止保存。

批、已发布、已下线等状态，状态变更需满足权限校验与流程条件。

提交审核的用例描述如表 3-11所示。该用例用于将草稿内容提交进入审批流程，作为“对外发布口径”的前置关卡。通过提审，平台能够把内容从可编辑状态迁移到待审批状态，并记录审批上下文与责任链路，为后续审批决策提供依据。

表 3-11: 提交审核用例描述表

ID	CM-09
用例名称	提交审核
参与者	业务运营
用例描述	将内容草稿提交进入审批流程，申请将内容上线并对外可用。
触发条件	草稿内容完成编辑，需要上线以支撑页面展示、检索或智能问答。
前置条件	1. 内容处于可提交状态且基础校验通过。 2. 当前用户具备提审权限。
后置条件	1. 系统创建审批记录并将内容状态更新为待审批。 2. 通知审批者。
基本流程	1. 用户在内容详情页点击提交审核。 2. 系统执行内容完整性校验与权限校验。 3. 系统创建审批单并将内容状态置为待审批。 4. 系统向用户返回提审成功提示。
替代流程	1. 若权限校验失败，系统拒绝提交并提示原因。 2. 若内容校验失败，系统提示需要补全或修正的字段。

内容审批的用例描述如表 3-12所示。审批者对待审批内容进行合规性与准确性检查，给出通过或拒绝结论，并将结果回写到内容状态与审批记录中。该用例通过流程约束与责任可追踪，确保发布内容具备统一口径并可被稳定

复用。

表 3-12: 内容审批用例描述表

ID	CM-10
用例名称	内容审批
参与者	业务负责人
用例描述	对待审批内容进行审核并给出通过或拒绝结论，决定内容能否上线并对外可见。
触发条件	审批者在审批列表中发现待处理任务或收到待审批通知。
前置条件	1. 内容状态为待审批。 2. 当前用户具备审批权限。
后置条件	1. 若审批通过，系统将内容状态更新为已发布，并发布内容变更事件供下游消费。 2. 若审批拒绝，系统将内容状态更新为未通过并记录拒绝原因。
基本流程	1. 审批者进入审批列表并打开目标审批单。 2. 审批者查看内容关键信息与变更点。 3. 审批者选择通过或拒绝并提交意见。 4. 系统更新审批单状态并回写内容状态。
替代流程	1. 若审批者选择拒绝，系统要求填写拒绝原因并通知发布者。

内容上线的用例描述如表 3-13 所示。上线用于控制内容在应用侧的可见性，是“发布口径”对外生效的关键动作。该用例将内容状态切换到已发布，并触发与内容变更相关的后续链路（如索引更新与消费侧可见性刷新），保证内容在展示、检索与问答场景中的一致性。

表 3-13: 内容上线用例描述表

ID	CM-04
用例名称	内容上线
参与者	业务负责人、业务运营
用例描述	控制内容在应用端的可见性。上线即发布内容使其对外可见。
触发条件	内容审核通过需发布。
前置条件	1. 上线前需通过审批流程。
后置条件	1. 更新内容发布状态（已发布）。 2. 触发索引更新事件，同步搜索与问答端数据。
基本流程	1. 用户对“待发布”内容点击“上线”。 2. 系统校验当前状态与权限。 3. 系统更新内容状态并发布更新事件到 MQ。
替代流程	1. 若状态校验失败或权限不足，系统拒绝操作并提示原因。

3.2.3 内容应用功能需求

内容应用功能的用例图如图 3-2 所示。内容应用模块的主要功能是对内容进行展示、检索与问答，以满足不同场景下的内容消费需求。一方面，平台需要支持面向门户与管理后台的页面化展示与检索能力，帮助用户通过目录导航、列表筛选与关键词搜索快速定位内容；另一方面，平台需要面向智能问答等上游智能应用提供检索增强生成能力，使用户能够以对话方式获取结构化且可追溯的内容依据。

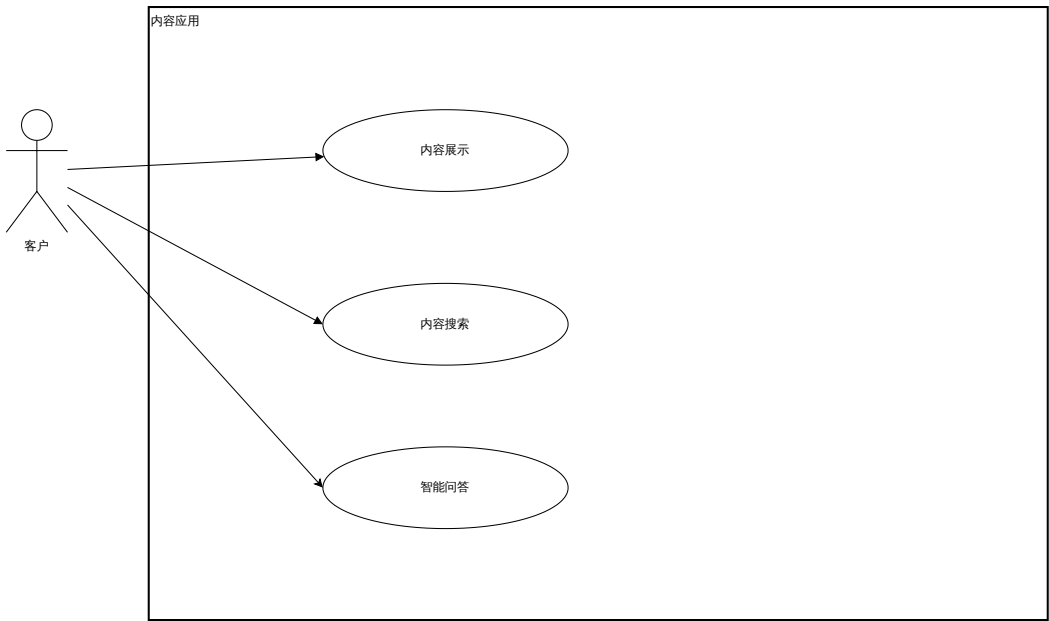


图 3-2: 内容应用用例图

- 内容应用模块的核心用例可归纳为以下几类。
- (1) 内容展示。面向页面化消费场景，平台需提供内容目录与列表能力，支持按空间、应用、类目与标签等维度进行内容聚合展示，并提供详情页渲染与内容关联跳转能力，保障内容在不同业务线与不同应用中可被稳定访问。
 - (2) 内容搜索。平台需提供关键词搜索与结构化筛选能力，支持对标题、正文、标签与类目等字段的组合查询，并保证检索结果在权限范围内可见。为提升体验，搜索结果需支持高亮、排序与分页等能力。
 - (3) 智能问答。平台需提供面向智能问答场景的内容支撑能力：当用户以自然语言提问时，系统能够在指定空间与应用范围内检索相关内容并生成回

答，同时返回可追溯的引用来源，以保障结果可解释并降低口径不一致风险。

内容展示用例描述如表 3-14所示。该用例覆盖用户以目录导航与列表聚合的方式浏览内容，并进入详情页获取正文与关联信息的典型流程。通过稳定的展示入口，平台能够将内容以结构化方式对外呈现，并记录访问行为用于后续效果评估与运营分析。

表 3-14: 内容展示用例描述表

ID	CA-01
用例名称	内容展示
参与者	客户，业务运营
用例描述	用户在内容门户或管理后台按目录导航与列表聚合浏览内容，并查看内容详情。
触发条件	用户希望通过类目、标签或列表入口快速定位并阅读内容。
前置条件	1. 目标应用存在已发布内容。 2. 用户具备对应空间与应用的访问权限。
后置条件	系统展示内容列表与详情，并记录访问行为数据用于后续分析。
基本流程	1. 用户进入内容门户或管理后台。 2. 用户通过类目树或标签进入内容列表。 3. 用户点击某一条内容进入详情页查看正文与关联信息。
替代流程	1. 若内容无权限访问或已下线，系统提示并返回可访问的替代内容或上级列表。

内容检索用例描述如表 3-15所示。该用例面向关键词检索与结构化筛选场景，强调在范围与权限约束下返回相关结果集合，并支持高亮、排序与分页等检索交互要素。通过检索用例，平台能够满足用户在不确定目录入口时的“快速定位”诉求，并沉淀检索行为用于优化召回与内容组织。

智能问答用例描述如表 3-16所示。该用例覆盖用户以自然语言提问、系统召回证据并生成回答的端到端过程，并要求输出携带可追溯引用以保障结果可解释。通过问答用例，平台能够在高频咨询场景下提供更高效率的内容获取方式，同时沉淀会话数据以支持后续治理与洞察。

表 3-15: 内容检索用例描述表

ID	CA-02
用例名称	内容检索
参与者	客户，业务运营
用例描述	用户通过关键词与筛选条件检索内容，系统返回权限范围内的相关结果列表。
触发条件	用户需要定位某一产品说明、业务规则或典型案例。
前置条件	1. 目标空间与应用存在已发布内容。 2. 用户具备对应空间与应用的访问权限。
后置条件	系统返回结果列表并记录检索行为数据用于后续分析。
基本流程	1. 用户进入内容门户或管理后台的搜索页面。 2. 用户输入关键词并选择筛选条件。 3. 系统执行权限过滤与检索召回。 4. 系统按相关性与业务规则排序并返回结果。
替代流程	1. 若无匹配结果，系统提示并引导用户调整关键词或筛选条件。

表 3-16: 智能问答用例描述表

ID	CA-03
用例名称	智能问答
参与者	客户
用例描述	用户以自然语言提问，系统结合召回证据生成回答并返回引用来源。
触发条件	用户希望通过对话快速获取业务规则解释或操作指导。
前置条件	1. 用户具备问答入口的访问权限。 2. 系统可获取足够的召回证据或具备降级策略。
后置条件	系统返回回答与引用，并记录会话明细与反馈信息用于评估。
基本流程	1. 用户在智能助手中输入问题。 2. 系统完成内容召回与证据组织。 3. 生成模型基于证据生成回答。 4. 系统返回回答与引用来源。
替代流程	1. 若问题不在知识覆盖范围内，系统提示并引导用户转人工或查看相关内容列表。

3.2.4 内容治理功能需求

内容治理功能的用例图如图 3-3 所示。内容管理模块为平台提供了统一的内容资产底座，内容应用模块实现了内容的展示与智能问答，然而随着内容规模扩大与消费频率提升，重复冗余、表述冲突与覆盖不足等质量问题会直接影响检索效果与问答准确性。因此，内容治理模块的主要功能是对内容质量进行指标化检测，并以任务形式推动治理闭环，与前述模块协同形成“生产—应用—治理—反馈”的完整链路，保证内容体系在持续演进过程中的可维护性与可

控性。

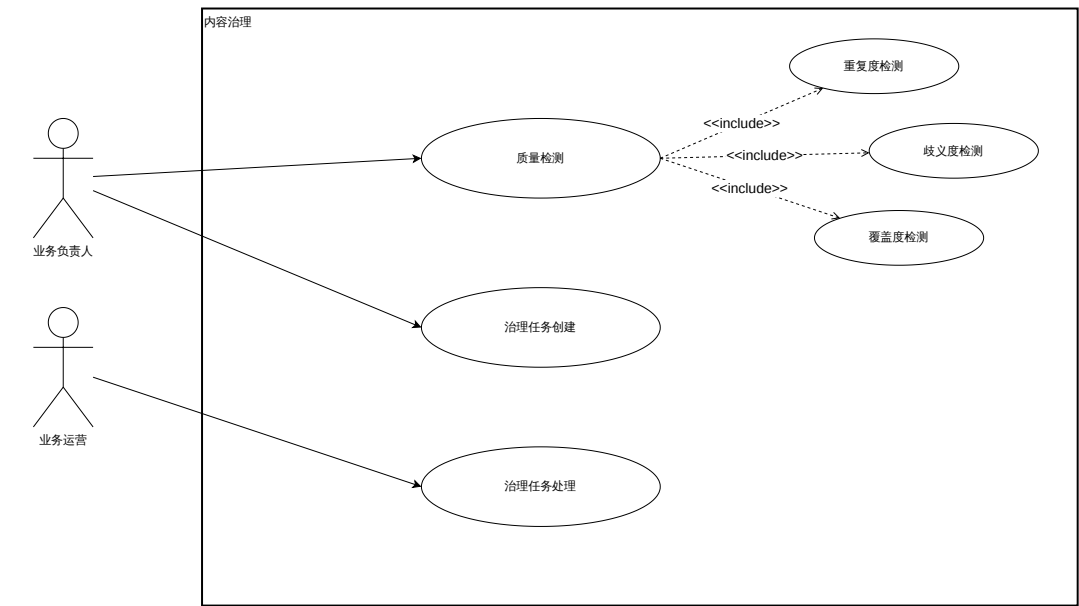


图 3-3: 内容治理用例图

内容治理模块的核心用例包括治理检测、治理任务创建与分发、治理处理与审核、治理任务复检回收与追踪等。平台将治理问题抽象为标准化任务，任务中应包含问题类型、问题定位（内容与片段）、影响范围、建议处理方式与责任归属，从而使治理过程可追踪、可统计。

(1) 重复度检测。识别内容体系中的重复条目与冗余片段，支持对重复内容进行聚合展示与合并建议，减少重复维护带来的成本与口径不一致风险。

(2) 歧义度检测。针对同一问题或同一概念在不同内容中的表述冲突与语义歧义进行检测，给出冲突片段与候选一致口径，降低智能问答场景下的误导概率。

(3) 覆盖度检测。结合用户查询与会话数据识别内容缺口，定位高频问题无有效内容支撑的领域，并形成补充内容的治理建议。

质量检测用例描述如表 3-17 所示。该用例用于在指定空间与应用范围内对内容执行重复度、歧义度与覆盖度检测，输出可定位的问题集合与治理建议。通过质量检测，平台能够将抽象的质量问题转化为可分析、可统计的结构化结果，为后续任务化治理提供输入。

治理任务创建用例描述如表 3-18 所示。该用例用于将检测结果转化为可流转的治理任务，明确任务类型、优先级、责任人与处理要求，并形成可追踪的

表 3-17: 质量检测用例描述表

ID	CG-01
用例名称	质量检测
参与者	平台负责人
用例描述	系统对指定范围内内容执行重复度、歧义度或覆盖度检测，输出可定位的问题集合与治理建议。
触发条件	需要对内容体系的质量问题进行阶段性治理或例行巡检。
前置条件	1. 指定空间与应用范围可用。 2. 已具备相似度计算、冲突判定或缺口识别能力。
后置条件	系统生成检测结果并可转化为治理任务。
基本流程	1. 用户在治理模块选择检测范围与检测类型。 2. 系统执行对应检测并生成问题定位与影响范围。 3. 系统输出问题集合与建议处理方式。
替代流程	1. 若检测范围过大导致资源不足，系统支持分批检测并输出进度。

状态流转记录。通过任务创建，平台将质量检测与人工处理过程连接起来，推动治理进入闭环。

表 3-18: 治理任务创建用例描述表

ID	CG-02
用例名称	治理任务创建
参与者	平台负责人
用例描述	将检测结果转化为可分发的治理任务，明确责任人与处理要求。
触发条件	检测结果需要进入治理闭环并推动相关人员处理。
前置条件	已产生可定位的质量问题集合。
后置条件	1. 系统创建治理任务并进入待处理状态。 2. 系统通知责任人。
基本流程	1. 用户在检测结果中选择问题条目并点击创建任务。 2. 用户设置任务类型、优先级与责任人。 3. 系统创建任务并生成问题定位与建议处理内容。
替代流程	1. 若同一问题已存在未完成任务，系统提示合并或追加处理。

治理任务处理用例描述如表 3-19 所示。该用例由责任人依据问题定位对内容执行合并、修订、补充或下线等操作，并提交处理说明与结果。通过任务处理与后续复检，平台能够持续收敛重复冗余与表述冲突等问题，提升内容体系的可维护性与可用性。

表 3-19: 治理任务处理用例描述表

ID	CG-03
用例名称	治理任务处理
参与者	业务运营
用例描述	责任人根据任务定位对内容进行合并、修订、补充或下线，并提交处理结果。
触发条件	责任人收到治理任务并需要完成内容修订。
前置条件	1. 责任人具备对应内容的编辑权限。 2. 任务处于可处理状态。
后置条件	1. 内容变更进入相应发布流程。 2. 任务状态更新并记录处理结果。
基本流程	1. 责任人打开治理任务查看问题定位与建议。 2. 责任人跳转至对应内容进行修订或合并。 3. 责任人提交任务处理说明并完成。 4. 系统更新任务状态并触发后续校验或复检。
替代流程	1. 若处理涉及多内容协同，系统支持拆分子任务并分别跟踪。

3.2.5 数据洞察功能需求

数据洞察功能的用例图如图 3-4所示。内容管理模块负责内容的生产与发布，内容应用模块实现内容的消费与问答，内容治理模块推动质量问题的检测与闭环处理，而数据洞察模块则作为平台的“反馈中枢”，对上述模块的运行效果进行量化评估，支撑运营决策与平台迭代。该模块通过对页面访问、检索与问答链路、用户会话等数据进行采集与分析，输出面向运营与治理的指标体系，使平台能够形成“应用驱动生产、数据反哺治理”的闭环。

数据洞察模块的关键用例包括页面数据监控、会话维度分析、内容维度分析与会话明细。平台通过对页面访问、检索与问答链路日志进行采集与聚合，支撑运营人员从页面、会话与内容三个视角对平台效果进行量化评估，并通过会话明细下钻实现问题定位，为内容生产与治理策略调整提供依据。

页面数据监控用例描述如表 3-20所示。该用例面向页面化入口的效果评估，通过 PV、UV、点击率等指标反映页面访问与交互情况，并支持按页面或组件维度下钻查看趋势与 TopN。通过页面监控，平台能够从入口侧识别流量变化与内容分发效果，为运营优化提供依据。

会话维度分析用例描述如表 3-21所示。该用例以会话作为观测对象，通过对相似 Query 聚类与指标统计，刻画智能问答链路的解决率、转人工率等效果，并定位高频问题与异常会话簇。通过会话视角的聚类与下钻，平台能够发

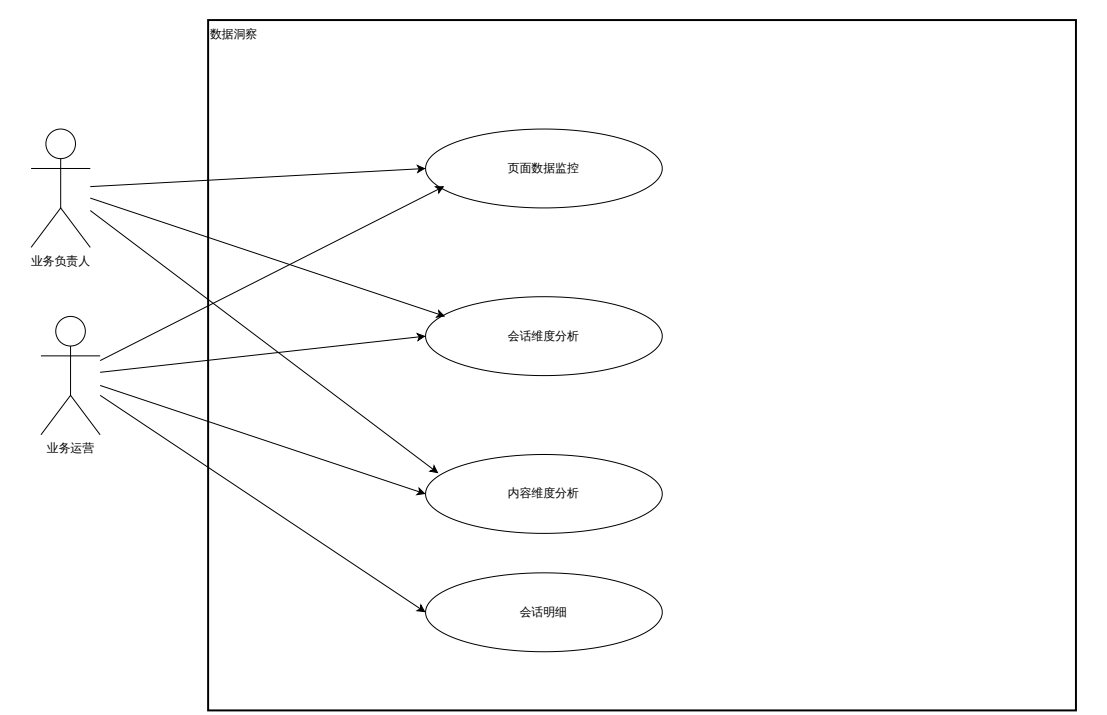


图 3-4: 数据洞察用例图

表 3-20: 页面数据监控用例描述表

ID	DI-01
用例名称	页面数据监控
参与者	业务负责人，业务运营
用例描述	用户查看页面层面的访问数据统计与趋势，分析 PV、UV 与点击率等指标，评估页面化展示的使用情况。
触发条件	需要评估某一空间/应用下页面化展示入口的访问与转化效果。
前置条件	1. 页面曝光与点击等事件已采集。 2. 指标已完成聚合计算。
后置条件	无
基本流程	1. 用户进入数据洞察模块并选择时间范围与空间/应用。 2. 系统展示页面 PV/UV/点击率趋势与 TopN。 3. 用户按页面/组件维度筛选并下钻查看访问或点击明细。
替代流程	1. 若数据量过大，系统按时间粒度或分页方式展示明细。

现知识薄弱点并为覆盖度治理提供输入。

内容维度分析用例描述如表 3-22 所示。该用例从内容资产视角统计内容被召回与被引用的频次等指标，以评估内容在检索与问答场景中的实际贡献度，并识别高价值内容与低效内容。通过内容维度的量化评估，平台能够为内容优化与治理策略调整提供数据支撑。

表 3-21: 会话维度分析用例描述表

ID	DI-02
用例名称	会话维度分析
参与者	平台负责人，业务负责人
用例描述	用户在会话维度对相似 Query 进行聚类分析，查看会话数量、模型识别解决率、意向转人工率与转人工率等指标，定位高频问题与异常会话簇。
触发条件	需要从会话视角评估智能问答链路效果或定位问题分布。
前置条件	1. 会话数据与问答链路事件已采集。 2. 会话聚类结果与指标已完成聚合计算。
后置条件	无
基本流程	1. 用户进入会话维度分析视图并选择时间范围与空间/应用。 2. 系统展示整体会话指标与聚类列表。 3. 用户选择某一聚类下钻查看聚类内的 Query 与会话明细。
替代流程	1. 用户可按时间粒度、聚类标题或转人工率等条件筛选聚类。

表 3-22: 内容维度分析用例描述表

ID	DI-03
用例名称	内容维度分析
参与者	业务负责人，业务运营
用例描述	用户从内容视角查看内容的被召回次数、被生成次数、意向转人工次数与转人工次数等指标，评估内容在智能问答场景中的实际使用效果。
触发条件	需要评估内容体系的使用效果并识别高价值或低效内容。
前置条件	1. 召回与生成链路事件已采集并关联到内容。 2. 内容维度指标已完成聚合计算。
后置条件	无
基本流程	1. 用户进入内容维度分析视图并选择时间范围与空间/应用。 2. 系统展示内容指标列表与 TopN。 3. 用户选择某一内容下钻查看命中的会话或 Query 明细。
替代流程	1. 用户可按内容类型、类目与标签筛选内容集合。

会话明细用例描述如表 3-23所示。该用例提供对单个会话的链路级还原能力，展示原始 Query、改写结果、召回证据与回答输出等关键字段，并支持按时间轴查看多轮交互。通过会话明细下钻，平台能够在异常指标出现时快速定位原因，并将发现的问题反馈到内容治理与检索策略迭代中。

表 3-23: 会话明细用例描述表

ID	DI-04
用例名称	会话明细
参与者	平台负责人，业务负责人，业务运营
用例描述	用户查看单个会话的详细信息，包括聚类类型、原始 Query、改写 Query、召回内容以及是否转人工等信息，用于定位会话异常与内容支撑问题。
触发条件	需要对某一聚类或某一异常指标进行下钻分析与问题定位。
前置条件	1. 会话事件已采集并能关联到会话 ID。 2. 会话明细数据可按权限查询。
后置条件	无
基本流程	1. 用户从会话维度分析或内容维度分析页面下钻进入会话明细。 2. 系统展示会话的原始 Query 与改写 Query、召回内容、回复内容与转人工信息。 3. 用户按时间轴查看多轮交互细节。
替代流程	1. 若用户无权限查看敏感字段，系统对用户标识与原始文本等字段脱敏展示。

3.2.6 系统非功能需求

在广告与商业化场景下，内容平台面临内容规模大、更新频繁、应用链路长且跨系统依赖多等特点，除功能需求外还需满足一系列非功能需求。

(1) 一致性与可追溯性。内容在发布前后存在草稿、待审批、已发布等状态，平台需保证状态流转的正确性；在问答与洞察场景中，通过会话 ID 与链路标识将问题输入、改写、召回证据、生成结果与转人工等事件关联，支撑会话明细下钻与问题定位。

(2) 安全与权限。平台在空间与应用维度进行权限隔离，并在内容粒度通过 content_auth 实现策略式授权；平台管理权限采用“角色-权限-作用域”的 RBAC 模型。

(3) 可扩展性。平台以统一内容模型承接多形态内容（knowledge、faq、course、case），并通过类目与标签机制支撑内容组织；治理规则与洞察指标按模块化方式演进，降低新增形态与新增治理策略的改造成本。

(4) 性能与稳定性。平台需同时支撑运营侧高频编辑发布与用户侧高并发浏览检索。在读链路中需以“发布口径”为准并尽量复用缓存与索引能力，以降低数据库压力；在写链路中通过事件机制将索引更新、统计计算等耗时操作异步化处理，避免阻塞主链路；同时建设必要的监控告警与降级策略，保证在异常场景下仍可提供可用的内容查询与问答服务。

3.3 系统整体设计

平台整体架构如图3-5所示。平台面向内容全生命周期，整体划分为内容管理、内容应用、内容治理与数据洞察四个核心模块，并在入口层、服务层与基础设施层形成清晰分工：入口层包含管理后台与内容门户等对外界面，并可对接上游智能助手作为问答入口；服务层承载内容生产与发布、展示与检索、问答与引用溯源、质量检测与任务闭环、事件采集与指标查询等能力；基础设施层提供关系型/文档型存储、全文检索与向量检索、消息队列以及数仓侧指标沉淀等通用支撑。为降低跨域耦合并提升吞吐，平台以消息队列作为关键协同机制，将发布后的索引更新、统计回收等耗时处理从主链路中解耦出来。

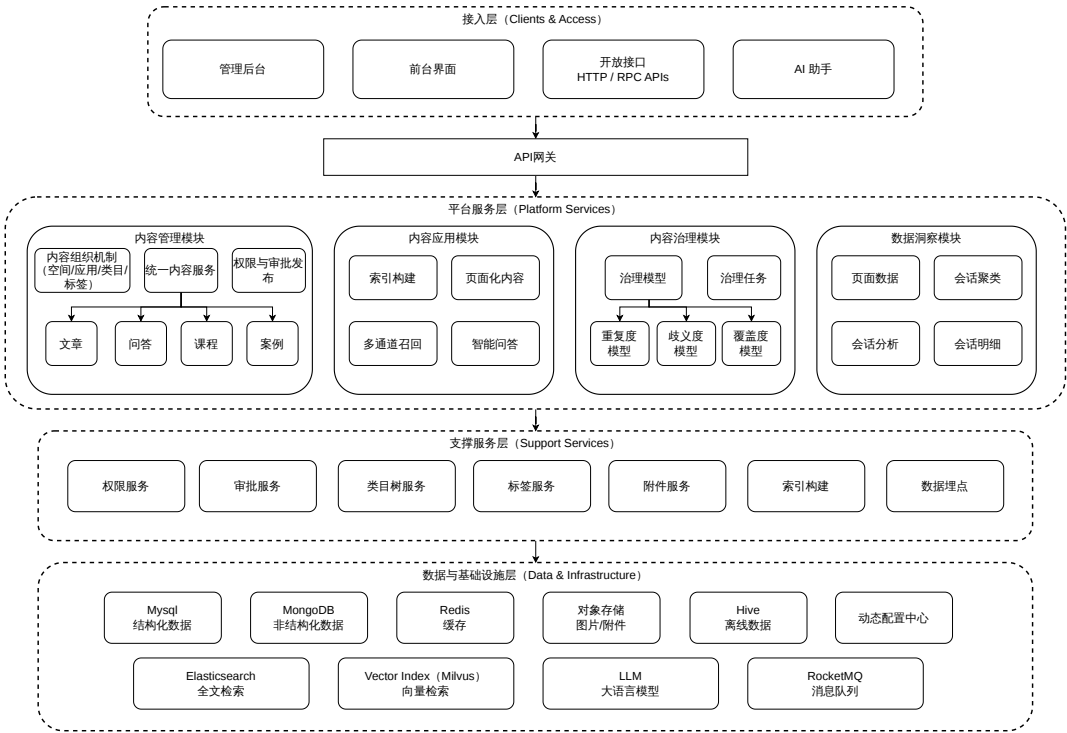


图 3-5: 系统架构图

为便于从不同维度刻画平台设计，本文在系统架构图基础上，引入逻辑视图、开发视图与物理视图进行补充说明：逻辑视图侧重业务能力与闭环关系，开发视图侧重工程模块划分与依赖关系，物理视图侧重部署形态与基础设施选型。

(1) 逻辑视图。图3-6从业务能力组织关系角度给出了平台的整体逻辑结构。平台由管理后台界面与前台界面两个入口承接不同使用方式，核心能力划

分为内容管理、内容应用、内容治理与数据洞察四个域：内容管理域承载内容组织体系、多形态内容管理与权限审批发布；内容应用域负责内容前置处理与索引构建、页面化内容应用、多通道召回与智能问答；内容治理域承载内容质量模型与治理任务线上化；数据洞察域负责数据采集、会话分析与会话聚类。上述能力域共同依赖缓存、结构化数据、非结构化数据、全文索引、向量索引与离线数据等存储对象，并与 SSO 登录、LLM 及日志记录等外部服务协同，从而形成“入口—核心能力—存储与外部支撑”相结合的逻辑分层。

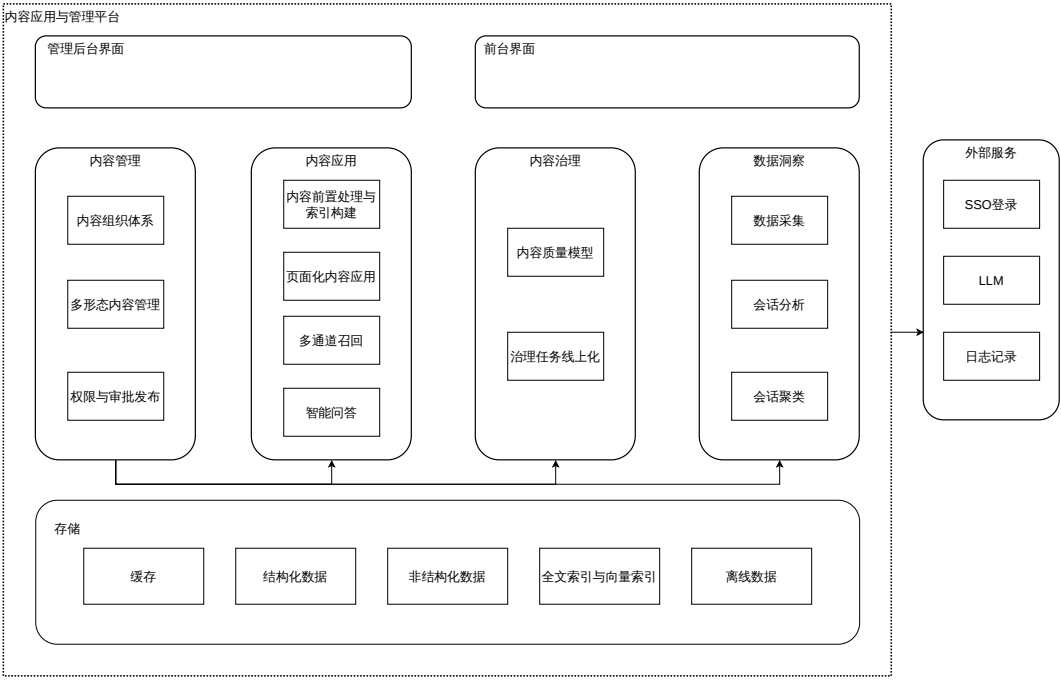


图 3-6: 逻辑视图

(2) 开发视图。图 3-7为平台的开发视图，用于描述工程实现层面的模块划分与调用关系。系统对外提供管理后台界面与前台界面两个前端入口，二者均采用 React 实现；后端则拆分为内容管理服务、内容应用服务、内容治理服务与数据洞察服务四个 SpringBoot 服务。职责分工上，内容应用服务负责向前台界面提供内容展示、检索与问答能力，内容管理服务、内容治理服务与数据洞察服务负责向管理后台界面提供配置、治理与运营分析能力。支撑关系上，内容管理服务通过 SSO 登录完成统一认证，内容应用服务调用 RAG 与 LLM 能力完成问答链路，数据洞察服务调用数据仓库提供指标查询与分析支撑。该视图体现了前端按入口区分、后端按能力拆分、支撑能力按服务依赖接入的工程组织方式。

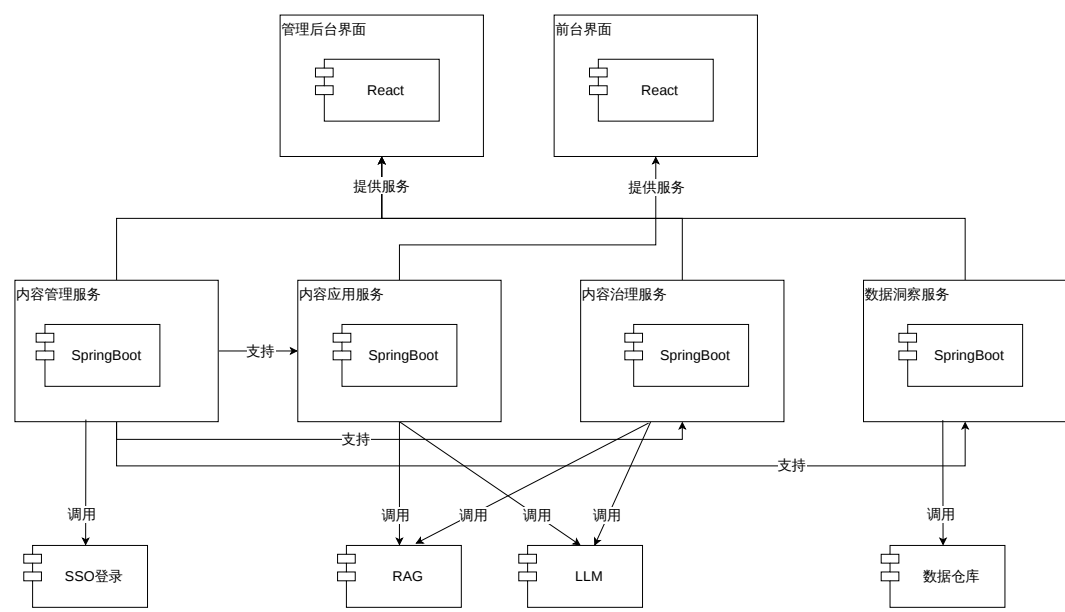


图 3-7: 开发视图

(3) 物理视图。图 3-8 为平台的物理视图，用于描述部署与运行环境中的组件构成。内容管理服务、内容应用服务、内容治理服务与数据洞察服务统一部署在 TCE 容器平台中，并共同对接 MySQL、Redis、MongoDB、Hive 与 RocketMQ 等基础组件：其中，MySQL 承载组织、内容元数据、权限与任务等结构化数据，Redis 提供缓存与分布式锁能力，MongoDB 用于存储灵活结构内容，Hive 承载数据洞察相关离线宽表与明细表，RocketMQ 用于解耦索引更新与统计回收等异步链路。与此同时，平台还接入 LLM、Elasticsearch 与 Milvus，分别支撑大模型生成、全文检索与向量检索能力。该视图体现了平台在容器化部署基础上，通过存储、检索、中间件与模型服务协同支撑业务运行的物理形态。

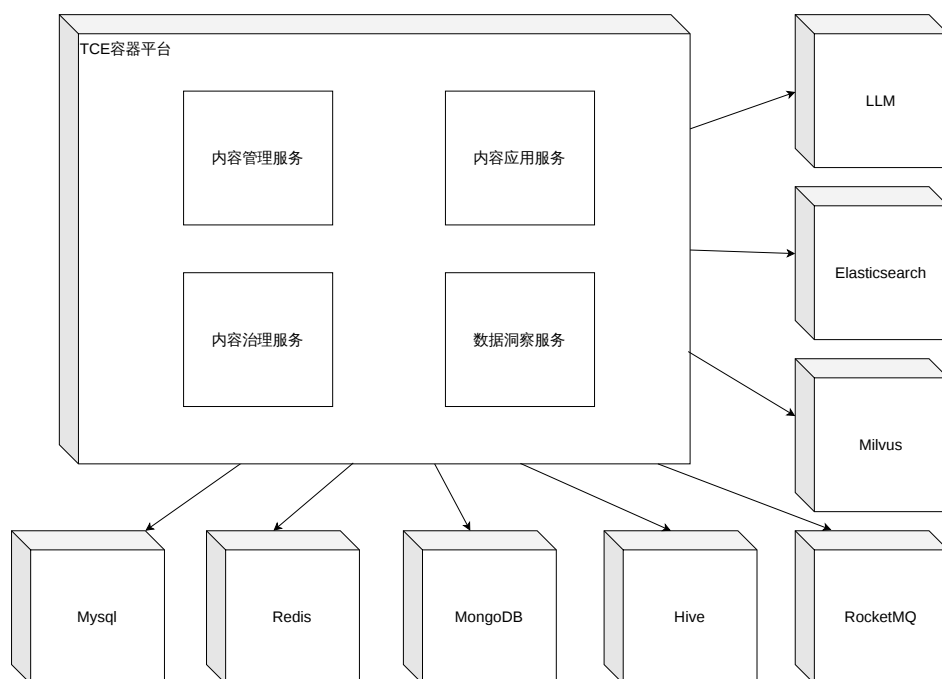


图 3-8: 物理视图

3.4 系统的模块设计

3.4.1 内容管理模块设计

内容管理模块是平台的内容资产底座，对应需求分析中的“内容管理功能需求”部分，负责内容组织体系建设、多形态内容管理以及权限与审批发布等能力。该模块的设计目标是以统一数据模型纳管多形态内容，并通过可追溯的权限与流程机制，保障内容从生产到发布的过程可控、结果可查，为后续内容应用与内容治理提供一致的数据输入。以下按“内容组织体系—多形态内容管理—权限与审批发布”的顺序展开。

（1）内容组织体系。平台采用“空间—应用—类目—标签”四级组织体系，用于统一表达业务归属、消费场景与内容组织方式。空间与应用承载业务隔离与场景划分的职责，类目与标签用于在应用内组织内容并支撑目录导航与聚合筛选。该体系的关键要点如下：

- **空间（Space）**：作为最高层级的业务容器，对应独立的事业部或业务线。空间实现数据物理隔离与人员权限边界，空间管理员拥有最高配置权限。
- **应用（Application）**：作为空间下的具体内容场景（如“客服知识库”、“销

售话术库”），对应独立的前端入口与配置集合。应用层级维护独立的内容类型开关、审批流配置与服务接入密钥。

- **类目（Category）**：作为应用内的树状导航结构，用于构建稳定的内容目录体系。
- **标签（Tag）**：作为跨类目的扁平化属性集合，用于灵活的内容聚合与运营筛选。

空间与应用管理的关键交互过程如图 3-9 所示。用户先创建空间，然后再空间下创建应用，创建空间和应用时，都会通过权限服务校验用户的权限。

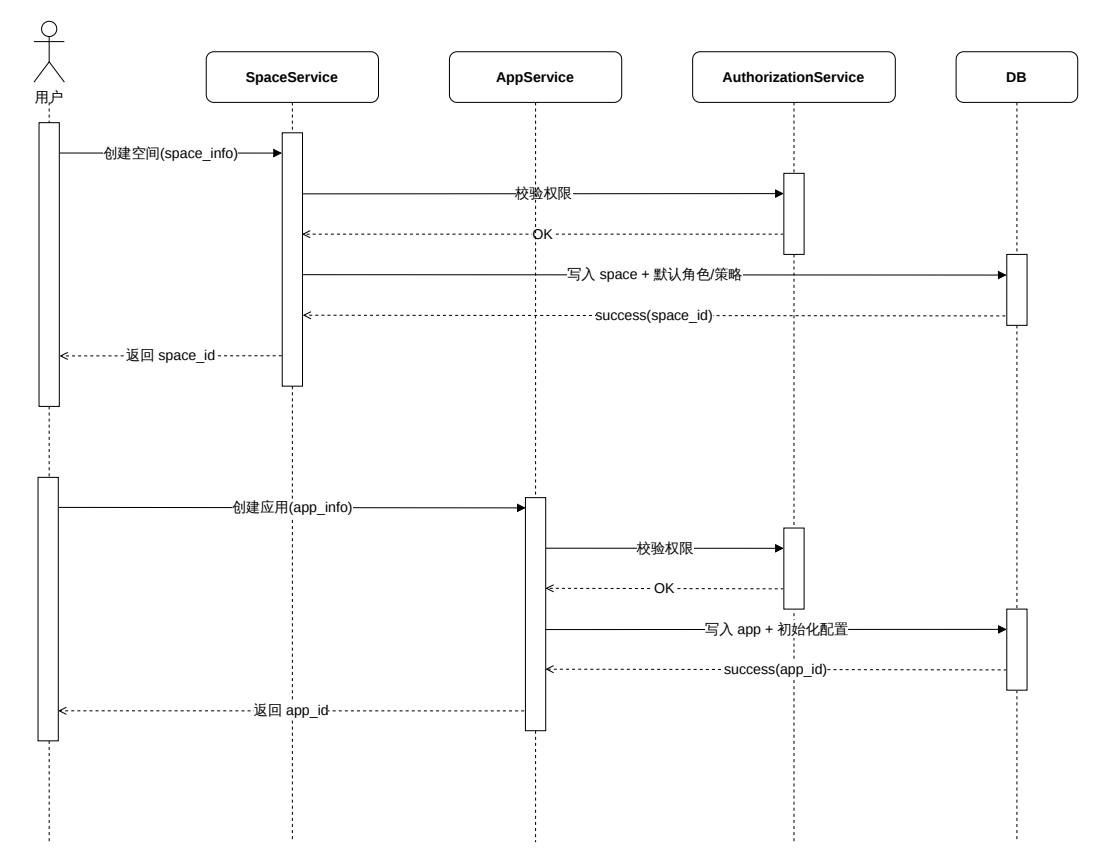


图 3-9: 空间与应用管理时序图

(2) 多形态内容管理。平台在内容层引入统一内容模型，使用 content_type 区分 knowledge、faq、course 与 case 四类内容形态，并由一张内容表（content）维护所有内容的公共元数据字段（内容 ID、类型、标题、摘要、正文引用、创建时间、更新时间、状态等）。不同内容类型的正文或结构字段存储在对应子表或文档库中，如 knowledge、faq、course 等；其中 case 为适配结构化字段灵活演进，其详情存储在 MongoDB 中。该设计保证了多形态内容在

管理侧具备一致的生命周期操作，并为内容创编与详情能力提供基础。

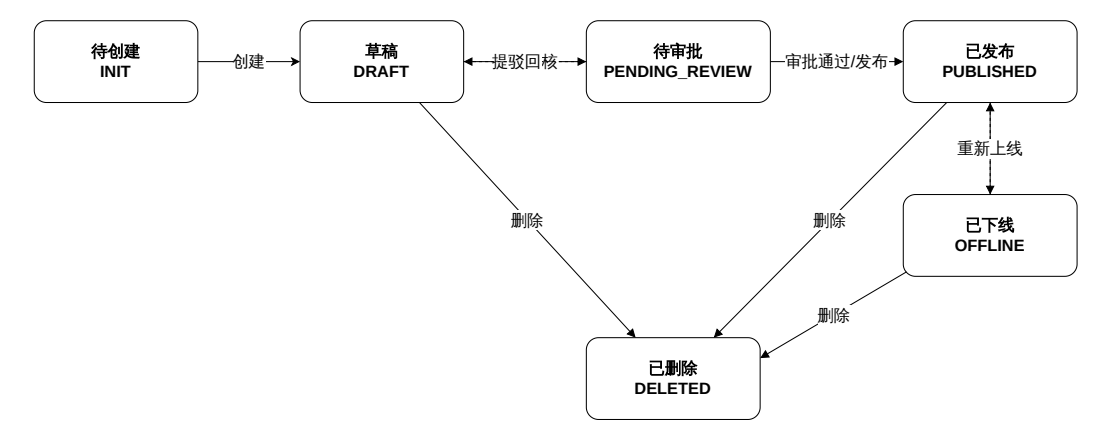


图 3-10: 内容状态机图

(3) 权限与审批发布。平台在空间与应用维度进行访问隔离，并在内容粒度通过 `content_auth` 实现策略式授权，以满足精细化操作控制需求。对于管理端操作，平台结合 RBAC 角色体系完成基础权限控制；对于内容对象，平台通过“授权对象 + 策略 + 变量 + 权限码”的模型表达细粒度权限。RBAC 权限矩阵如表 3-24 所示。

表 3-24: RBAC 权限矩阵表

角色（作用域）	全局配置/空间创建	空间内应用/成员/审批/发布	内容增删改/提审	类目/标签维护	查看/检索
平台负责人（GLOBAL）	✓	✓	✓	✓	✓
业务负责人（SPACE）		✓	✓	✓	✓
业务运营（APPSPACE）			✓	✓	✓

权限体系在工程实现中由 RBAC 与 `content_auth` 两部分叠加组成：RBAC 负责模块与操作级的基础权限，`content_auth` 负责内容对象级的策略授权。

审批发布链路如图 3-11 所示，在内容发布环节，平台遵循“编辑—提审—审批—发布”的流转机制：内容上线前需要通过审批流程，审批通过后内容服务更新内容状态为已发布，并发布内容更新事件到消息队列，供下游索引更新与相关统计链路异步消费，以降低主链路耦合并提升整体吞吐。

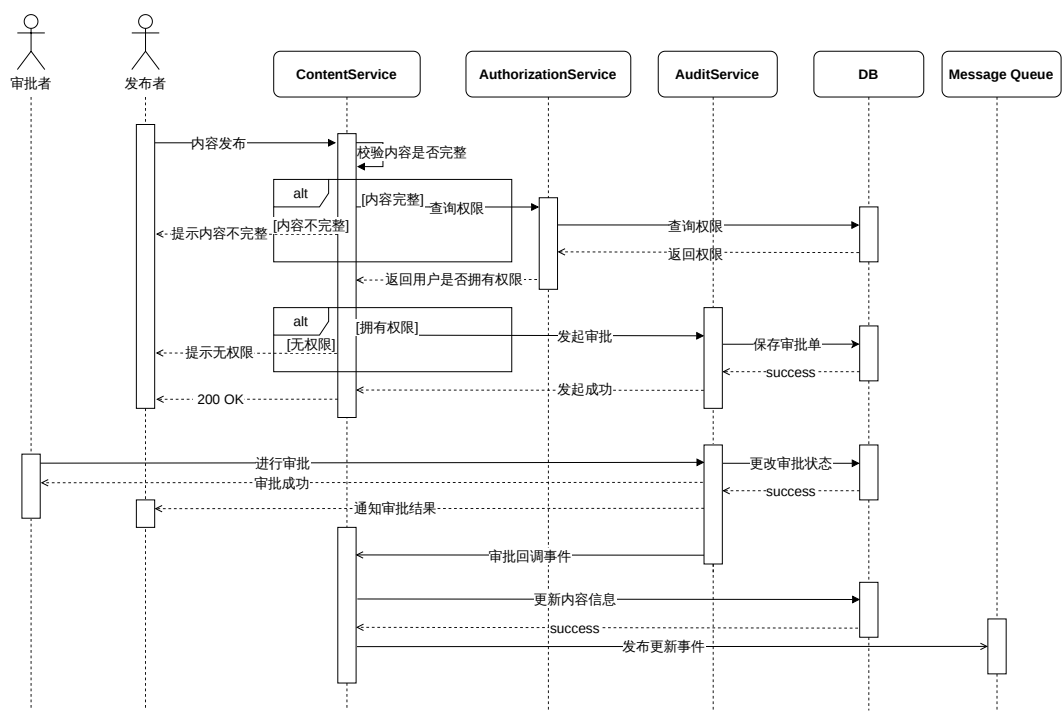


图 3-11: 内容发布流程时序图

3.4.2 内容应用模块设计

内容应用模块负责将管理好的内容资产转化为可被消费的服务，支持“页面化展示”与“智能问答”两大核心场景。

(1) 多形态内容前置处理与索引构建。为支撑高效检索与问答，内容在发布后需经过前置处理与索引构建流程。对于页面化展示与搜索，系统构建基于 ES 的全文索引（包含标题、摘要、元数据与权限信息）；对于智能问答，系统构建基于向量数据库的切片索引（将正文切分为语义完整的片段并向量化）。该过程由 MQ 消息驱动，确保索引与数据库状态准实时一致；在包含结构化字段或多模态信息的内容场景中，类似的前置归一与索引组织同样是提升检索可用性的关键环节^[2]。索引构建流程如图 3-12 所示。

(2) 页面化内容应用机制。面向门户与管理后台，提供基于 ES 索引的统一内容展示服务，包括类目导航、标签筛选、列表分页与全文检索。系统利用 ES 的倒排索引能力实现全量字段过滤（含权限控制），并结合数据库回查机制保障详情页的强一致性。

(3) 多通道召回。面向智能问答场景，系统采用“ES 全文检索 + Embedding 向量召回”的双路架构。全文通道侧重精确关键词匹配与元数据过滤，向量

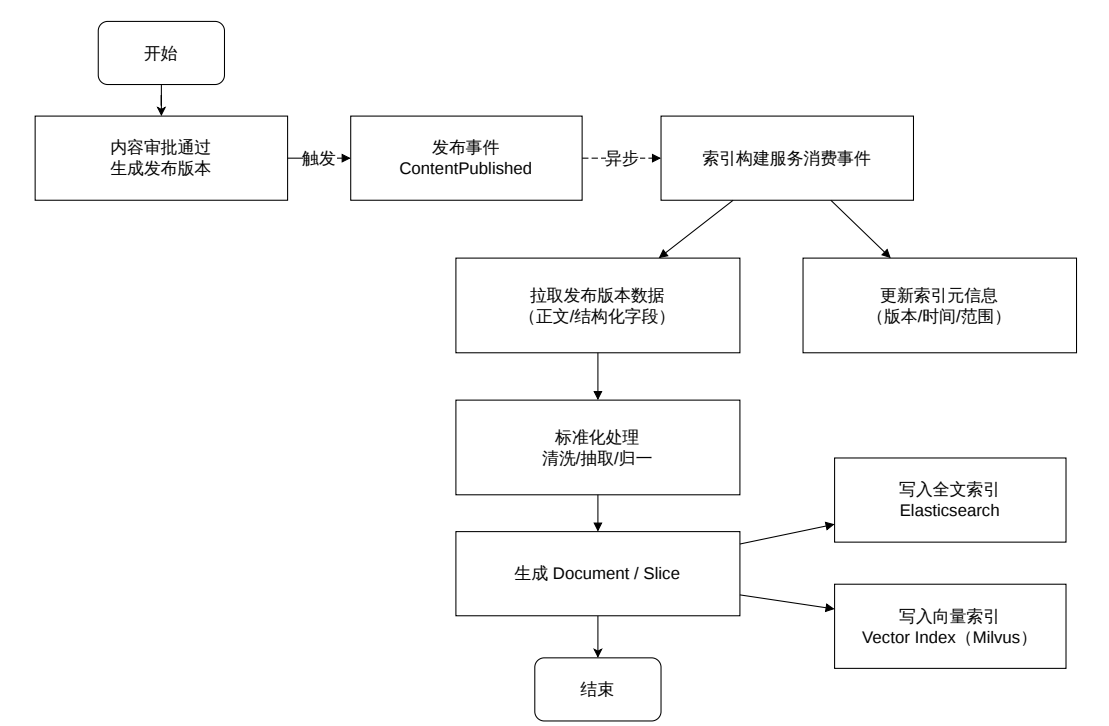


图 3-12: 索引构建流程图

通道侧重语义理解与长尾问法覆盖。两路召回结果经融合去重与重排后，输出 TopN 高质量切片作为回答依据，这类“多路召回 + 融合重排”的组合在工程实践中能够有效提升召回质量与可控性^{[2][3]}。

(4) 智能问答流程。系统将召回的证据切片注入大模型 Prompt，结合业务约束生成回答，并在输出中附带可溯源的引用标记。该流程实现了“问答-证据-治理”的闭环，有效降低了模型幻觉风险。完整的智能问答流程如图 3-13 所示。

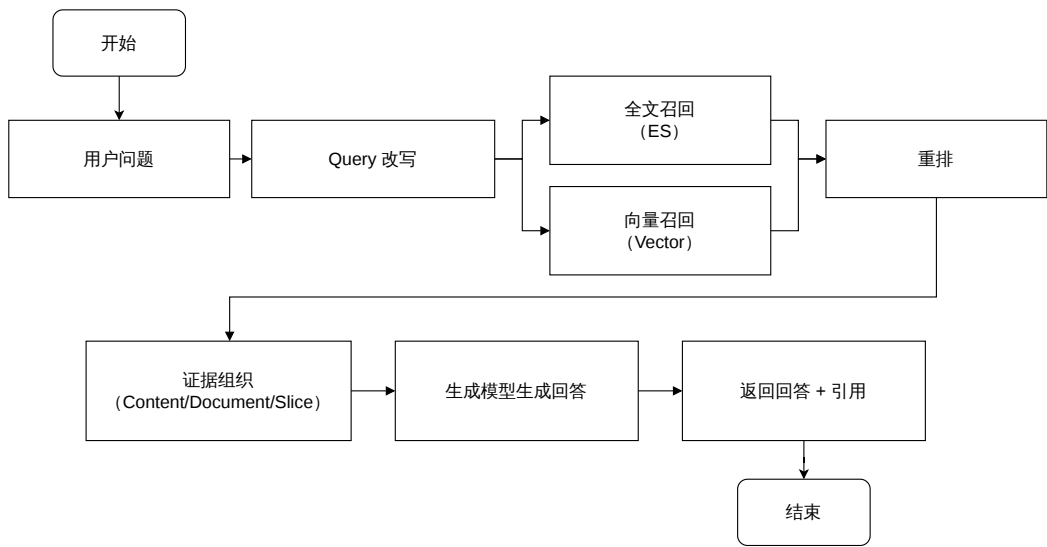


图 3-13: 智能问答流程图

3.4.3 内容治理模块设计

内容治理模块面向内容质量问题构建“发现问题—任务化闭环—复检回收”的治理体系，围绕重复度、歧义度与覆盖度三个治理维度，将质量问题从结果侧可观测化，并转化为可定位、可执行、可统计的治理工作，最终反哺内容资产与检索问答链路，形成持续演进机制。

（1）治理目标与度量。内容质量体系从内容覆盖、内容质量与效果评估三个方向建立指标口径，并结合客观质量与主观反馈实现质量度量。在内容覆盖方面，关注在给定阈值下用户原声是否能够召回到相关知识；在内容质量方面，关注内容重复与口径冲突带来的冗余与歧义，其中内容元数据的完整性与一致性等维度同样会影响质量评估口径^{[2][3]}；在效果评估方面，关注内容在检索与问答链路中的命中、引用与会话解决效果等指标，为治理优先级与策略调整提供依据^{[2][3]}。

（2）重复度/歧义度检测模型。重复度与歧义度检测的核心目标是降低内容冗余、消除逻辑冲突。系统设计了“粗召回 + 精判别”的两阶段检测模型：首先利用向量检索引擎快速筛选出高相似度的候选内容对，将计算范围从全量空间缩小至局部邻域；随后引入大语言模型作为裁判员，对候选内容对进行深层语义比对。若发现内容高度重叠则判定为重复，若发现同一主体下的描述存在逻辑冲突则判定为歧义。该模型在保证检测精度的同时，有效解决了大规模内容库两两比对的性能瓶颈问题^{[2][3]}。此外，在大规模数据场景中，引入近重复

序列搜索等工程化方法有助于进一步降低去重与相似性检索的成本^[21]。对于口径冲突等更复杂的矛盾检测，可结合形式化约束与大模型判别实现自动化识别^[21]。

重复度与歧义度检测的整体流程示意如图 3-14 所示。

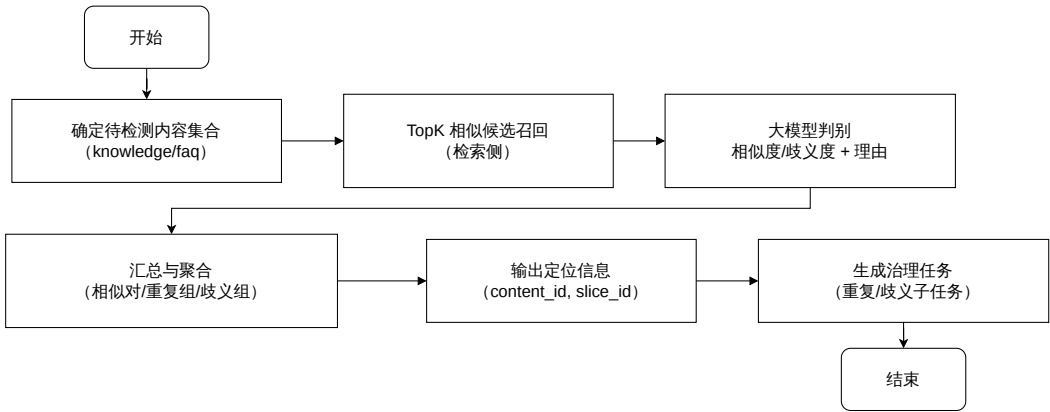


图 3-14: 重复度与歧义度模型检测流程示意图

(3) 覆盖度检测机制。覆盖度检测旨在持续发现知识盲区。系统采用“需求驱动”的检测机制，以用户的真实会话日志为输入：首先通过意图分类筛选出业务咨询类 Query，再结合语义聚类技术将零散的未命中 Query 聚合为“高频缺失问题簇”。在查询意图发现研究中，针对海量 Query 的序列聚类、聚类标签生成与聚类间探索式组织也被用于从无监督日志中抽取稳定主题，这与本平台通过问题簇识别知识缺口的思路具有一致性^[21]。通过计算代表性 Query 的召回率与回答质量，系统能够精准定位内容缺失点，并生成对应的覆盖度治理任务，推动知识库的针对性补充^[21]。

(4) 质量治理任务线上化。为将质量治理从离线跑数与人工推动升级为线上闭环，平台将治理过程抽象为主任务与子任务两层结构。主任务负责一次治理计算的配置与状态管理，包含任务名称、空间、会话周期与召回阈值等上下文信息；主任务进入计算中后生成治理任务列表，并在计算完成后对外展示结果。子任务主要包含覆盖任务与重复歧义任务两类：覆盖任务承载缺知识聚类结果，支持人工标记“无需补充/需要补充”并填写说明，系统每日巡检若发现已可召回到相关答案则自动完成；重复歧义任务承载重复对与歧义对结果，支持人工判断是否需要下线并完成处理标记，系统每日巡检若发现内容变更、下线或删除后不再存在重复/歧义，则自动完成。此外，针对算法误判等特殊场景，支持责任人申请“强制完成”，经审批通过后直接关闭任务，防止无效治

理。质量治理任务线上化的结构与关键流程如图 3-15所示。

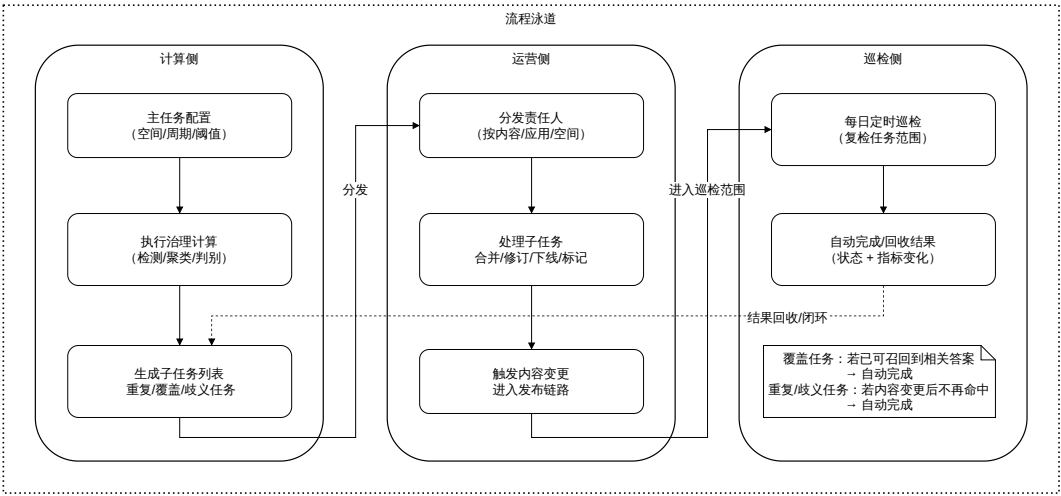


图 3-15: 质量治理任务线上化流程示意图

(5) 复检回收与链路一致性。治理任务处理通常会引发内容修订与再次发布，平台需要保证“内容资产—索引—问答结果”在治理后保持一致。对于需要修订的内容，治理处理应与内容管理的提审与发布流程打通，发布后由索引构建服务消费发布事件完成增量索引更新；随后对任务范围执行复检回收，确认质量问题是否消除，并将复检结果与指标变化回收至数据洞察体系，用于形成治理前后对比与策略迭代。

3.4.4 数据洞察模块设计

数据洞察模块为平台提供效果评估与问题定位能力，是“应用驱动生产、数据反哺治理”的关键闭环环节。该模块的实现链路采用“前端埋点与服务端事件采集 → 数仓统一加工与指标口径沉淀 → 洞察服务查询与权限控制 → 前端可视化与下钻明细”的模式：数据加工与指标口径由数仓侧负责，本平台侧重点在事件规范、采集链路、与数仓/BI 的对接以及查询服务建设^{[7][9]}。数据采集与指标生成链路示意如图 3-16所示。

(1) 数据采集。数据采集需要覆盖页面化消费链路 with 智能问答链路两类场景。在页面化链路中，平台采集曝光、点击、搜索输入与结果点击等行为数据；在问答链路中，平台采集问题输入、召回证据、生成结果与转人工等链路数据。为避免采集口径不一致导致分析偏差，平台对埋点事件与字段进行统一定义，例如 page_view、click、query_submit、rag_recall、llm_generate、hand-

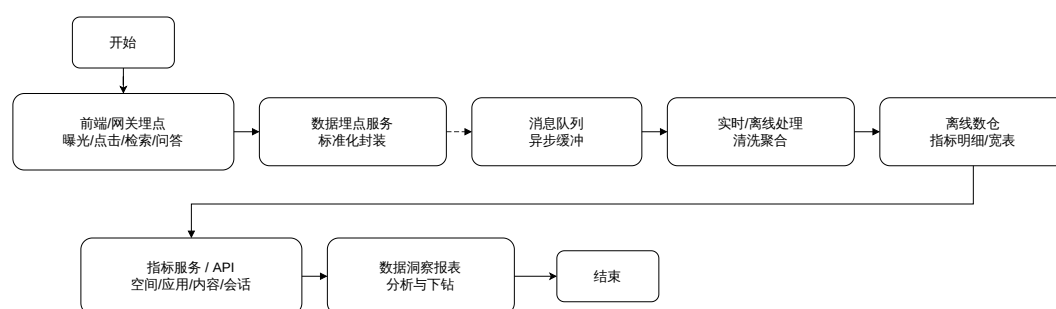


图 3-16: 数据采集与指标生成链路示意图

off_intent 与 handoff_success 等，并统一携带 app_id、space_id、session_id、trace_id 与 content_id 等公共维度字段，便于端到端追踪与明细还原。

为支撑事件的可扩展与跨链路关联，平台将事件抽象为“公共维度 + ext 扩展字段”的统一模型，并以 trace_id 与 session_id 建立会话与链路关联。

(2) 指标体系与分析模型设计。为全面评估平台运行效果，平台构建了覆盖页面、会话与内容三个视角的指标体系，并支持从聚合指标下钻至会话明细。页面视角关注页面访问量（PV）、用户量（UV）与点击率，评估内容门户的使用情况；会话视角以会话为单位，关注会话总量、模型识别解决率、意向转人工率与实际转人工率，评估智能问答链路的整体效能；内容视角关注内容在 RAG 链路中的被召回次数、被生成引用次数以及关联的转人工会话数，识别高价值内容与低效内容。围绕平台质量、用户满意度与持续使用意愿之间关系的研究也表明，访问与交互效果指标对于评估平台应用价值具有直接意义^[7]。此外，平台提供会话明细查询能力，支持追溯单个会话的原始 Query、改写结果、召回证据与生成回答，为问题定位与策略优化提供微观依据。

(3) 会话聚类机制设计。会话聚类是平台从统计视角识别高频问题与知识盲区的核心机制。该机制通过语义向量技术将海量零散的历史 Query 归并为有限的语义主题簇，将非结构化的日志转化为结构化的“问题簇”。聚类结果包含聚类中心（代表性 Query）、聚类规模（热度）以及聚类主题标签（由大模型总结生成）。通过会话聚类，运营人员可以快速发现高频且低解决率的头部问题，并将其作为覆盖度治理任务的输入，从而实现“数据反哺治理”的闭环^[7]。

3.5 数据库设计

图3-17所示为平台数据库的关键实体关系图，图中给出了内容、组织边界、权限控制、治理任务与数据洞察相关的核心实体及其关系。整体上，平台以内容（content）为核心业务对象，由空间（space）与应用（application）形成组织边界，由类目、标签与多形态内容子表支撑内容组织，由 RBAC、内容权限与审批表支撑访问控制与发布流程，由治理任务表支撑治理闭环，由数仓侧明细/指标表支撑数据洞察模块。

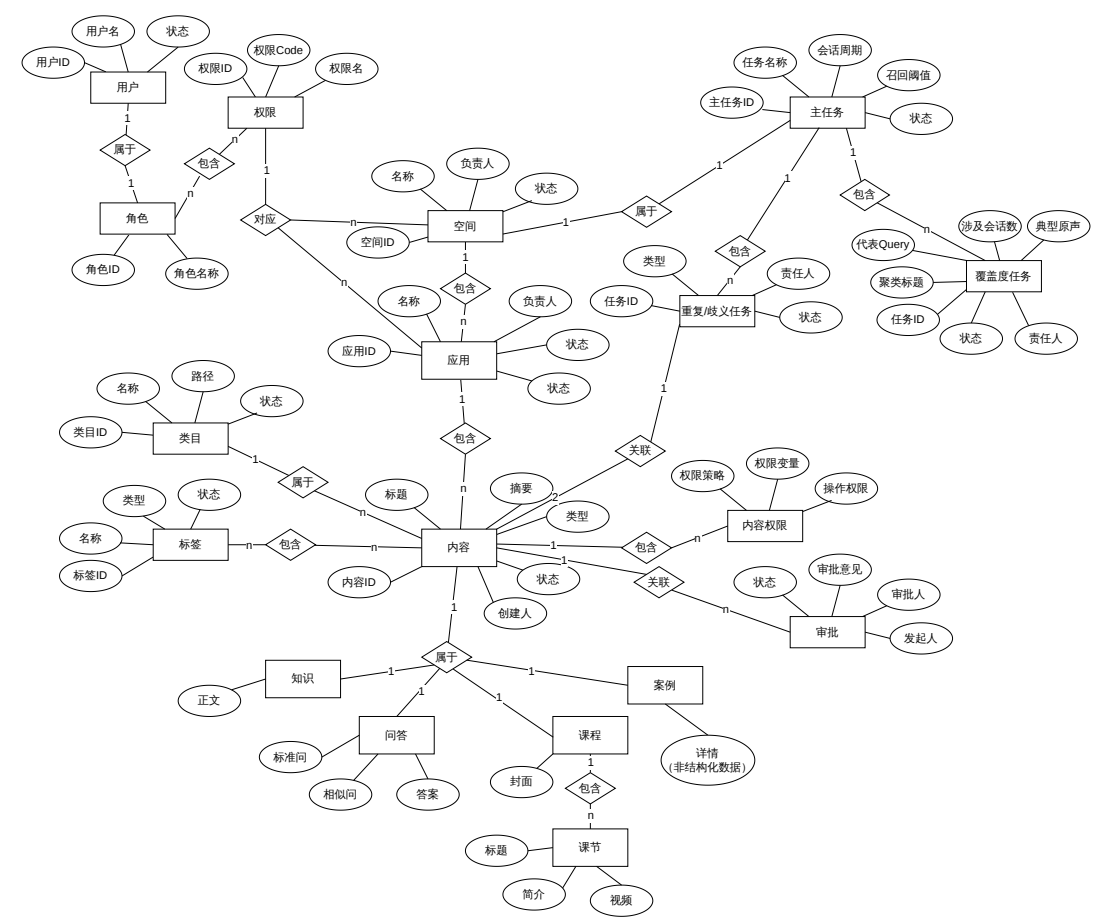


图 3-17: 数据库 E-R 图

下面结合各表的字段设计对关键数据对象进行说明。

(1) 空间与应用表。空间与应用是平台进行内容组织、权限隔离与场景划分的基础边界。空间表详情如表3-25所示，用于存储业务线级别的顶层组织信息，是应用归属、内容隔离和管理范围划分的基础。其中，id 为主键，name 用于标识空间名称，owner_id 用于记录空间负责人，status 用于控制

空间是否启用，`create_time` 与 `modify_time` 用于记录创建与变更时间，便于后续审计和运维管理。

表 3-25: 空间表 (space)

字段名	字段中文名	字段类型	说明
id	空间 ID	bigint	主键
name	空间名称	varchar(255)	业务标识
owner_id	负责人 ID	bigint	平台用户标识
status	状态	varchar(50)	ENABLED/DISABLED
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

应用表详情如表 3-26所示，用于描述空间下的具体业务载体，不同应用可以承载门户、智能助手等不同内容消费场景。表中 `space_id` 用于建立应用与空间之间的从属关系，`name` 用于标识应用名称，`type` 用于区分应用类型，`status` 用于控制应用启停，`create_time` 与 `modify_time` 则用于记录应用生命周期中的时间信息。空间表与应用表共同定义了平台的组织边界，也为后续内容归属、权限控制和数据统计提供了统一作用域。

表 3-26: 应用表 (application)

字段名	字段中文名	字段类型	说明
id	应用 ID	bigint	主键
space_id	空间 ID	bigint	外键，关联 space
name	应用名称	varchar(255)	业务标识
type	应用类型	varchar(50)	门户/智能助手等
status	状态	varchar(50)	ENABLED/DISABLED
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

(2) 统一内容表。内容表是整套数据库设计中的核心主表，表结构如表 3-27所示，用于统一记录各类内容的归属信息、基础元数据和生命周期状态。表中 `space_id` 与 `app_id` 分别标识内容所属空间和创建应用，`content_type` 用于区分知识、问答、课程和案例等不同内容类型，`title` 与 `summary` 承担检索、展示和列表摘要等作用，`status` 用于标识草稿、待审、已发布、下线与删除等状态，`create_time` 与 `modify_time` 用于记录

内容的创建与更新时间。通过该表，平台可以在统一模型下管理多形态内容，并将共性字段从具体子表中抽离出来。

表 3-27: 内容表 (content)

字段名	字段中文名	字段类型	说明
id	内容 ID	bigint	主键
space_id	空间 ID	bigint	外键，关联 space
app_id	归属应用 ID	bigint	内容创建所在应用
content_type	内容类型	varchar(50)	knowledge/faq/course/case
title	标题	varchar(255)	
summary	摘要	varchar(1024)	
status	状态	varchar(50)	DRAFT / PENDING_REVIEW / PUBLISHED / OFFLINE / DELETED
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

(3) 多形态内容子表。不同内容类型的差异字段存储在对应子表中，以保持统一内容模型的同时兼顾字段扩展性。

知识内容表详情如表 3-28 所示，用于存储知识类内容的正文信息。该表通过 `content_id` 与统一内容表关联，`body` 字段承载富文本或 Markdown 等正文内容，是后续内容切片、语义检索和问答引用的主要文本来源。由于知识类内容以长文本为主，因此将正文单独拆分到子表中，有助于控制主表宽度并提升扩展性。

表 3-28: 知识内容表 (knowledge)

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
body	正文	longtext	富文本/Markdown 等

问答内容表详情如表 3-29 所示，用于存储 FAQ 类内容的核心问答对。表中 `question` 用于记录标准问题，`similar_questions` 用于维护相似问集合，以支撑问法归并与召回扩展，`answer` 用于记录标准答案，以上字段均通过 `content_id` 与内容主表关联。该表使平台能够在统一内容模型下对问答内容进行结构化管理，并支撑标准问匹配和答案直接返回等场景。

表 3-29: 问答内容表 (faq)

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
question	标准问题	varchar(1024)	
similar_questions	相似问	varchar(1024)	可选
answer	答案	longtext	

课程内容表详情如表 3-30 所示，用于存储课程类内容的扩展信息。该表通过 `content_id` 与内容主表建立一对一关系，`cover_url` 用于记录课程封面等展示属性，便于在课程列表、推荐卡片等场景中进行可视化呈现。由于课程的展示形态与普通知识内容存在差异，因此平台通过独立子表补充其专属字段。

表 3-30: 课程内容表 (course)

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
cover_url	封面	varchar(1024)	可选

课节表详情如表 3-31 所示，用于细化课程内部结构，将一门课程拆分为多个可独立编排和展示的课节单元。表中 `course_id` 用于关联所属课程，`title` 记录课节标题，`lesson_no` 用于定义课节顺序，`content` 用于承载课节文本、字幕或讲义内容。该表支持课程内容按章节组织，也便于后续按课节维度进行展示和检索。

表 3-31: 课节表 (course_lesson)

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
course_id	课程 ID	bigint	外键，关联 course
title	课节标题	varchar(255)	
lesson_no	课节序号	int	课程内排序
content	课节内容	longtext	文本/字幕等

案例详情集合详情如表 3-32 所示。由于案例类内容的详情字段差异较

大、变化较快，平台将其扩展信息聚合存储在 MongoDB 集合中。集合中 `content_id` 用于关联统一内容表中的案例主记录，`detail` 以 JSON 形式承载其余结构化与非结构化扩展字段，`_id` 作为 MongoDB 主键保证文档唯一性。该设计有利于在保持主数据统一管理的同时，为案例类内容提供更灵活的字段扩展能力。

表 3-32: 案例详情集合 (MongoDB: case_detail)

字段名	字段中文名	字段类型	说明
<code>_id</code>	主键 ID	ObjectId	MongoDB 主键
<code>content_id</code>	内容 ID	bigint	关联 content.id
<code>detail</code>	详情字段	json	其余结构化/非结构化字段

(4) 类目与标签表。为支撑内容在页面化展示与检索场景下的组织与聚合，平台提供类目树与标签体系，并通过关联表建立内容与类目、标签之间的绑定关系。

标签表详情如表 3-33 所示，用于维护应用维度下的标签字典，是内容打标、筛选与聚合展示的重要基础。表中 `app_id` 限定标签的作用域，`name` 用于记录标签名称，`type` 用于区分标签类别，`status` 用于控制标签是否生效，时间字段用于记录标签创建和修改时间。通过该表，平台可以在应用范围内维护统一的标签语义。

表 3-33: 标签表 (tag)

字段名	字段中文名	字段类型	说明
<code>id</code>	标签 ID	bigint	主键
<code>app_id</code>	应用 ID	bigint	标签所属应用
<code>name</code>	标签名称	varchar(255)	
<code>type</code>	标签类型	varchar(50)	
<code>status</code>	状态	varchar(50)	ENABLED/DISABLED
<code>create_time</code>	创建时间	datetime	
<code>modify_time</code>	更新时间	datetime	

内容-标签关联表详情如表 3-34 所示，用于建立内容与标签之间的多对多绑定关系。表中 `content_id` 与 `tag_id` 分别关联内容表和标签表，`create_time` 用于记录绑定建立时间。该表使同一内容可以被打上多个标

签，也使同一标签可以服务于多条内容，从而支撑内容检索、标签聚合和推荐分析等功能。

表 3-34: 内容-标签关联表 (content_tag_ref)

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
tag_id	标签 ID	bigint	外键，关联 tag
create_time	创建时间	datetime	

类目表详情如表 3-35 所示，用于维护应用内的树状类目结构，是平台进行目录化展示与导航管理的基础。表中 **parent_id** 用于构建父子层级关系，**name** 用于记录类目名称，**path** 用于保存祖先链路编码，便于快速定位和批量查询，**status** 用于控制类目是否启用，时间字段则用于记录类目维护过程中的变更信息。类目表与标签表相互补充，分别承担层级组织与语义标注的职责。

表 3-35: 类目表 (category)

字段名	字段中文名	字段类型	说明
id	类目 ID	bigint	主键
app_id	应用 ID	bigint	类目所属应用
parent_id	父类目 ID	bigint	根节点为 0 或空
name	类目名称	varchar(255)	
path	路径	varchar(1024)	祖先链路编码，可选
status	状态	varchar(50)	ENABLED/DISABLED
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

内容-类目关联表详情如表 3-36 所示，用于记录内容与类目之间的归属关系，支撑内容按目录体系进行管理、导航展示与聚合查询。表中 **content_id** 和 **category_id** 分别关联内容与类目，**create_time** 用于记录建立关联的时间。通过该表，平台可以灵活维护内容在类目体系中的挂载关系，而无需在内容主表中直接固化复杂层级字段。

表 3-36: 内容-类目关联表 (content_category_ref)

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
content_id	内容 ID	bigint	外键, 关联 content
category_id	类目 ID	bigint	外键, 关联 category
create_time	创建时间	datetime	

(5) 平台 RBAC 表。平台管理权限采用 RBAC 模型落地, 通过用户、角色、权限及其关联表表达授权关系, 并通过作用域字段实现全局、空间和应用级授权隔离。

用户表详情如表 3-37 所示, 用于存储平台内的用户身份信息, 是权限绑定和操作审计的基础。表中 **id** 为用户主键, **name** 用于展示用户名或账号标识, **status** 用于控制用户是否可用。该表保存的是平台侧管理所需的用户基础信息, 为角色分配和审批流转等功能提供主体标识。

表 3-37: 用户表 (user)

字段名	字段中文名	字段类型	说明
id	用户 ID	bigint	主键
name	用户名	varchar(255)	
status	状态	varchar(50)	ENABLED/DISABLED

角色表详情如表 3-38 所示, 用于抽象平台中的职责类别, 是权限聚合与授权配置的核心载体。表中 **id** 为角色主键, **name** 用于定义平台负责人、业务负责人、业务运营等不同职责类型。通过角色这一中间层, 平台可以将多个原子权限进行组合, 降低直接面向用户授权的复杂度。

表 3-38: 角色表 (role)

字段名	字段中文名	字段类型	说明
id	角色 ID	bigint	主键
name	角色名称	varchar(255)	平台负责人/业务负责人/业务运营等

权限表详情如表 3-39 所示, 用于维护系统中的原子权限点。表中 **code** 为权限码, 例如 **SPACE_CREATE**、**CONTENT_PUBLISH**, 用于在程序中进行精确鉴权; **name** 用于记录权限的可读描述, 便于后台管理和权限配置。角色表

与权限表共同构成 RBAC 模型中的能力定义层。

表 3-39: 权限表 (permission)

字段名	字段中文名	字段类型	说明
id	权限 ID	bigint	主键
code	权限码	varchar(255)	如 SPACE_CREATE、CONTENT_PUBLISH
name	权限名称	varchar(255)	可读描述

角色-权限关联表详情如表 3-40 所示，用于建立角色与权限之间的多对多映射关系。表中 `role_id` 与 `permission_id` 分别关联角色和权限，使一个角色可以聚合多项权限，也允许同一权限被多个角色复用。该表是将权限定义转化为可配置角色能力的关键纽带。

表 3-40: 角色-权限关联表 (role_permission)

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
role_id	角色 ID	bigint	外键，关联 role
permission_id	权限 ID	bigint	外键，关联 permission

用户-角色绑定表详情如表 3-41 所示，用于记录用户在不同作用域下拥有的角色，是 RBAC 模型真正落地到用户授权的核心表。表中 `user_id` 和 `role_id` 分别关联用户与角色，`scope_type` 用于区分全局、空间和应用三类授权范围，`scope_id` 用于标识具体的空间或应用对象。通过该表，平台能够实现同一用户在不同空间或应用下拥有不同管理职责的分层授权能力。

表 3-41: 用户-角色绑定表 (user_role_binding)

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	主键
user_id	用户 ID	bigint	外键，关联 user
role_id	角色 ID	bigint	外键，关联 role
scope_type	作用域类型	varchar(20)	GLOBAL/SPACE/APP
scope_id	作用域 ID	bigint	空间 ID/应用 ID，可为空

(6) 内容权限表 (`content_auth`)。内容级权限以“内容 + 策略 + 变量 + 权限码”的方式表达，其表结构如表 3-42 所示。该表用于表达内容粒度的可见性

与可操作性约束，是对平台级 RBAC 模型的进一步补充。表中 `content_id` 用于标识被授权的具体内容，`auth_tactic` 用于定义权限策略类型，如员工匹配、部门匹配或角色匹配，`auth_variable` 用于记录策略变量值，`auth_code` 用于定义可执行操作集合。通过该表，平台能够在统一角色权限之外，对特定内容设置更细粒度的访问控制规则。

表 3-42: 内容权限表 (content_auth)

字段名	描述	类型	备注
id	主键 ID	bigint	自增主键
content_id	内容 ID	bigint	被授权对象的唯一标识
auth_tactic	权限策略	varchar(50)	策略类型（如员工匹配、部门匹配等）
auth_variable	权限变量	varchar(200)	策略变量值（如员工 ID、部门 ID 等）
auth_code	操作权限	varchar(20)	权限码集合（如 r/w/u 等）

（7）内容审批表。内容审批表详情如表 3-43 所示，用于记录内容从提审到审批完成的全过程信息，是内容发布流程中的关键过程表。表中 `content_id` 用于关联被审批内容，`submitter_id` 和 `reviewer_id` 分别记录提交人与审批人，`status` 用于标识待审、通过和驳回等审批状态，`comment` 用于补充审批意见，`create_time` 与 `modify_time` 用于追踪流转时间。该表为内容发布、审批留痕和责任追踪提供了结构化依据。

表 3-43: 内容审批表 (content_approval)

字段名	字段中文名	字段类型	说明
id	审批单 ID	bigint	主键
content_id	内容 ID	bigint	外键，关联 content
submitter_id	提交人 ID	bigint	平台用户标识
reviewer_id	审批人 ID	bigint	平台用户标识
status	审批状态	varchar(50)	PENDING / APPROVED / REJECTED
comment	审批意见	varchar(1024)	可选
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

（8）治理任务表。内容治理模块将检测结果以治理任务形式线上化流

转，主任务负责配置与状态管理，重复歧义任务与覆盖度任务承载具体治理问题。

主任务表详情如表 3-44 所示，用于记录一次治理任务的整体配置与执行状态，是治理流程的主控表。表中 `app_id` 限定任务所属应用范围，`name` 用于标识任务名称，`session_period` 用于定义统计窗口，`recall_threshold` 用于配置召回或覆盖阈值，`status` 用于标识任务处于计算中、计算完成或已完成等阶段，时间字段则用于记录任务执行过程。该表统一管理一次治理计算的上下文信息。

表 3-44: 主任务表 (govern_main_task)

字段名	字段中文名	字段类型	说明
id	主任务 ID	bigint	主键
name	任务名称	varchar(255)	
app_id	应用 ID	bigint	外键，关联 application
session_period	会话周期	varchar(50)	天/周/月等
recall_threshold	召回阈值	decimal(10,4)	相似度/覆盖阈值
status	状态	varchar(50)	计算中/计算完成/已完成等
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

重复歧义任务表详情如表 3-45 所示，用于记录重复或歧义内容对的治理问题。表中 `main_task_id` 用于关联所属主任务，`task_type` 用于区分重复和歧义两类问题，`content_id_a` 与 `content_id_b` 用于标识待处理的内容对，`status` 用于记录任务状态，`assignee_id` 与 `result` 分别记录责任人与处理结论。通过该表，平台可以对重复和歧义问题进行逐条分派、处理和回收。

覆盖度任务表详情如表 3-46 所示，用于记录高频但覆盖不足的问题簇，是覆盖度治理的主要承载表。表中 `cluster_title` 用于概括缺失知识主题，`representative_query` 用于记录代表性问法，`example_queries` 用于保存典型原声样本，`query_count` 用于衡量问题影响范围，`status` 与 `assignee_id` 则用于任务分派、处理和复检回收。该表将会话聚类结果转化为可运营、可跟踪的治理对象。

表 3-45: 重复歧义任务表 (govern_dup_amb_task)

字段名	字段中文名	字段类型	说明
id	任务 ID	bigint	主键
main_task_id	主任务 ID	bigint	外键, 关联 govern_main_task
task_type	子任务类型	varchar(50)	DUPLICATE/AMBIGUOUS
content_id_a	内容 A	bigint	外键, 关联 content
content_id_b	内容 B	bigint	外键, 关联 content
status	状态	varchar(50)	TODO / DONE 等
assignee_id	责任人	bigint	平台用户标识
result	处理结果	varchar(255)	下线/保留/合并等
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

表 3-46: 覆盖度任务表 (govern_coverage_task)

字段名	字段中文名	字段类型	说明
id	任务 ID	bigint	主键
main_task_id	主任务 ID	bigint	外键, 关联 govern_main_task
cluster_title	聚类标题	varchar(255)	缺失知识簇的主题摘要
representative_query	代表性 Query	varchar(1024)	用于召回检测的代表性问法
query_count	涉及会话数	int	该问题簇包含的历史会话数量
example_queries	典型原声	json	簇内典型用户原声列表, 用于辅助治理
status	状态	varchar(50)	TODO / DONE 等
assignee_id	责任人	bigint	平台用户标识
create_time	创建时间	datetime	
modify_time	更新时间	datetime	

(9) 数据洞察数仓表。数据洞察模块的指标计算与口径沉淀主要在数仓侧完成, 平台侧通过查询接口对接数仓侧明细表与指标宽表。

事件明细表详情如表 3-47 所示, 用于统一沉淀页面浏览、点击、检索提交与召回等原始行为, 是数据洞察链路的底层明细来源。表中 `event_name` 用于标识事件类型, `app_id` 与 `space_id` 用于标识业务作用域, `user_id` 与 `visitor_id` 用于区分登录用户与匿名访客, `session_id` 与 `trace_id` 用于建立会话和链路关联, `ext` 用于承载事件特有扩展字段。通过该表, 平台可

以为后续聚合统计、链路还原和问题排查提供完整原始数据。

表 3-47: 事件明细表 (ODS: di_event_ods)

字段名	字段中文名	字段类型	说明
event_name	事件名	string	page_view/click/query_submit/rag_recall 等
app_id	应用 ID	bigint	
space_id	空间 ID	bigint	
user_id	用户 ID	bigint	可为空
visitor_id	匿名访客 ID	string	可为空
session_id	会话 ID	string	
trace_id	链路标识	string	
content_id	内容 ID	bigint	可为空
timestamp	时间戳	datetime	
channel	渠道	string	可为空
device	设备	string	可为空
ext	扩展字段	json	事件特有字段

页面指标宽表详情如表 3-48 所示，用于按日汇总页面访问与点击表现，是评估页面化内容应用效果的基础统计表。表中 dt 为统计日期分区字段，app_id 与 space_id 用于限定统计范围，pv 和 uv 分别衡量访问规模，click_cnt 与 ctr 用于评估交互表现。该表为页面访问趋势分析、应用运营评估和内容投放优化提供直接依据。

表 3-48: 页面指标宽表 (DWS: di_page_metric_d)

字段名	字段中文名	字段类型	说明
dt	统计日期	date	分区字段
app_id	应用 ID	bigint	
space_id	空间 ID	bigint	
page_id	页面 ID	string	
pv	PV	bigint	
uv	UV	bigint	
click_cnt	点击数	bigint	
ctr	点击率	decimal(10,4)	可选

内容指标宽表详情如表 3-49 所示，用于汇总内容在检索与问答链路中的使用效果，是从内容视角评估平台运行质量的重要宽表。表中 content_

`id` 与 `content_type` 用于定位具体内容对象, `recall_session_cnt` 与 `generate_session_cnt` 用于衡量内容被召回与被生成引用的会话规模; `handoff_intent_session_cnt` 与 `handoff_session_cnt` 用于衡量内容引发转人工的程度。该表可以直接支撑高价值内容识别、低效内容发现和治理优先级排序。

表 3-49: 内容指标宽表 (DWS: `di_content_metric_d`)

字段名	字段中文名	字段类型	说明
<code>dt</code>	统计日期	<code>date</code>	分区字段
<code>app_id</code>	应用 ID	<code>bigint</code>	
<code>space_id</code>	空间 ID	<code>bigint</code>	
<code>content_id</code>	内容 ID	<code>bigint</code>	
<code>content_type</code>	内容类型	<code>string</code>	<code>knowledge / faq / course / case</code>
<code>category_id</code>	类目 ID	<code>bigint</code>	可为空
<code>recall_session_cnt</code>	被召回会话数	<code>bigint</code>	
<code>generate_session_cnt</code>	被生成会话数	<code>bigint</code>	
<code>handoff_intent_session_cnt</code>	意向转人工会话数	<code>bigint</code>	
<code>handoff_session_cnt</code>	转人工会话数	<code>bigint</code>	

会话聚类指标宽表详情如表 3-50 所示, 用于汇总问题簇的热度和解决效果, 是从主题簇视角观察高频问题的重要数据表。表中 `cluster_id` 与 `cluster_title` 用于标识聚类主题, `session_cnt` 与 `query_cnt` 用于衡量簇规模, `solve_rate`、`handoff_intent_rate` 与 `handoff_rate` 用于评估簇内问题的解决程度和转人工情况。该表能够帮助运营人员快速识别高频且待优化的问题簇。

会话明细表详情如表 3-51 所示, 用于保留单次问答会话的检索、回答与转人工等关键信息, 是问题排查和效果复核的重要明细表。表中 `session_id` 用于标识单次会话, `cluster_id` 用于关联问题簇, `raw_query` 与 `rewritten_query` 分别记录用户原始问题和改写结果, `recalled_contents` 用于记录召回证据, `answer` 用于记录生成回复, `handoff_intent` 与 `handoff_success` 用于记录转人工意向和结果, `session_time` 用于标识会话发生时

表 3-50: 会话聚类指标宽表 (DWS: di_session_cluster_d)

字段名	字段中文名	字段类型	说明
dt	统计日期	date	分区字段
app_id	应用 ID	bigint	
space_id	空间 ID	bigint	
cluster_id	聚类 ID	string	
cluster_title	聚类标题	string	
session_cnt	会话数	bigint	
query_cnt	Query 数	bigint	
solve_rate	解决率	decimal(10,4)	模型识别解决率
handoff_intent_rate	意向转人工率	decimal(10,4)	
handoff_rate	实际转人工率	decimal(10,4)	

间。该表为链路还原、问题定位和聚类回溯提供了最细粒度的数据支撑。

表 3-51: 会话明细表 (DWD: di_session_detail)

字段名	字段中文名	字段类型	说明
session_id	会话 ID	string	主键之一
cluster_id	聚类 ID	string	可为空
raw_query	用户原声	string	可脱敏
rewritten_query	改写 Query	string	可脱敏
recalled_contents	召回内容	json	含 content_id&title 等
answer	回复内容	string	可脱敏
handoff_intent	意向转人工	tinyint	0/1
handoff_success	是否转人工	tinyint	0/1
user_id	用户 ID	bigint	可脱敏
user_type	用户类型	string	内部 / 外部 / 匿名等
session_time	会话时间	datetime	

3.6 本章小结

本章围绕面向广告与商业化场景的内容应用与管理平台展开分析与设计。首先从业务与系统视角给出了平台的整体定位与关键链路，明确平台由内容管理、内容应用、内容治理与数据洞察四个核心域构成，并以统一内容模型、权限体系与事件机制实现协同。随后在需求分析中按照模块给出了核心用例与用例描述表，明确平台在页面展示、内容搜索与智能问答场景下的能力边界，以及治理闭环与效果评估的关键需求。在总体设计与模块设计部分，本章说明了内容发布后的事件驱动机制与索引构建策略，并给出内容切片、多通道召回与治理任务线上化等关键实现思路。最后在数据库设计部分对平台的权限相关核心数据对象与存储选型进行了说明，为后续系统实现与效果评估奠定基础。

第四章 内容应用与管理平台的实现

4.1 内容管理模块的实现

内容管理模块是平台的内容资产底座，承担内容生产、组织、权限与发布等职责，是后续内容应用与内容治理的数据输入来源。实现层面，本节按“内容组织体系—多形态内容管理—权限与审批发布”的顺序展开，说明模块如何围绕统一内容模型完成业务归属组织、多形态内容生产维护以及对外发布口径控制。内容管理模块的关键服务端接口定义如表 4-1 所示。

表 4-1: 内容管理模块接口表

服务名称	接口名称	功能描述
内容组织体系	createSpace	创建空间，作为业务归属与成员管理边界。
	createApp	在空间内创建应用并维护场景配置与内容范围。
	upsertCategory / moveCategory	维护应用内类目树结构，支撑目录导航与组织管理。
	upsertTag	维护应用内标签体系，支撑跨类目聚合与筛选。
多形态内容管理	createContent / updateContent	创建与编辑内容元数据，并维护多形态内容的结构与正文。
	saveDraft / getDraft	保存草稿并获取可编辑版本，支撑内容生产过程的迭代。
	getDetail	按内容形态加载详情正文或结构字段并组装统一详情对象。
权限与审批发布	checkPermission / getContentAuth	基于 RBAC 与 content_auth 进行访问控制与策略查询。
	submitReview / approve / reject	发起提审、审批或驳回，驱动内容状态流转。
	publish / offline	发布生成对外可用的发布版本，或下线回收内容。

表 4-1 中列出的接口覆盖了内容管理模块的三条主链路：内容组织体系（空间/应用/类目/标签）、多形态内容管理（创建/编辑/草稿/详情）、以及权

限与审批发布（RBAC/content_auth 与发布状态机）。通过这三条主链路，平台在管理侧完成内容的归属与组织、多形态内容的生产与维护，以及对外可用口径的控制。

4.1.1 内容组织体系

面对海量内容，平台将内容组织抽象为“空间—应用—类目—标签”四级体系，用于统一表达业务归属、消费场景与内容组织方式。空间与应用用于划定数据与权限边界；类目与标签用于在应用内组织内容，支撑目录导航与灵活聚合。

（1）空间与应用隔离。平台以空间（Space）与应用（App）作为组织与管理边界：空间用于承载业务归属与成员管理，应用用于定义具体的内容消费场景与配置集合（如可用内容形态、默认权限策略与审批流模版）。在工程实现上，内容对象统一归属到特定空间与应用，使内容管理、权限控制与发布口径能够在明确范围内进行约束与治理。

（2）类目树与标签体系。在应用内部，平台提供结构化与扁平化两种组织方式。类目树采用路径枚举（Path Enumeration）模型存储，每个节点维护 `parent_id` 与 `path`（如 0/10/105），支持 $O(1)$ 复杂度的子树查询与路径解析，为前端提供高性能的目录导航。标签体系则采用扁平集合管理，支持多值绑定与灵活聚合。

这一多维组织体系最终通过统一内容模型落地：每条内容记录均冗余存储 `space_id`、`app_id`、`category_id` 与 `tag_ids`，确保了内容资产在存储层具备清晰的业务归属与属性特征，为后续的权限控制与高效分发奠定了数据基础。

图4-1所示为应用管理的效果图。在页面的上半部分，可以通过关键词、空间名称、应用状态进行筛选应用。在页面的中间部分，则展示了筛选后的所有应用，并展示了应用名称、ID、发布状态、创建人、更新时间等信息，通过应用管理界面，可以快速地定位到用户所需的应用中。

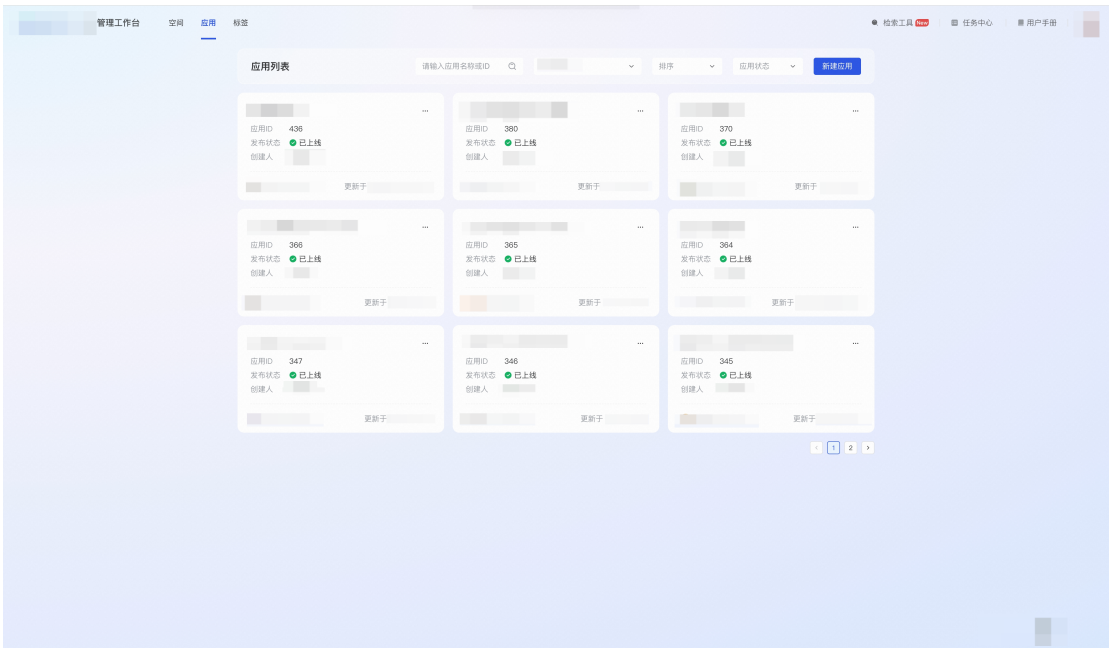


图 4-1: 应用管理效果图

4.1.2 多形态内容管理

多形态内容统一建模是内容管理模块实现的基础。平台将 knowledge、faq、course、case 四类内容抽象为统一的内容实体 content，并在子表中分别维护不同形态的结构与正文字段，从而实现“统一元数据口径、差异字段独立演进、发布状态可控”的工程目标。其核心实现要点包括统一元数据字段设计、多形态差异字段承载、面向发布的状态约束以及跨链路可回溯的内容标识体系。

本节围绕“内容创建与编辑、版本链维护、状态流转约束”给出实现说明，并通过关键实现片段支撑论述。内容管理链路的状态流转可与图 3-10 对应理解。

(1) 统一内容实体与多形态子表。平台在关系型数据库中维护 content 表与多形态子表：content 表记录空间、归属应用、类型、标题、摘要、状态等元数据；不同内容形态的正文与结构字段落在对应子表中，例如 knowledge.body、faq.question/answer、course_lesson.content。该设计使列表、权限、检索与治理能够基于统一的 content 标识协同，同时允许不同形态按需演进字段结构。

(2) 内容创建与草稿维护。内容创建与编辑保存的核心是保证元数据表与子表的写入一致性。创建时，服务端在同一事务内写入 content 元数据，并根

据 `content_type` 写入对应子表的正文或结构字段；草稿保存时，更新 `content` 元数据并同步更新子表内容，使得管理端能够以“草稿”为口径持续迭代内容。内容创建与草稿保存的关键实现片段如图 4-2 所示。

```
1 public Long createContent(CreateReq req) {
2     Long contentId = contentDao.insertContent(req.spaceId, req.appId, req.
        type, req.title, req.summary, "DRAFT");
3     if ("knowledge".equals(req.type)) {
4         knowledgeDao.insert(contentId, req.body);
5     } else if ("faq".equals(req.type)) {
6         faqDao.insert(contentId, req.question, req.answer);
7     } else {
8         // 委托给其他形态的DAO处理 (course, case, etc.)
9         contentExtensionService.saveExtension(req.type, contentId, req.
            extensionData);
10    }
11    return contentId;
12 }
13
14 public Long saveDraft(SaveDraftReq req) {
15     contentDao.updateMeta(req.contentId, req.title, req.summary);
16     if ("knowledge".equals(req.type)) {
17         knowledgeDao.updateBody(req.contentId, req.body);
18     } else if ("faq".equals(req.type)) {
19         faqDao.update(req.contentId, req.question, req.answer);
20     }
21     return req.contentId;
22 }
23 }
```

图 4-2: 内容创建与版本管理关键代码

(3) 内容详情读取。管理端在查看内容详情时，服务端先加载 `content` 元数据（空间、归属应用、类型、状态等），再按 `content_type` 读取对应子表的正文或结构字段并组装为统一的详情对象。例如 `knowledge` 读取正文 `body`，`faq` 读取 `question/answer`，`course` 聚合课节列表，`case` 从 MongoDB 读取 `detail(json)`。该机制使得多形态内容能够以统一接口对外提供详情能力，并确保详情口径与内容状态及权限裁决保持一致。

(4) 状态与发布口径约束。内容生命周期至少包含草稿、待审批、已发布、已下线等状态，平台在状态机约束下实现编辑、提审、审批、发布与下线等操作。实现中，读链路对外仅返回已发布状态内容；写链路在提审时冻结变更并进入待审批状态，审批通过后更新内容状态为已发布并触发发布事件，驱动下游索引更新与数据回收，避免“草稿可见”导致的口径漂移。

图 4-3 所示为文章管理的列表页效果图。在页面靠左侧，展示了应用的类目树结构，通过类目树可以方便地对文章进行分类与筛选。在页面上边，则支

持了对文章的筛选操作，包括关键词、ID、审核状态、发布状态等。在页面中间，展示了应用下的文章列表，每个文章展示了标题、类目节点、发布状态、审核状态、标签、可见范围、创建人等信息，用户可以通过点击文章进行详情查看。通过此界面，用户可以直观高效地对文章进行管理与操作。问答、课程、案例的管理界面类似，这里不再展开。

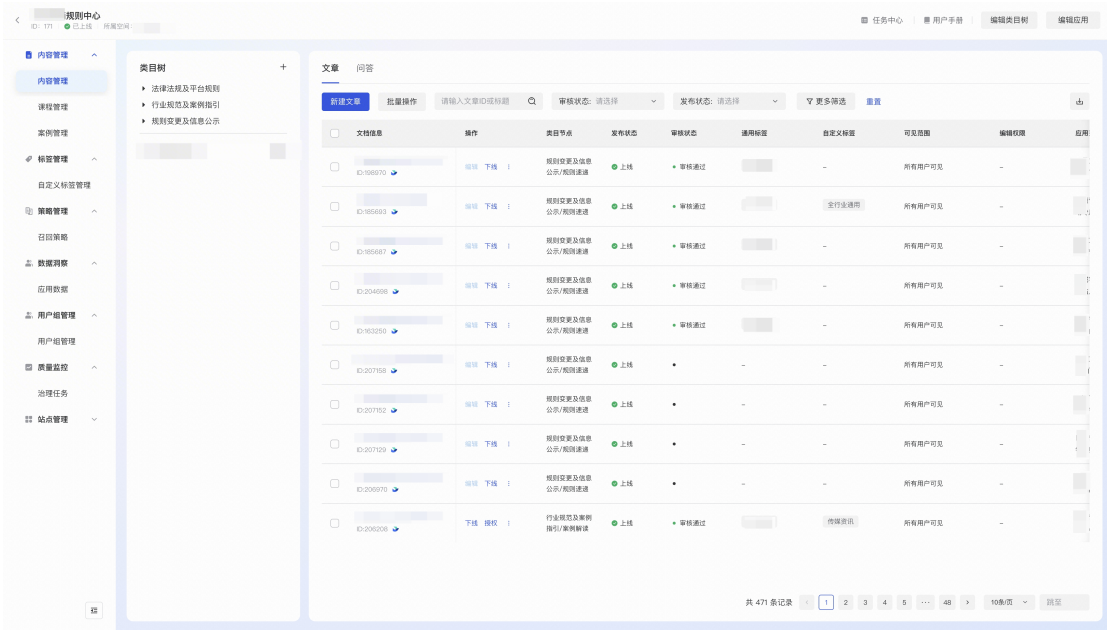


图 4-3: 文章列表效果图

图4-4展示了文章创编页的实际效果。该界面以正文编辑为核心，支持标题录入、富文本排版以及图片、表格等内容编排，整体交互形态与常见文本编辑页一致。通过在统一编辑界面中完成正文撰写与版式调整，平台能够降低内容创编门槛，并保证文章在生产阶段具备较好的可读性与结构化表达效果。

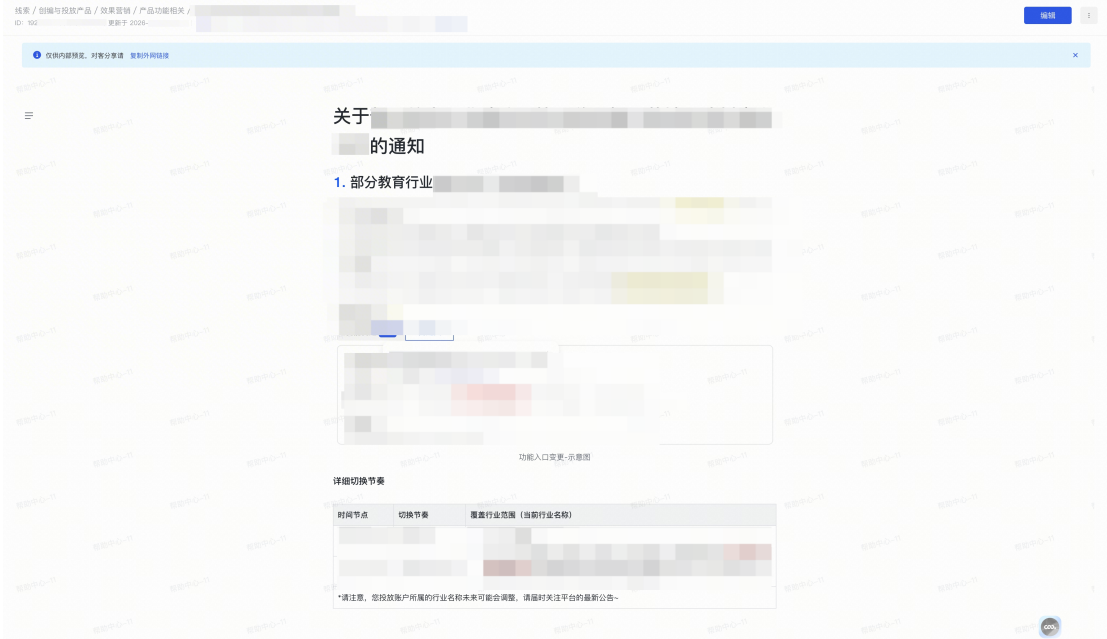


图 4-4: 文章创编效果图

4.1.3 权限与审批发布

内容管理模块同时面向管理后台与对外消费链路提供能力，因此权限体系需要同时覆盖“后台可操作范围”与“内容可见/可操作范围”。平台采用“RBAC + content_auth”的组合实现：RBAC 用于在空间与应用维度进行角色授权与模块级/操作级控制，content_auth 用于在内容粒度表达更细的策略约束，以支撑精细化可见范围以及合规要求。

(1) RBAC 权限控制实现。RBAC 以用户—角色—权限项映射为基础，角色在空间或应用维度生效，权限项覆盖空间/应用配置、内容创建编辑、删除、提审、审批、上下线与查看检索等后台操作。实现中，管理后台接口在处理请求时首先完成身份校验与 RBAC 权限校验，确保调用方具备对应操作能力，再进入后续对象级策略判断，避免越权调用对系统造成影响。对外消费链路以“内容可见性”为核心，主要依赖 content_auth 进行对象级裁决，不引入 RBAC 模块/操作级校验。

在鉴权决策上，平台将请求解析为 user_id + scope_type + scope_id + permission_code 四元组：scope 用于表达授权生效范围（GLOBAL/SPACE/APP），permission_code 用于表达具体操作。鉴权时先根据请求携带的 space_id、app_id 解析出作用域，再判断用户在该作用域下是否绑定了包含目标

权限码的角色，从而实现跨空间隔离与空间内精细授权。

为降低鉴权对管理后台高频访问的额外开销，平台在接入层对 RBAC 结果进行缓存：以 `user_id + scope_type + scope_id + permission_code` 作为缓存键，缓存值为鉴权是否通过及可选的角色信息。缓存更新遵循“写后失效”的原则：当空间负责人变更成员角色、或平台负责人调整角色-权限关系时，管理侧接口同步触发对应用户的缓存失效，保证权限变更能够在可接受延迟内生效。对于未命中缓存的请求，服务端从 `user_role_binding` 与 `role_permission` 表加载关系并完成裁决，再回填缓存。

(2) `content_auth` 策略授权实现。针对内容可见性与细粒度动作控制，平台以“内容 + 策略 + 变量 + 权限码”的模型表达策略：策略通过 `auth_tactic` 与 `auth_variable` 描述可见范围或动作约束，`auth_code` 表达具体权限项。鉴权决策流程在实现上以“策略加载—变量解析—动作匹配—结果裁决”为主线，并在读链路对召回候选进行二次过滤，保证检索与问答场景下不会返回越权结果。

在工程实现中，`content_auth` 的裁决遵循“动作优先”的原则：当请求为查看、编辑、删除、上下线等具体动作时，服务端先对目标内容加载其授权规则集合，并过滤出包含目标动作码的规则子集；随后按 `auth_tactic` 执行策略匹配。根据项目口径，`auth_code` 以精简集合表达动作权限，例如 `r/w/u` 分别对应“读/写/上下线”等能力，单条规则可携带多个动作码以减少规则数量。动作码与典型业务动作的映射如表 4-2 所示。

表 4-2: `content_auth` 动作码映射示例

动作码	典型动作
r	内容可见、详情读取、检索/问答召回结果可返回
w	内容编辑、删除、类目/标签绑定等写操作
u	上线/下线、提审/审批相关的发布控制动作

在策略匹配层，平台支持免登录可见、外部登录用户可见、内部员工可见、员工匹配、部门匹配、角色匹配与自定义表达式匹配等策略。一条内容可同时配置多条 `content_auth` 规则，且同一动作码（例如读权限 `r`）允许存在多条并行生效的规则（如“员工匹配”与“部门匹配”同时配置），命中任意一条规则即可视为授权通过。实现上，服务端先将访问者信息解析为可用于授权判断的“权限标签集合”（例如 `internal`、`employee:123`、`department:xxx`、`role:yyy` 等），再将 `content_auth` 记录映射为对应的标签匹配条件并执行集合包含判

断。这种设计解耦了规则配置与匹配逻辑，使得策略扩展无需修改核心校验代码。策略裁决的关键实现片段如图 4-5 所示。

```

1 public boolean checkContentAuth(long contentId, String actionCode,
2     Visitor v) {
3     Set<String> grants = buildGrants(v);
4     List<AuthRule> rules = authDao.listRules(contentId);
5     for (AuthRule r : rules) {
6         if (!r.authCode.contains(actionCode)) continue;
7         if (matchRule(r, grants, v)) return true;
8     }
9     return false;
10 }
11 private Set<String> buildGrants(Visitor v) {
12     Set<String> grants = new HashSet<>();
13     // 基础身份标签
14     if (v.userId == null) grants.add("anonymous");
15     else if (!v.isInternal) grants.add("external");
16     else grants.add("internal");
17
18     // 组织架构与角色标签
19     if (v.employeeId != null) grants.add("employee:" + v.employeeId);
20     for (String deptId : v.deptIds) grants.add("department:" + deptId);
21     for (String roleId : v.roleIds) grants.add("role:" + roleId);
22     return grants;
23 }
24
25 private boolean matchRule(AuthRule r, Set<String> grants, Visitor v) {
26     if ("custom".equals(r.authTactic)) {
27         return exprEngine.eval(r.authVariable, v.attrs);
28     }
29     // 统一构建规则Key: 无参策略直接匹配类型, 有参策略匹配 类型:值
30     String key = r.authVariable == null || r.authVariable.isEmpty()
31         ? r.authTactic
32         : r.authTactic + ":" + r.authVariable;
33     return grants.contains(key);
34 }
35

```

图 4-5: content_auth 策略匹配关键代码

为降低读链路开销，列表与检索场景均将权限校验逻辑前置至检索引擎（ES）：服务端在构建查询时，将访问者的权限集合（如 dept_id、role_id）转化为 ES 的 Term 或 Terms 条件，与索引中的 content_auth 字段进行匹配。这利用了 ES 对 BitSet 的高效处理能力，避免了传统模式下先召回大量 ID 再逐条鉴权带来的性能瓶颈，同时保证了分页结果的准确性。

（3）审批与发布流程实现。为保证内容对外可用前的准确性与合规性，平台将发布动作纳入流程化审批控制：提审接口会创建审批记录并将内容状态置为待审批；审批通过后更新内容状态为已发布，同时产生内容发布事件驱动

下游异步链路；审批驳回则回写拒绝原因并将内容退回可编辑状态。发布与下线接口会在保证幂等性的前提下触发事件，以驱动索引增量更新与统计回收，确保内容资产、索引与问答结果在发布前后保持一致。

审批服务以 `content_approval` 表承载审批单，核心字段包含 `content_id`、`submitter_id`、`reviewer_id` 与 `status`。提审时服务端校验内容必填字段、权限与当前状态，创建审批单并冻结内容进入待审批；审批通过与驳回均以审批单状态作为驱动依据，避免直接以内容状态作为唯一判定导致竞态。为避免重复点击造成的重复发布，审批通过接口在更新审批状态前检查是否已处于 `APPROVED`，若已通过则直接返回并不重复触发事件，从而在“至少一次调用”语义下保持幂等。

审批与发布链路的实现重点在于两点：其一是以状态机约束操作合法性，避免非法状态流转；其二是发布后通过事件驱动触发下游异步处理，使“发布版本”在索引与数据侧尽快达成一致。

图4-6展示了发布申请界面的实际效果。该界面将发布设置与审核设置串联为流程化步骤，用户需要补充已过PR、所在团队、申请人以及审批流预览等信息后方可提交发布。通过在申请阶段显式展示审批节点与流程路径，平台将“提审”从单一按钮动作扩展为可核验、可追踪的发布申请过程，从而与前述审批状态机和审批单模型形成一致的实现口径。

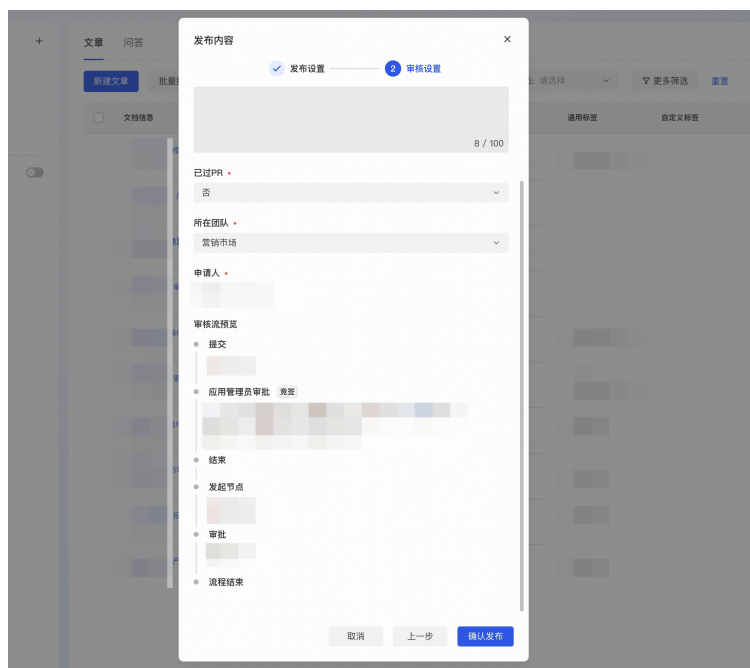


图 4-6: 发布申请效果图

关键实现片段如图 4-7 所示。

```
1 public void approve(ApproveReq req) {
2     authService.checkPermission(req.auditor, req.spaceId, "CONTENT_APPROVE"
3     );
4     Approval a = approvalDao.get(req.approvalId);
5     if (a.isApproved()) return;
6     if (!a.isPending()) {
7         throw new BizException("invalid approval status");
8     }
9     approvalDao.pass(req.approvalId, req.comment);
10    contentDao.updateStatus(a.getContentId(), "PUBLISHED");
11    eventBus.publish(new ContentPublishedEvent(a.getContentId(), a.
12        getSpaceId(), a.getAppId(), a.getId()));
13 }
```

图 4-7: 审批通过与发布事件触发关键代码

4.2 内容应用模块的实现

内容应用模块面向最终用户与业务运营，提供内容展示、内容搜索与智能问答等能力。与内容管理模块不同，内容应用链路强调高并发读性能、低延迟与可解释性：其一，所有对外可见结果必须以“发布版本”为输入口径；其二，检索与问答必须结合权限与范围配置做过滤，保证不引入越权风险；其三，生成式回答需提供证据引用，降低幻觉风险并支撑后续治理与优化^[2?]。

内容应用模块的关键服务端接口定义如表 4-3 所示。

表 4-3: 内容应用模块接口表

服务名称	接口名称	功能描述
页面化展示	listContents / getDetail	基于空间/应用/类目/标签组织内容列表与详情页。
内容检索	search	多字段关键词检索与结构化筛选，结果受范围与权限控制。
智能问答	ask	RAG 多通道召回证据并生成回答，返回可追溯引用。
索引构建	onContentPublished	消费发布事件，完成切片与全文/向量索引增量更新。

本节按“多形态内容索引 → 页面化内容应用 → 多通道召回 → 智能问答”的逻辑顺序展开，依次阐述内容应用模块在写入、读取、检索与生成四个关键环节的实现机制。

4.2.1 多形态内容前置处理与索引构建

为支持“页面化展示/搜索”与“RAG 召回”两大核心场景，平台构建了基于 ES 的全文索引与基于 Embedding 的向量索引（底层为 Milvus 向量数据库）^[2?]。该链路由统一的 MQ 消息（content_update/publish）驱动：当内容发生更新或发布时，索引服务消费消息，执行“数据同步 → 切片处理 → 双路索引”的流程，确保索引状态与数据库保持准实时一致。

（1）切片处理策略。切片是 RAG 检索的基础，其粒度直接影响召回的准确性与答案生成的质量。若切片过大，包含过多无关信息，会干扰模型生成；若切片过小，语义不完整，模型难以理解上下文。平台针对不同形态制定了差异化的切片策略：

1. **文章 (knowledge)**：以固定长度为基础切片，并尽量保持段落语义完整性，避免跨段切分导致上下文断裂；切片携带文章标题、摘要、类目、标签等元数据。
2. **问答 (faq)**：问题与答案作为完整语义单元不做切片处理，可分别入库并建立索引；召回阶段按场景组合使用（例如仅对 question 检索或对 question+answer 联合检索）。
3. **课程 (course)**：一个课节作为一个切片；考虑到成本，实现中通过大模型提取课节字幕，并通过大模型进行总结，作为课节内容。
4. **案例 (case)**：每个案例作为一个切片；切片携带案例标题、摘要、类目、标签等元数据。

此外，对于各种内容中存在的图片与表格类内容，在切片时需要特殊处理。针对图片内容，平台对比了“多模态 Embedding”与“视觉大模型（VLM）理解”两种方案。实测发现，多模态 Embedding 虽然支持图文同空间检索，但对图片内的细粒度文本信息（如海报文字、截图数据）敏感度不足。因此，平台最终采用视觉大模型理解方案：利用大模型对图片进行 OCR 与语义理解，生成详细的文本描述，将描述文本与图片链接一起作为切片入库。这种方式将图片检索降维为文本检索，显著提升了对图内文字的召回能力。针对表格内容，避免直接按字符切断，而是按行保持语义完整并携带表头信息，将其转化为 Markdown 格式入库，以提升表格类知识在问答中的可用性^[2?]。

在切片前的文本规范化阶段，系统会对原始 HTML 进行清洗：去除无意义的样式标签（如 style、span）与乱码噪声，同时保留关键的结构化标记

（如标题 `h1-h6`、列表 `li`、图片占位符）。保留这些结构信息有助于大模型理解上下文层级，并在生成答案时能够正确引用原文的结构。

在切片结构上，平台将每个切片建模为包含 `slice_id`、`content_id`、`space_id`、`app_id`、`content_type`、`text` 等字段的最小证据单元，并可携带 `position` 用于表示在正文中的定位信息（例如段落序号或课节序号）。该设计使得召回结果既能用于智能问答的证据输入，也能在输出侧映射为“内容标题 + 片段摘要 + 定位信息”的引用，降低用户在溯源时的跳转成本。

切片生成的工程实现可以拆分为“内容提取 → 文本规范化 → 按类型切片 → 补充元信息”四步。内容提取阶段按 `content_type` 从对应子表或文档库加载正文或结构字段；文本规范化阶段对富文本标记、空白与不可见字符做清洗，并统一标点与换行以减少 `embedding` 噪声；按类型切片阶段根据不同形态选择不同的语义单元；最后补充元信息阶段将标题、摘要、类目、标签、权限等字段写入切片结构，便于召回与过滤。切片构建的关键实现片段如图 4-8 所示。

```

1 public List<Slice> buildSlices(Content c) {
2     if ("knowledge".equals(c.type)) {
3         String body = normalize(knowledgeDao.getBody(c.id));
4         return splitByParagraphOrLen(c, body); // 段落优先，必要时按长度切分
5     } else if ("faq".equals(c.type)) {
6         Faq f = faqDao.get(c.id);
7         return buildFaqSlices(c, f.question, f.answer); // question/answer 两个语义单元
8     } else {
9         // 其他形态(course/case/image/table)按各自策略切分
10        ...
11    }
12 }
13

```

图 4-8: 切片构建关键代码

（2）索引同步与分发链路。索引构建服务作为 MQ 消费端，在收到变更消息后，首先回查数据库拉取最新的全量业务信息（含正文、权限规则、标签、类目等）。为保证切片与原文的一致性，系统采用“内容粒度的全量覆盖策略”。当内容发生变更并发布时，索引服务会触发该内容的完整重建流程：首先根据最新正文重新生成所有切片，随后在索引层执行“先删后写”操作，即清理该 `content_id` 下的所有旧切片数据，并批量写入新生成的切片。这种策略避免了复杂的增量比对逻辑，确保了索引中的切片始终与最新发布版本保持一致。

在完成切片生成后，系统基于“统一的切片策略”并行执行两条分发逻辑：

- **同步到 ES（文本倒排索引）**：将切片文本及其元数据（如 `content_id`, `auth`, `status` 等）写入 Elasticsearch。此索引利用倒排机制，侧重“精确关键词匹配”，同时服务于页面化内容应用与 RAG 文本召回。
- **同步到向量库（语义向量索引）**：对切片文本进行 Embedding 向量化，将向量及其元数据写入向量数据库。此索引利用向量相似度，侧重“语义理解”，专门服务于 RAG 语义召回。

(3) 全量元信息绑定与权限前置过滤。为实现高效的统一检索与权限控制，系统在构建索引时采用了“富切片（Rich Chunk）”模式：每个切片文档中不仅包含正文文本与向量，还全量冗余了所属内容的元数据，包括 `app_id`、`content_id`、`status`、`category_id`、`tag_ids` 以及用于权限控制的 `content_auth` 规则集合。

这一设计将复杂的关联查询转化为单文档的属性过滤，使得检索引擎能够独立承担所有筛选职责。在召回阶段，系统将过滤条件与查询词一同提交给检索引擎。这种前置过滤机制利用了搜索引擎的高效筛选能力，避免了传统“先召回后过滤”模式下因权限拦截导致的分页空洞与结果总数不准问题，显著提升了检索性能与用户体验。

(4) 关键实现片段。索引构建核心在于“事件消费 → 获取内容 → 切片 → 双索引写入”。其关键实现片段如图 4-9 所示。

```
1 public void onContentPublished(ContentPublishedEvent e) {  
2     Content c = contentDao.get(e.contentId);  
3     if (!c.isPublished()) return;  
4     if (indexMetaDao.exists(e.contentId, c.modifyTime, "FULLTEXT")) return;  
5     List<Slice> slices = sliceService.buildSlices(c);  
6     esClient.upsertDocuments(c.spaceId, c.appId, c.id, c.modifyTime, slices  
7         );  
8     vectorClient.upsertEmbeddings(c.spaceId, c.appId, c.id, c.modifyTime,  
9         slices);  
10    indexMetaDao.markDone(e.contentId, c.modifyTime);  
11 }
```

图 4-9: 索引构建关键代码

4.2.2 页面化内容应用机制

页面化内容应用机制用于支撑内容门户与管理后台的通用浏览与检索场景，包括目录导航、列表分页、详情查看以及关键词搜索。如图 4-10 所示，前

台页面上部提供统一搜索入口，支持用户通过关键词快速定位内容；页面主体区域按标签对文章、课程等内容进行聚合展示，便于用户在浏览中完成筛选与跳转。除图示布局外，平台亦支持在同一查询与渲染链路下切换不同样式与组织方式，此处不再展开。



图 4-10: 典型前台界面效果图

页面化内容应用机制的核心架构采用了“ES 统一索引过滤 + DB 实时详情回查”的读写分离模式。

(1) 统一索引与过滤。列表、类目、标签页与内容搜索页不再依赖数据库的复杂查询，而是优先查询 ES 宽表索引。服务端利用 ES 的 Filter 子句完成状态、类目、标签及权限的快速过滤与分页，获取目标内容 ID 集合。这种模式利用了倒排索引的高效性，避免了数据库全表扫描。对于搜索场景，在 Filter 基础上叠加 Match/MultiMatch 子句进行全文相关性算分与高亮处理。

(2) 权限前置过滤。权限控制直接参与倒排索引的交集运算。服务端将用户的权限特征（如 Dept/Role）转化为 ES 查询条件，与索引中的 content_auth 字段进行匹配，确保分页结果中的每一条记录都是用户有权访问的，解决了应用层后置过滤导致的分页空洞问题。

(3) 实时详情回查。获得 ID 列表后，服务端通过主键批量回查数据库（或缓存）获取最新的标题、摘要与状态信息。详情页则在完成对象级权限校验后，根据内容形态读取对应子表的正文或结构字段（如 knowledge 的 body、

course 的 lessons)。这种“索引圈选 +DB 兜底”的策略既保证了筛选性能，又确保了最终展示内容的强一致性。

页面化展示的关键实现片段如图 4-11 所示。

```
1 public Page<ContentItemDTO> listContents(ListReq req) {
2     Scope scope = scopeService.resolve(req.spaceId, req.appId, req.operator
3     );
4     if ("ADMIN".equals(req.source)) {
5         authService.checkPermission(req.operator, req.appId, "CONTENT_READ");
6     }
7     return contentQueryDao.queryPublishedVisible(scope, req.categoryId, req
8     .tags, req.page, req.size);
9 }
10
11 public ContentDetailDTO getDetail(DetailReq req) {
12     Scope scope = scopeService.resolve(req.spaceId, req.appId, req.operator
13     );
14     if ("ADMIN".equals(req.source)) {
15         authService.checkPermission(req.operator, req.appId, "CONTENT_READ");
16     }
17     Content c = contentDao.get(req.contentId);
18     authService.checkContentVisible(req.operator, scope, c.id);
19
20     // 按内容形态加载正文或结构字段
21     if ("knowledge".equals(c.type)) {
22         return renderService.renderKnowledge(c, knowledgeDao.getBody(c.id));
23     } else if ("faq".equals(c.type)) {
24         Faq f = faqDao.get(c.id);
25         return renderService.renderFaq(c, f.question, f.answer);
26     } else {
27         return renderService.renderExtension(c);
28     }
29 }
```

图 4-11: 页面化内容展示关键代码

4.2.3 多通道召回

多通道召回用于在智能问答场景下提供高质量证据集合。实现层面，平台采用“ES 全文检索 +Embedding 语义召回”的双路并行架构，融合去重后进行重排，最终输出与问题最相关的 TopN 切片。

(1) 双路召回机制。平台并行执行两条召回链路：

- **全文检索通道**：基于 ES 的 Match/MultiMatch 查询，利用 BM25 算法对 Title、Summary、Text 进行相关性算分。此通道优势在于精确关键词命中（如错误码、专有名词），且天然支持类目、标签与权限的 Filter 过滤。

● **向量召回通道**：基于 Embedding 模型将用户 Query 转化为向量，在向量数据库中检索 TopK 相似切片。此通道优势在于语义理解与跨词匹配（如“如何退款”匹配“退货流程”），且同样支持基于元数据（如权限、类目、标签）的 Filter 过滤，弥补了关键词匹配的不足。在工程实践中，稠密检索往往需与 BM25 插值以提升召回稳定性^[2]。

（2）融合与重排。两路召回结果在服务层进行加权融合与去重，保留得分最高的切片版本。融合后的候选集进入重排阶段（可选 Cross-Encoder 精排或规则排序），综合相关性分数、发布时间与点击热度等特征输出最终 TopN^[2]。

（3）关键参数与实现。召回阶段的关键参数包括 TopK 与 MinScore：TopK 用于控制返回候选切片数量，MinScore 用于设置最小相似度阈值以过滤低相关候选。表 4-4 给出了关键参数的工程含义。

表 4-4: RAG 召回关键参数

参数	含义
TopK	单通道召回的候选上限，用于控制延迟与后续重排开销
MinScore	最小相似度阈值，低于阈值的候选直接丢弃以提升精度与可解释性
TopN	重排后最终用于证据组织的切片数量，受上下文窗口与引用展示限制

重排实现采用“粗排 + 精排”的两阶段策略：粗排阶段以召回通道得分与轻量特征（标题命中、关键词覆盖）进行线性加权，快速裁剪候选集合；精排阶段对裁剪后的候选计算更细粒度特征，例如 query 与 slice 的语义相似度、与标题/摘要的覆盖度、以及同一 content 的多样性约束，并输出最终 topN 证据。重排的关键实现片段如图 4-12 所示。

```
1 public List<SliceHit> rank(String query, List<SliceHit>
   hits, int topn) {
2     for (SliceHit h : hits) {
3         double score = 0;
4         score += 0.5 * h.recallScore;
5         score += 0.3 * h.keywordCover;
6         score += 0.2 * h.titleHit;
7         h.finalScore = score;
8     }
9     List<SliceHit> sorted = sortByScore(hits);
10    return diversifyByContent(sorted, topn);
11 }
```

图 4-12: 重排关键代码

4.2.4 智能问答流程实现

智能问答流程将 RAG 检索得到的证据集合注入提示词中生成回答，并在输出侧附带引用信息，形成“问答—证据—治理”的闭环入口。实现目标是：在保证回答准确性的前提下，降低幻觉风险，并通过引用定位到具体内容片段以支撑后续治理。如图 4-13 所示为智能问答的效果图，包含模型回答、引用信息等。

(1) 问题改写。为了提升召回的准确性，在执行召回前，系统先调用 LLM 对用户原始 Query 进行改写。通过 Prompt 约束，模型需提取“对象、场景、诉求”三要素，并输出“完整表述”和更接近知识标题的“主要问题”。问题改写提示词模板如图 4-14 所示。

(2) 证据检索与组织。系统执行“改写后 Query → 双路召回 → 融合重排”的链路获取 TopN 切片。随后进行证据聚合：将重排后的切片按 Content 维度聚合，避免同一内容的多个片段重复占用 Prompt 空间，并为每条证据分配唯一 Key（如 S1、S2），便于模型引用。证据检索关键代码如图 4-15 所示。

(3) Prompt 构造与上下文控制。系统将用户问题、业务约束与聚合后的证据集合拼装为提示词。提示词结构包含：

- **系统约束**：禁止编造、必须基于证据回答、禁止偏袒长文。
- **任务描述**：规定 Markdown 输出格式与引用标记样式（如 [引用]<key1, key2>）。
- **证据集合**：填入聚合后的标准格式证据（含 Title、Text、Key）。

生成回答提示词模板如图 4-16 所示。

(4) 回答生成与引用解析。模型生成回答后，系统解析文本中的引用标

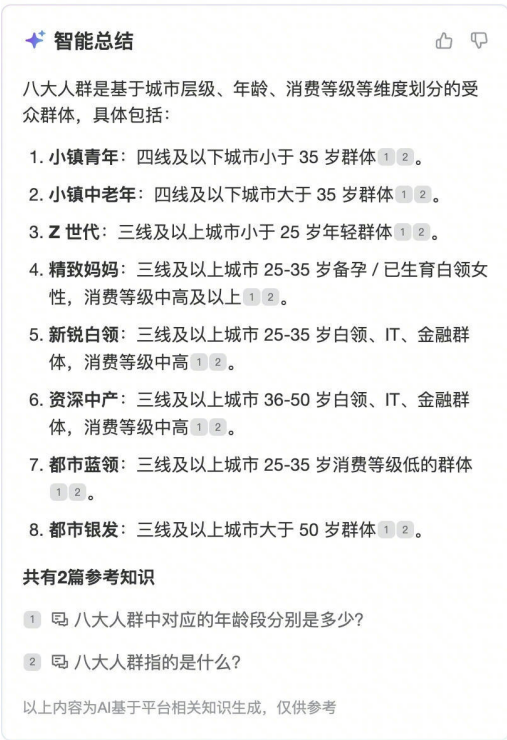


图 4-13: 智能问答效果图

记，通过 citationParser 将其映射回 slice_id，并进一步回溯到 content_id 与定位字段，最终生成前端可展示的“标题 + 摘要 + 跳转链接”引用列表。

（5）安全风控。仅依靠 Prompt 对模型输出进行约束并不可靠，系统引入了独立的安全风控服务对模型生成内容进行后置检查。该服务基于敏感词库与安全分类模型，拦截涉政、涉黄、暴力等违规内容；若风控校验未通过，系统将屏蔽生成结果并返回默认兜底文本，确保对外输出的合规性。智能问答关键代码如图 4-17 所示。

```

1  # 角色
2  你是商业化业务知识助手，负责将用户问题改写为更易检索的表达。
3
4  # 输入
5  <context>
6  {history_messages}
7  </context>
8  <query>
9  {raw_query}
10 </query>
11
12 # 定义
13 对象：问题围绕的主体/概念（名词为主）
14 场景：问题发生的背景/条件/状态（可包含多个限定）
15 诉求：用户希望获取的信息或完成的动作（是什么/为什么/怎么办等）
16
17 # 任务
18 1) 结合上下文抽取对象/场景/诉求三要素，若缺失可用上下文补全
19 2) 若query包含错别字或拼音，先修正再改写
20 3) 严格基于query与上下文，禁止引入未提及的新要素与操作
21 4) 生成两种表达：
22    - 完整表述：必须包含对象/场景/诉求，用于问答理解
23    - 主要问题：更接近知识库标题，更书面精炼，用于召回
24 5) 字数限制：每行不超过30字；句尾不使用任何标点
25 6) 输出格式必须严格为两行
26
27 # 输出格式
28 【完整表述】：{full_query}
29 【主要问题】：{main_question}
30

```

图 4-14: 问题改写提示词模板

```

1  public EvidencePack retrieveEvidence(AskReq req) {
2      Scope scope = scopeService.resolve(req.spaceId,
3          req.appId, req.operator);
4      authService.checkPermission(req.operator, req.appId,
5          "QA_READ");
6      String rewritten = rewriteService.rewrite(req.query);
7      String q = queryNorm.normalize(rewritten);
8      List<SliceHit> esHits = esClient.search(q, scope,
9          req.topkEs, req.minScoreEs);
10     List<SliceHit> vecHits = vectorClient.search(q, scope,
11         req.topkVec, req.minScoreVec);
12     List<SliceHit> merged =
13         mergeService.dedupAndMerge(esHits, vecHits);
14     List<SliceHit> reranked = rerankService.rank(q, merged,
15         req.topn);
16     return evidenceBuilder.build(reranked);
17 }

```

图 4-15: RAG 召回与重排关键代码


```

1 # system prompt (excerpt)
2 你是商业化业务知识问答专家，擅长解答广告/电商/商业化相关问题。
3 回答必须遵守：
4 1) 必须基于参考资料作答，禁止编造数据、结论与规则，禁止出现矛盾结论
5 2) 禁止偏袒篇幅较长或位置靠前的资料，需要综合判断相关性
6 3) 答案使用中文，markdown输出，结构清晰，字数不超过500字
7 4) 若答案使用了任意参考资料的知识，
8     必须在相关位置插入引用标记：
9     [引用]<key1,key2,...> (key来自证据集中的ID或序号，逗号分隔)
10 5) 禁止出现“文档/参考资料/根据xxx”等暴露资料来源的字眼
11 6) 参考资料可能包含图片占位：![desc](http...)
12     若使用图片信息可原样输出该图片
13 7) 若参考资料不足以回答，
14     给出保守回答并提示进一步确认或补充信息
15
16 # user prompt (template)
17 禁止创造公式！
18 <question>
19 {raw_query}
20 {rewritten_query}
21 </question>
22 <doc>
23 {evidence_documents_with_keys}
24 </doc>
25

```

图 4-16: 生成回答提示词模板

```

1 public AskResp ask(AskReq req) {
2     EvidencePack pack = retrieveEvidence(req);
3     if (pack.isEmpty()) {
4         coverService.recordMiss(req);
5         return AskResp.miss("未命中可用内容，建议补充知识");
6     }
7     String prompt = promptBuilder.build(req.query, pack);
8     String answer = llmClient.generate(prompt);
9     List<Citation> cites =
10         citationParser.parseOrFallback(answer, pack);
11     return AskResp.ok(answer, cites);
12 }

```

图 4-17: 智能问答关键代码

4.3 内容治理模块的实现

内容治理模块面向内容规模化带来的质量问题，构建“自动检测—任务化分发—人工处理—复检回收”的闭环机制。实现上，平台以重复度、歧义度与覆盖度三类问题为核心，将离线分析结果转化为可流转的线上治理任务，并与内容管理的修订与发布流程打通，保证治理处理后可触发索引更新与指标回收。

内容治理模块的关键服务端接口定义如表 4-5 所示。

表 4-5: 内容治理模块接口表

服务名称	接口名称	功能描述
任务管理	createMainTask / getMainTaskDetail	创建治理主任务并管理计算状态与配置。
检测计算	runDuplicateAmb / runCoverage	执行重复/歧义检测与覆盖度检测，产出任务候选。
任务流转	claimTask / submitResult / markDone	指派或认领任务，提交处理结论并更新状态。
复检回收	recheck / autoClose	对任务范围周期复检，自动关闭已消除问题。

治理任务线上化需要在存储层固化“主任务-子任务”的任务模型。主任务负责一次治理计算的配置与状态管理，子任务承载具体问题内容与处理结论。治理任务相关表的核心字段如表 4-6 所示。

表 4-6: 治理任务数据表（核心字段）

表名	关键字段（示例）	说明
govern_main_task	id, name, space_id, session_period, recall_threshold, status	治理主任务，承载会话周期、阈值等参数与计算状态。
govern_dup_amb_task	id, main_task_id, app_id, task_type, content_id_a, content_id_b, status, assignee_id, result	重复/歧义子任务，描述内容对、任务类型与处理结果。
govern_coverage_task	id, main_task_id, app_id, cluster_title, representative_query, query_count, status	覆盖度子任务，描述缺失知识簇的主题、代表性问法与热度。

在线治理闭环包含“计算生成 → 人工处理 → 系统巡检回收”三类动作：主任务创建后进入计算中状态，计算完成后生成子任务列表；业务人员在任务页面对重复/歧义任务标记处理结果（下线/保留/合并等），对覆盖任务标记是否需要补充内容并完成处理；系统侧每天定时巡检自上次巡检以来发生内容变更、上线、下线或删除的内容，复用 LLM 判别与召回评估对关联任务进行复检，若问题已消除则自动将子任务标记为完成，从而避免任务长期堆积造成统计失真。

4.3.1 重复度与歧义度检测算法实现

针对海量内容的重复与歧义检测，若采用两两比对的暴力算法，其复杂度为 $O(n^2)$ ，在工程上不可接受。因此，平台设计了“向量粗召回 + LLM 精判别”的两阶段检测算法，在保证检测精度的同时显著降低计算开销。

(1) 第一阶段：基于向量相似度的候选生成（Coarse Recall）。该阶段的目标是快速筛选出“可能存在问题”的内容对。算法利用向量索引的高效检索能力，对空间内的每一篇内容 C_i ，在向量空间中检索其 TopK 相似邻居集合 $\{N_1, N_2, \dots, N_k\}$ 。若 $Sim(C_i, N_j)$ 超过预设阈值（如 0.8），则将 (C_i, N_j) 标记为一个待判别候选对。这一步将检测范围从全量空间缩小到了高相似度的局部邻域。

(2) 第二阶段：基于大模型的语义判别（Fine-grained Judgment）。该阶段的目标是对候选对进行精确的语义裁决。系统将候选对 (C_i, N_j) 的标题与正文拼装入特定的判别提示词（Prompt），调用大语言模型进行逻辑判断：

- **重复度判别**：模型需对比两篇内容的主体与核心描述。若主体相同且描述高度重叠，判定为“重复”，并输出重叠比例。
- **歧义度判别**：模型需识别两篇内容是否描述同一业务主体但存在事实冲突（如对同一活动的起止时间描述不一致）。若存在逻辑矛盾，判定为“歧义”，并提取冲突摘要。

这种算法利用向量检索解决了规模化问题，利用大模型的语义理解能力解决了传统基于关键词（如 Jaccard 相似度）无法识别语义重复或逻辑冲突的难题。判别流程的关键代码与提示词模板分别如图 4-19 与图 4-18 所示。

重复度与歧义度判别的关键代码如图 4-19 所示。该实现负责装载两条内容的纯文本表示，基于提示词模板拼装输入并调用大模型生成结构化判别结果，最后将返回的 JSON 解析为统一的判别对象，以便后续任务聚合与状态流转复用。

(3) 结果聚合与任务生成。检测结果按内容对聚合，并与内容元信息、引用范围等字段拼装形成可流转的治理子任务。重复/歧义检测的关键实现片段如图 4-20 所示。

```

1 # 重复度判别 (template)
2 你的任务是判断两篇内容的重复程度,
3 分级为: 完全重复/极度重复/中度重复/轻微重复/不重复。
4 <title1>{title1}</title1>
5 <content1>{content1}</content1>
6 <title2>{title2}</title2>
7 <content2>{content2}</content2>
8 请按步骤判断:
9 1) 参考标题提炼两篇内容的主体对象与描述点
10 2) 判断主体是否一致; 若一致, 逐一对比描述点,
11    并估算重复占比 (取占比更高的一侧)
12 3) 按分级标准输出最终结论
13 输出必须为JSON (不要输出其他说明):
14 {"analysis": "主体与描述点对比、重复占比依据", "decision": "重复度分级"}
15
16 # 歧义度判别 (template)
17 你的任务是判断两组内容在“相同主体”下的描述是否存在冲突。
18 <business1>{business1}</business1>
19 <title1>{title1}</title1>
20 <content1>{content1}</content1>
21 <business2>{business2}</business2>
22 <title2>{title2}</title2>
23 <content2>{content2}</content2>
24 规则:
25 1) 先识别主体并判断是否为同一主体
26    (需结合业务场景、限定词、活动/行业等, 不可模糊联想)
27 2) 若无相同主体, 直接判定为一致
28 3) 若主体相同, 仅基于文本中“直接、明确、无歧义”的事实
29    判断是否冲突, 禁止推断
30 4) 冲突示例: 支持/不支持、时间点或覆盖范围不一致、
31    数值口径或公式组成不一致
32 5) 示例/URL/举例可忽略, 不作为冲突依据
33 输出必须为JSON (不要输出其他说明):
34 {"entity": "存在不一致的主体(无则为空)",
35  "consistency": "一致/不一致",
36  "conflicts": [{"description_A": "不一致描述A", "description_B": "不一致描述B"}],
37  "conflict_summary": "判定依据与命中的规则"}
38

```

图 4-18: 重复度与歧义度判别提示词模板

```

1 public LlmJudgeResp dupAmbJudge(Long a, Long b) {
2     String contentA = contentLoader.loadPlainText(a);
3     String contentB = contentLoader.loadPlainText(b);
4     String prompt = promptTpl.render(contentA, contentB);
5     String json = llmClient.generate(prompt);
6     return jsonParser.parse(json);
7 }

```

图 4-19: 重复度与歧义度判别关键代码

```
1 public void runDupAmb(MainTask task) {  
2     for (Long contentId :  
3         taskScopeDao.listContents(task.spaceId)) {  
4         List<Long> candidates =  
5             vectorClient.topKSimilar(contentId,  
6                 task.recallThreshold, task.topk);  
7         for (Long otherId : candidates) {  
8             LlmJudgeResp r = llmJudge.dupAmbJudge(contentId,  
9                 otherId);  
10            if (r.isDuplicate() || r.isAmbiguous()) {  
11                dupAmbTaskDao.upsert(task.id, task.appId,  
                    r.toTask(contentId, otherId));  
            }  
        }  
    }  
}
```

图 4-20: 重复度与歧义度检测关键代码

4.3.2 覆盖度检测实现

覆盖度检测旨在发现系统中缺失或薄弱的知识点。面对海量的用户会话日志，逐条进行覆盖度判别既不经济也无必要。因此，本项目设计了“意图筛选—会话聚类—代表性 Query 判别”的漏斗型处理路径，在大幅降低大模型计算开销的同时，确保治理任务聚焦于高频共性问题。

(1) 意图筛选与需求聚类。系统首先引入意图分类模型，对全量用户 Query 进行清洗，过滤掉闲聊与操作指令，仅保留“业务咨询”类 Query。随后，复用数据洞察模块的会话聚类能力（见 4.4.3 节），将语义相近的 Query 聚合为簇，并提取每个簇的代表性 Query 作为该需求的“指纹”。

(2) 基于代表性 Query 的覆盖度判别。针对筛选出的代表性 Query，系统执行“召回 + 判别”的标准化检测流程：

- **召回粗筛**：执行标准 RAG 检索，若 Top1 相似度低于阈值（如 0.6），直接判定为“召回缺失”。
- **LLM 精判**：对于相似度尚可但内容存疑的情况，调用大模型进行精细化评估。模型接收代表性 Query 与召回证据，判断现有知识是否能充分回答该问题。

覆盖度判别提示词模板如图 4-21 所示。

(3) 结果映射与任务生成。若代表性 Query 被判定为“未覆盖”，则整个聚

```
1 # 角色
2 你是内容覆盖度评估专家，
3 负责判断“目标知识点”是否被证据内容覆盖。
4
5 # 输入
6 <item>
7 {item_type}: {item_value}
8 </item>
9 <query>
10 {represent_query}
11 </query>
12 <docs>
13 {topk_evidence_with_keys_and_text}
14 </docs>
15
16 # 规则
17 1) 仅基于docs判断，禁止引入外部知识与推断
18 2) 若docs中能直接回答/定义/说明该item，则判定为覆盖
19 3) 若仅弱相关或无法支撑结论，则判定为未覆盖
20 4) 输出必须为JSON，便于程序解析落表
21
22 # 输出(JSON)
23 {
24   "covered": true/false,
25   "score": 0.0-1.0,
26   "evidence": ["key1", "key2"],
27   "reason": "覆盖/不覆盖的简要依据",
28   "suggestion": "若未覆盖，给出应补充的要点"
29 }
30
```

图 4-21: 覆盖度判别提示词模板

类簇被标记为“缺知识”状态。系统将该簇下的所有原始 Query 关联至同一治理任务，便于内容运营人员在处理时参考用户的真实问法，从而补充更精准的知识内容。

4.3.3 治理任务线上化与闭环机制

治理任务线上化模块负责将前述算法检测出的“问题候选”转化为可流转、可追踪的治理工单，并提供从“任务分发”到“人工处理”再到“效果复检”的全流程闭环能力。该模块将离线分析结果与在线内容生产链路打通，确保每一个治理动作（如补充知识、合并重复项）都能实时反哺到知识库中，从而实现质量的持续提升。如图 4-22 所示为治理任务线上化的效果图。

（1）任务模型与生成。治理任务采用“主任务（Batch）—子任务（Item）”的双层结构：主任务定义了单次治理的执行上下文（如空间范围、时间窗口、算法版本）；子任务则是具体的待处理实体。对于重复/歧义治理，子任务对应

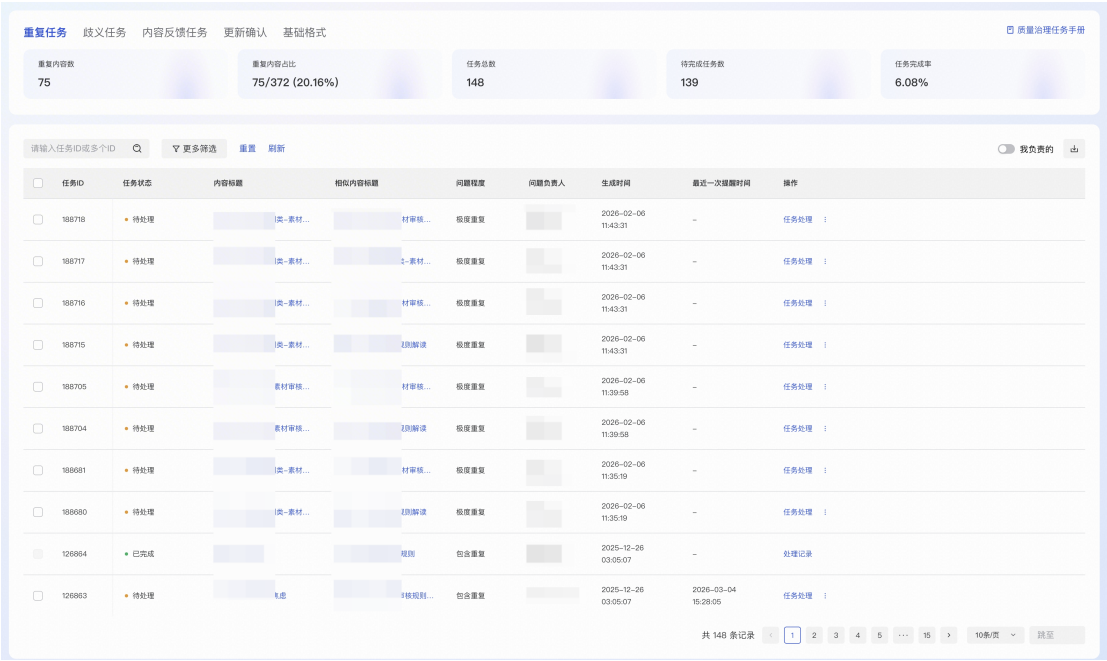


图 4-22: 治理任务线上化效果图

一个“待判别的内容对”；对于覆盖度治理，子任务对应一个“待补充的缺知识簇”。任务生成过程通过异步 Job 批量写入数据库，并采用幂等策略防止重复生成：系统以 (TaskType, EntityID) 为唯一键，确保同一问题在未关闭前不会重复创建新任务。

（2）任务分发与状态流转。子任务分发支持“指派”与“认领”两种方式：指派模式由空间负责人按业务域将任务分配到具体责任人；认领模式允许运营人员从任务池领取处理。任务处理动作与内容管理链路深度集成：重复/歧义任务的处理（下线/合并）直接触发内容的下线或修订；覆盖任务的处理（补充内容）触发新内容的创建或已有内容的更新。

任务状态机在实现上严格约束流转逻辑，确保并发安全。主任务状态包括“新建/计算中/计算完成”，子任务状态包括“待处理/处理中/已完成/已关闭”。表 4-7 给出了主任务与子任务的状态口径示例。

表 4-7: 治理任务状态口径示例

任务类型	状态	含义
主任务	NEW/RUNNING/ FINISHED	新建、计算中、计算完成
子任务	TODO/DOING/ DONE/CLOSED	待处理、处理中、已完成、已关闭（复检确认问题消除）

(3) 系统复检与任务回收。为确保治理动作真实有效，平台设计了“系统复检”闭环机制。对于已处理的任务，系统每日定时执行算法复检，仅当重复/歧义消除或覆盖率达标时才自动关闭任务，防止无效修复；针对算法误判或特殊业务场景，支持用户发起审批流程，通过人工审核后强制关闭任务。

复检回收的关键实现片段如图 4-23 所示。

```
1 public void submitTaskResult(Long taskId, String result) {
2     // 1. 更新人工处理结果
3     Task t = taskDao.get(taskId);
4     t.setResult(result);
5     t.setStatus("DONE"); //
6     // 标记为已处理，等待系统巡检或人工强制关闭
7     taskDao.update(t);
8 }
9
10 public void dailyRecheck(Date since) {
11     // 3. 定时巡检：处理任务流程外的变更
12     List<Long> changed =
13         contentDao.listChangedPublishedContents(since);
14     for (Long contentId : changed) {
15         for (DupAmbTask t :
16             dupAmbTaskDao.listByContent(contentId)) {
17             if (recheckService.checkDupAmb(t))
18                 dupAmbTaskDao.close(t.id);
19         }
20     }
21 }
```

图 4-23: 治理任务主动复检与巡检关键代码

4.4 数据洞察模块的实现

数据洞察模块面向业务与运营提供“可观测 + 可分析 + 可追溯”的闭环能力。整体实现上采用“前端埋点与服务端事件采集 → 数仓统一加工与指标口径沉淀 → 洞察服务查询与权限控制 → 前端可视化与下钻明细”的链路。数据加工与口径沉淀由数仓侧负责，本项目侧重点在事件规范、采集链路、与数仓的对接与查询服务建设。

数据洞察模块的关键服务端接口定义如表 4-8 所示。

表 4-8: 数据洞察模块接口表

服务名称	接口名称	功能描述
事件上报	reportEvent	统一事件模型上报与关键链路服务端埋点。
指标查询	queryPageMetrics / queryContentMetrics	按时间范围与维度查询页面/内容指标宽表。
会话分析	querySessionCluster / querySessionDetail	查询聚类指标并下钻会话明细。

4.4.1 内容应用数据采集机制

数据采集需要覆盖页面化消费链路 & 智能问答链路两类场景。在页面化链路中，采集曝光、点击、搜索输入与结果点击等行为数据；在问答链路中，采集问题输入、召回证据、生成结果与转人工等链路数据。平台对事件名与字段进行统一定义，例如page_view、click、query_submit、rag_recall、llm_generate、handoff_intent 与handoff_success 等，并统一携带app_id、space_id、session_id、trace_id 与content_id 等公共维度字段，便于端到端追踪与明细还原。

为保证事件口径可复用且便于扩展，平台将每条事件抽象为“公共维度 + 事件特有扩展字段”的统一结构：公共维度用于跨事件关联与链路追踪，扩展字段用于承载不同事件的特有信息（例如检索关键词、召回通道、候选数量等）。事件明细的核心字段设计如表 4-9 所示。

扩展字段 ext 用于承载不同事件的特有信息。以智能问答链路为例，rag_recall 事件会携带召回通道、TopK、MinScore、候选数量与最终证据集合摘要；llm_generate 事件会携带 prompt 长度、生成耗时、引用数量、是否触发退化等信息。表 4-10 给出了典型事件的 ext 字段示例，用于支撑后续“链路分析 → 问题定位 → 策略优化”的闭环。

采集链路上，前端埋点负责覆盖页面曝光、点击、搜索等用户侧行为；服务端埋点覆盖 RAG 召回、重排、生成与转人工等关键节点，保证链路关键事件不丢失。服务端在生成 trace_id 后将其贯穿写入召回与生成事件，使得洞察侧可以按 trace_id 还原单次问答的证据与决策链路。为避免重复上报导致的指标膨胀，平台在采集接入层对同一 (trace_id, event_name) 做幂等去重，并对异常流量进行限流保护，保证事件链路稳定。

表 4-9: 事件明细核心字段设计

字段名	类型	说明
event_name	string	事件名 (page_view/click/query_submit/rag_recall/llm_generate 等)
app_id / space_id	bigint	组织边界维度, 用于范围过滤与聚合
user_id / visitor_id	bigint/string	用户标识 (登录用户/匿名访客), 用于 UV 与分群分析
session_id	string	会话标识, 用于多轮对话与漏斗分析
trace_id	string	链路标识, 用于端到端追踪与去重
content_id	bigint	内容标识, 可为空 (例如曝光/点击未关联内容时空)
timestamp	datetime	事件发生时间
channel / device	string	渠道与设备维度, 可为空
ext	json	扩展字段 (query、召回通道、得分、引用列表等)

表 4-10: ext 字段示例 (典型事件)

事件名	ext 字段示例
rag_recall	query, rewritten_query, channels(es/vec), topk, min_score, hit_cnt, evidence_ids
llm_generate	model, prompt_len, latency_ms, token_out, citation_cnt, degraded(bool)
handoff_intent	reason, confidence, stage(recall/generate)

4.4.2 会话分析与效果指标设计

在指标体系设计上, 平台从页面、会话与内容三个视角承接数仓侧产出的聚合指标: 页面监控关注 PV/UV/点击率; 会话分析关注会话数量、模型识别解决率、意向转人工率与转人工率; 内容分析关注内容被召回次数、被生成次数以及转人工相关指标。洞察服务提供按时间范围、空间/应用、渠道与页面等维度的只读查询接口, 支持从聚合图表下钻到会话明细。

会话分析在实现上以 session_id 组织多轮交互, 并以 trace_id 贯穿单轮问答的召回、重排与生成等关键事件, 从而支持在洞察侧还原一次问答的证据与决策链路。聚合视图同时提供“会话级”和“聚类级”两种口径: 会话级用于观察整体趋势与转化漏斗, 聚类级用于定位高频问题簇并下钻到原始 Query 与引用证据, 形成“主题簇 → 明细会话 → 引用内容”的端到端分析路径。如

图 4-24所示，会话分析页面上部以图表汇总宏观指标，下部以列表呈现按聚类划分的会话指标，便于快速定位重点问题并进一步追溯证据来源。

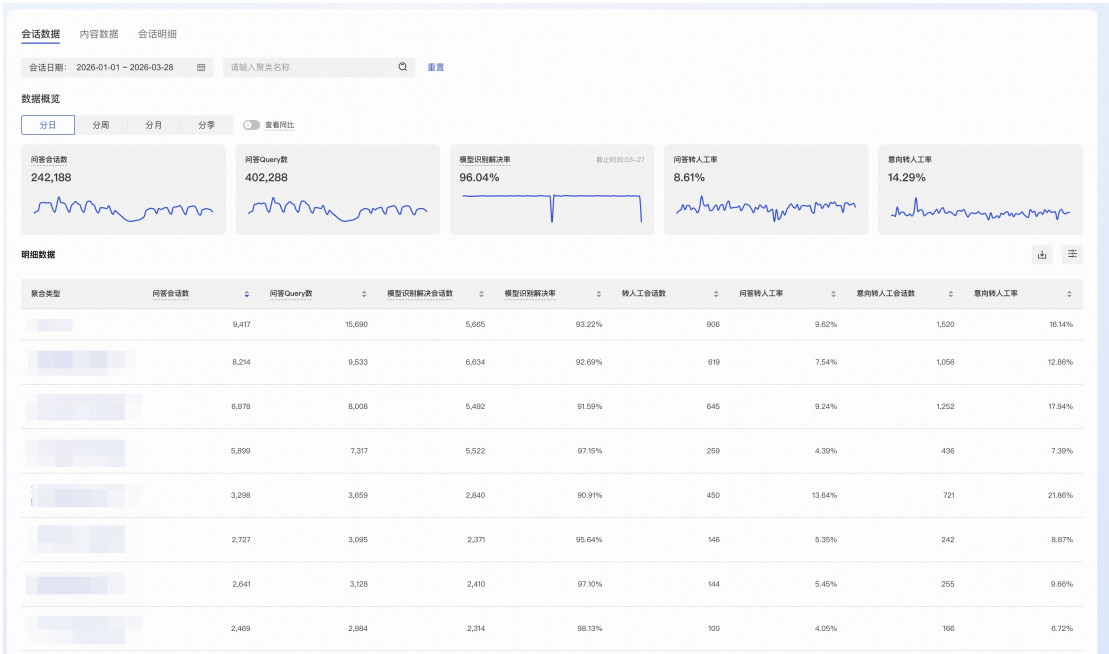


图 4-24: 会话分析效果图

4.4.3 会话聚类与主题总结

会话聚类用于将大量零散的历史 Query 归并为有限的语义主题簇，其核心动机在于“从统计视角识别高频问题与薄弱点”。单纯的会话列表难以暴露共性问题，而聚类可以将语义相似的 Query（如“怎么退款”、“退货流程”、“钱什么时候退回”）聚合为一个主题（如“售后退款咨询”），从而帮助运营快速发现高频诉求或未覆盖的知识盲区。

（1）聚类构建机制。聚类构建采用基于语义相似图的连通分量方法：将每条 Query 向量化写入向量索引，设定最小相似度阈值 `min_score`，相似度 $\geq$ 阈值的 Query 之间建立边；从未归类 Query 出发递归扩展其邻居集合，形成一个连通分量作为聚类。聚类构建的关键实现片段如图 4-25所示。

（2）主题总结。会话聚类完成后需要生成可读的主题标题与关键词，便于在洞察看板与治理覆盖任务中定位问题簇。平台在实现中使用大模型对同一聚类的代表性 Query 集合进行总结，输出摘要、关键词与代表 Query 列表，并将摘要作为 `cluster_title` 写入聚类指标宽表。聚类主题总结提示词模板如

```

1 public List<Cluster> buildClusters(List<QueryItem> items,
   double minScore) {
2     Set<String> visited = new HashSet<>();
3     List<Cluster> clusters = new ArrayList<>();
4     for (QueryItem q : items) {
5         if (visited.contains(q.id)) continue;
6         Queue<String> queue = new ArrayDeque<>();
7         Cluster c = new Cluster();
8         queue.add(q.id);
9         visited.add(q.id);
10        while (!queue.isEmpty()) {
11            String cur = queue.poll();
12            c.add(cur);
13            for (Neighbor n : vectorIndex.neighbors(cur,
14                minScore)) {
15                if (visited.add(n.id)) queue.add(n.id);
16            }
17        }
18        clusters.add(c);
19    }
20    return clusters;
21 }

```

图 4-25: 会话聚类关键代码

图 4-26 所示。

```

1 # 任务
2 你是文本总结专家。
3 给定一组语义相近的query，提炼共性并生成主题信息。
4
5 # 输入
6 句子用两个竖线分隔：
7 <sentences>
8 {q1 || q2 || q3 ...}
9 </sentences>
10
11 # 输出要求
12 1) summary: 30-50字主题摘要，直接给结论，
13    不以“这些句子...”开头
14 2) keyword: 5-7个关键词，避免过于宽泛的词，
15    需加具体场景限定
16 3) represent: 1-15条代表性query
17 4) 仅输出JSON，不要输出任何解释
18
19 # 输出 (JSON)
20 {"summary":"...", "keyword":["kw1", "kw2"], "represent":["q1", "q2"]}
21

```

图 4-26: 聚类主题总结提示词模板

指标查询接口在实现上遵循只读原则：洞察服务不直接参与指标计算，

仅对接数仓侧沉淀的宽表与明细表。为支撑高频查询，洞察服务对热点维度（例如近 7 天、近 30 天、单空间 TopN）使用缓存或物化视图降低延迟，并对查询接口增加时间范围与返回条数限制，避免明细下钻对后端与数仓造成过高压力。

4.5 本章小结

本章对内容管理、内容应用、内容治理与数据洞察四个模块的关键实现进行了说明。内容管理模块以统一内容模型与状态机约束实现多形态内容的规范化生产与发布口径控制，并通过权限机制支撑越权防护；内容应用模块在发布事件驱动下构建索引，提供页面化展示、检索与 **RAG** 问答能力，并在输出侧返回可映射的证据引用；内容治理模块将重复、歧义与覆盖不足等问题线上化为治理任务并与发布链路打通，形成可闭环的质量改进机制；数据洞察模块通过统一事件采集与指标查询支撑效果评估与问题定位，为后续实验评估与治理收益分析提供数据基础。

第五章 内容应用与管理平台测试

本章按“测试环境—单元测试—功能测试”的结构展开，对内容应用与管理平台进行测试说明。测试目标包括：验证平台核心功能满足第三章需求分析中定义的业务用例；验证关键技术链路（发布口径、权限过滤、事件驱动索引、RAG 问答与治理闭环）在典型场景下能够稳定运行；并通过单元测试降低回归成本，提升迭代效率。

5.1 系统测试环境

平台在实现上由内容管理、内容应用、内容治理与数据洞察等模块构成，依赖关系型数据库与文档数据库存储内容资产，并在内容发布后构建全文索引与向量索引以支撑检索与智能问答。由于论文工程无法直接提供企业内部真实部署细节，本节从“部署形态 + 关键依赖组件 + 测试工具链”三个维度描述测试环境口径，便于复现测试结论。平台测试环境描述如表 5-1 所示。

表 5-1: 测试环境描述表

项目	说明
部署形态	服务以容器化方式部署于 Linux 环境，按模块拆分并独立扩缩容。
开发环境	开发与调试环境为 macOS 或 Linux，使用浏览器与接口调试工具完成联调。
数据存储	关系型数据库用于内容元数据、权限与任务数据；文档数据库用于案例详情等灵活结构字段。
检索与向量索引	全文检索索引用于关键词召回与高亮展示；向量索引用于语义召回与长尾覆盖，二者共同支撑 RAG。
消息通知与异步链路	内容发布后通过消息队列触发索引增量更新与数据回收等异步任务。
外部能力	智能问答与治理判别依赖大模型服务，图文与图片类检索依赖多模态 embedding 能力。
测试工具	前端功能测试使用 Chrome；接口与链路测试使用 Postman 或同类 HTTP 工具；必要时结合日志与 trace_id 进行链路定位。

5.2 单元测试

单元测试用于在最小可控范围内验证模块内部逻辑正确性，并通过 Mock 隔离外部依赖以降低测试不稳定因素。平台的单元测试重点覆盖三类逻辑：第一类是权限裁决与状态机约束（例如 RBAC 与 content_auth 匹配、审批状态流转）；第二类是内容处理与索引构建的可重复性（例如切片构建规则、slice_id 稳定生成）；第三类是治理与洞察的关键算法单元（例如 TopK 候选生成、任务幂等 upsert 与复检回收）。

单元测试的基本原则如下：（1）边界清晰。对外部服务（数据库、索引服务、消息队列与大模型服务）通过 Mock 替身模拟输入输出，仅验证当前模块的业务逻辑与异常处理。（2）可重复执行。对时间、随机数与外部依赖进行固定化处理，避免测试随运行环境变化而不稳定。（3）覆盖关键分支。对成功路径、非法状态、权限拒绝、幂等重试与异常退化等关键分支分别构造用例，降低线上故障风险。

5.3 功能测试

功能测试用于验证平台对外可见能力与关键链路在真实交互下满足预期。本节基于第三章的核心用例集合，对内容管理、内容应用、内容治理与数据洞察模块设计功能测试用例。测试用例采用“步骤-预期结果”形式描述，覆盖正常流程、非法状态、权限约束、异步一致性与退化策略等场景。

5.3.1 内容管理功能测试

创建空间与应用是平台组织与权限的起点，用例用于验证空间隔离与应用边界能够正确生效。创建空间/应用的测试用例如表 5-2 所示。

内容生产链路覆盖创建、编辑、提审、审批与发布等过程，且要求读链路对外仅展示已发布内容。内容发布审批的测试用例如表 5-3 所示。

表 5-2: 创建空间/应用测试用例

用例编号	TC01
测试内容	创建空间与应用
测试步骤	1) 进入空间管理创建空间并填写负责人。 2) 在空间内创建应用并选择支持的内容形态与组织方式。 3) 进入成员与角色页面绑定默认角色。
预期结果	1) 空间与应用创建成功并可查询详情。 2) 默认角色初始化完成且创建人具备对应权限。 3) 不同空间的数据互相隔离。

表 5-3: 内容提审与审批发布测试用例

用例编号	TC02
测试内容	内容提审、审批通过/驳回与发布口径
测试步骤	1) 创建一条 knowledge 内容并保存草稿。 2) 发起提审进入待审批状态。 3) 审批人执行驳回并填写意见。 4) 再次提审并审批通过，内容进入已发布状态。 5) 在门户与检索侧分别访问该内容。
预期结果	1) 草稿状态不对外可见。 2) 驳回后内容回到可编辑状态并保留意见。 3) 审批通过后内容对外可见，门户与检索均仅返回已发布口径。

5.3.2 内容应用功能测试

内容检索用于验证多字段召回、高亮与过滤逻辑是否正确。内容检索测试用例如表 5-4 所示。

表 5-4: 内容检索测试用例

用例编号	TC04
测试内容	内容检索与高亮展示
测试步骤	1) 在应用内准备多条不同类型内容并发布。 2) 使用关键词分别命中 title 、 summary 与正文切片。 3) 设置类目/标签/类型过滤并分页查询。 4) 对不同权限用户重复检索。
预期结果	1) 关键词命中不同字段均可召回且排序合理。 2) 返回结果包含高亮片段与摘要。 3) 结构化过滤与分页生效。 4) 越权内容不会出现在检索结果中。

智能问答用于验证“问题改写 + 多通道召回 + 重排 + 引用输出”的端到端链路，以及未命中与超时场景下的退化策略。智能问答测试用例如表 5-5 所示。

表 5-5: 智能问答测试用例

用例编号	TC05
测试内容	RAG 智能问答端到端链路
测试步骤	1) 输入口语化问题触发问题改写。 2) 观察 ES 与向量两路召回候选数量与 MinScore 过滤。 3) 检查重排后 TopN 证据集合与 content 聚合情况。 4) 生成回答并检查引用映射到 content 与 slice。 5) 构造未命中与大模型超时场景。
预期结果	1) 改写后召回稳定性提升且不改变问题语义。 2) 证据集合与问题强相关并包含可追溯引用。 3) 未命中返回补充建议并进入覆盖度候选。 4) 超时退化为检索摘要返回，且权限过滤始终生效。

5.3.3 内容治理与数据洞察功能测试

重复/歧义治理用于验证候选召回、LLM 判别与任务生成的闭环过程。重复/歧义治理测试用例如表 5-6所示。

表 5-6: 重复/歧义治理测试用例

用例编号	TC06
测试内容	重复/歧义检测与子任务生成
测试步骤	1) 在空间内准备两条高度相似内容并发布。 2) 创建治理主任务并触发重复/歧义检测计算。 3) 查看生成的重复/歧义子任务与内容对。 4) 修改其中一条内容并重新发布，等待日巡检复检。
预期结果	1) 候选召回能够命中相似内容对并生成子任务。 2) 子任务幂等写入，不重复生成。 3) 内容修订后复检能够关闭已消除问题的子任务。

覆盖度治理用于验证“洞察聚类筛选薄弱主题 → 生成覆盖任务 → 补充内容 → 复检回收”的闭环。覆盖度治理测试用例如表 5-7所示。

数据洞察用于验证事件规范、trace_id 链路追踪与下钻能力。数据洞察测试用例如表 5-8所示。

表 5-7: 覆盖度治理测试用例

用例编号	TC07
测试内容	覆盖度任务生成与复检回收
测试步骤	1) 在问答链路制造未命中问题并沉淀为会话数据。 2) 在洞察侧聚类生成低解决率主题。 3) 在治理侧创建主任务并生成覆盖子任务。 4) 补充相关内容并发布。 5) 等待巡检复检回收。
预期结果	1) 覆盖任务能够以 <code>cluster_id</code> 或代表性 <code>query</code> 定位问题主题。 2) 内容补充发布后复检能够将覆盖任务自动关闭。 3) 任务状态与处理人记录完整可追溯。

表 5-8: 数据洞察链路测试用例

用例编号	TC08
测试内容	<code>trace_id</code> 链路追踪与会话下钻
测试步骤	1) 发起一次问答请求并记录 <code>trace_id</code> 。 2) 校验 <code>rag_recall</code> 与 <code>llm_generate</code> 等事件明细携带相同 <code>trace_id</code> 与 <code>session_id</code> 。 3) 在洞察侧按聚类主题下钻到会话明细。 4) 按 <code>trace_id</code> 还原证据集合与引用内容。
预期结果	1) 关键事件均被采集且字段口径一致。 2) 可按 <code>trace_id</code> 复现单次问答的召回与生成链路。 3) 聚类下钻可定位高频主题与薄弱点，为治理任务生成提供输入。

5.4 本章小结

本章对内容应用与管理平台的测试环境、单元测试与功能测试进行了说明。测试结果表明：平台在组织与权限边界、内容发布口径、事件驱动索引更新、RAG 问答引用可追溯、以及治理任务线上化闭环等关键链路上能够满足预期；数据洞察侧通过统一事件模型与 `trace_id` 关联实现链路还原与主题聚类下钻，为覆盖度治理与持续优化提供了数据基础。

第六章 总结与展望

6.1 总结

随着广告与商业化业务的快速发展，业务知识与运营规范等专业内容不断沉淀并快速演进，内容资产规模增长带来了内容分散、复用困难、维护成本高与质量不可控等问题。围绕上述问题，本文面向广告与商业化场景设计并实现了一套内容应用与管理平台，以内容全生命周期为主线，对内容建模、组织、应用与治理进行系统化建设，从而提升内容资产的统一管理能力、应用效果与质量可控性。

本文工作的主要贡献可概括为以下四个方面。

(1) 构建了基于空间与应用分层的内容组织模型，并实现多形态内容的统一管理能力。平台将空间作为业务隔离边界，将应用作为消费场景与配置边界，支持文章、问答、课程与案例等多种内容形态的规范化管理；通过类目树与标签体系完成内容组织，并以审批发布流程约束对外口径，保证内容在不同消费入口下的一致性。

(2) 实现了面向内容应用的检索与智能问答能力，并以发布口径与权限过滤贯穿读链路。平台在页面化展示与内容检索的基础上，引入 RAG 智能问答能力：通过内容切片、全文索引与向量索引的双通道检索，提高召回的准确性与覆盖率；通过问题改写、多通道召回、重排与证据组织生成高相关证据集合，并在输出侧返回可映射引用，降低幻觉风险并支持后续治理。

(3) 构建了以重复度、歧义度与覆盖度为核心的内容治理机制，并实现治理任务线上化闭环。平台结合向量检索与大语言模型完成对重复与歧义问题的自动检测与结构化判别，并将结果转化为可流转的治理任务；覆盖度治理复用洞察侧会话聚类结果定位薄弱主题并生成覆盖任务，结合内容补充、发布与复检回收形成闭环机制，使治理从一次性离线分析转化为可持续运营。

(4) 实现了数据洞察能力为应用与治理提供数据支撑。平台定义统一事件模型与关键链路埋点规范，通过 `session_id` 与 `trace_id` 实现端到端追踪与明细还原；洞察侧提供会话聚类下钻与指标查询能力，用于定位高频问题与薄弱

点，并为覆盖度治理与策略优化提供依据。

通过第五章的单元测试与功能测试验证，平台的组织与权限边界、审批发布口径、事件驱动索引更新、RAG 问答引用可追溯、治理任务线上化与复检回收等关键链路能够满足预期，为广告与商业化场景下的内容资产管理与应用提供了可落地的工程方案。

6.2 展望

虽然本文实现的平台能够覆盖内容管理、应用与治理的核心需求，但仍有进一步优化空间，主要体现在以下几个方面。

(1) 内容形态与结构化表达仍需增强。当前平台已支持文章、问答、课程与案例四类基础内容形态，并在切片阶段对图片与表格提供了特殊处理策略。后续可进一步引入多模态大模型（Multimodal LLM）对视频、音频内容进行深度理解与切片索引，并支持对更复杂的结构化知识（如规则流程图、指标定义表）的自动抽取与 Schema 化，使非文本内容也能被高质量地召回与问答。

(2) 索引与检索链路的成本与时延优化。平台采用事件驱动的增量索引构建机制，但在内容规模进一步增大时仍需更精细的分批、优先级与资源隔离策略。在 RAG 链路上，可探索基于强化学习（Reinforcement Learning）的 Prompt 自动优化机制，根据用户反馈动态调整 Prompt 结构与证据组织方式；同时引入检索策略自适应（Adaptive Retrieval），根据问题复杂度动态选择单路或双路召回，在降低时延的同时保持高召回率。

(3) 治理自动化与可操作性提升。重复与歧义治理目前以任务化分发与人工处理为主，后续可探索“建议式治理”，例如利用大模型自动生成合并后的权威内容候选、下线建议与引用调整方案，并将治理建议与审批发布流程更紧密地联动。此外，覆盖度治理可进一步与知识图谱结合，通过图谱补全技术发现潜在的知识缺失点，使覆盖分析既反映真实用户需求，也能对齐业务知识体系结构。

后续工作将围绕上述方向持续迭代平台能力，进一步提升内容资产的应用效率、问答体验与治理闭环效果，为广告与商业化业务提供更稳定、可控与可持续演进的内容基础设施。

版权及论文原创性说明

任何收存和保管本论文的单位和个人，未经作者本人授权，不得将本论文转借他人并复印、抄录、拍照或以任何方式传播，否则，引起有碍作者著作权益的问题，将可能承担法律责任。

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写的作品成果。本文所引用的重要文献，均已在文中以明确方式标明。本声明的法律结果由本人承担。

作者签名：_____

_____年____月____日

《学位论文出版授权书》

本人完全同意《中国优秀博硕士学位论文全文数据库出版章程》（以下简称“章程”），愿意将本人的学位论文提交“中国学术期刊（光盘版）电子杂志社”在《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》中全文发表。《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》可以以电子、网络及其他数字媒体形式公开出版，并同意编入《中国知识资源总库》，在《中国博硕士学位论文评价数据库》中使用和在互联网上传播，同意按“章程”规定享受相关权益。

作者签名：_____

_____年____月____日

论文题名	面向广告与商业化场景的内容应用与管理平台的设计与实现				
研究生学号		所在院系		学位年度	2026
论文级别	☐ 学术学位硕士 ☐ 专业学位硕士 ☐ 学术学位博士 ☐ 专业学位博士 (请在方框内画钩)				
作者 Email					
导师姓名					

论文涉密情况：

☐ 不保密

☐ 保密，保密期(_____年____月____日至 _____年____月____日)

注：请将该授权书填写后装订在学位论文最后一页（南大封面）。

